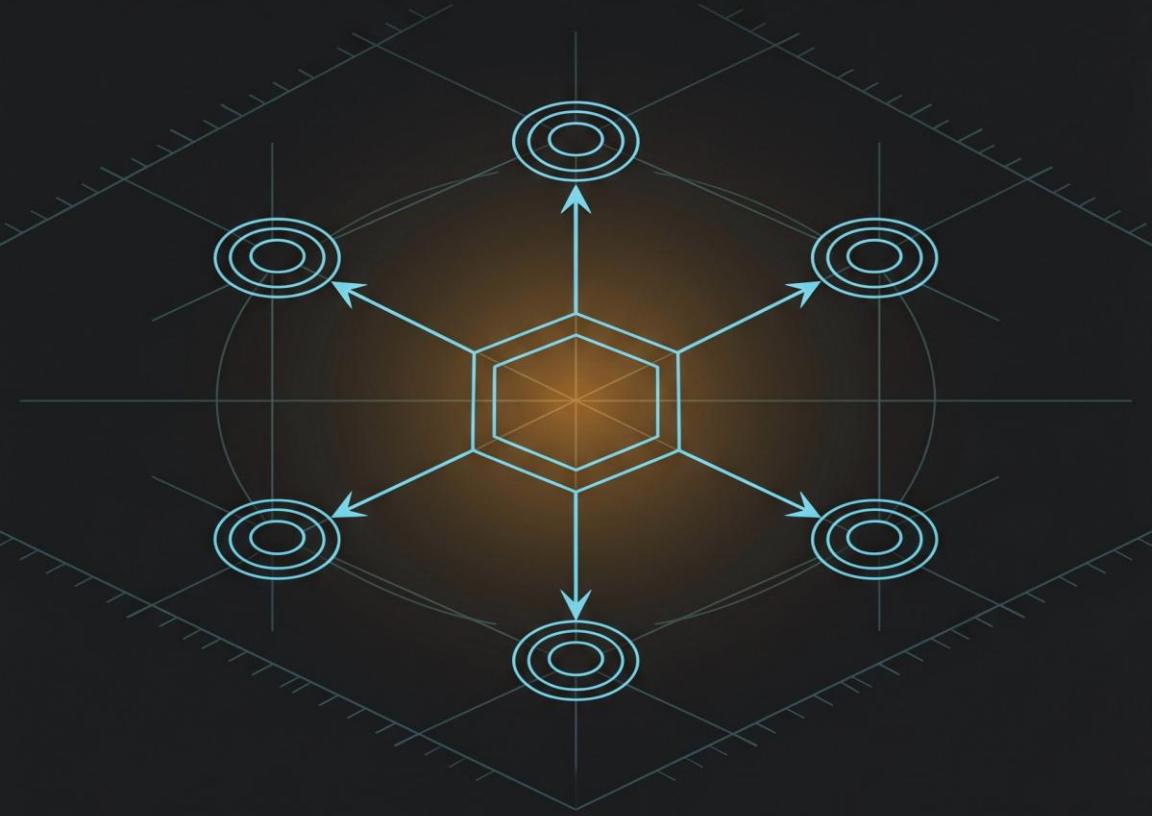
CLAUDE CERTIFIED ARCHITECT FOUNDATIONS

Patterns, Pitfalls, and Production Trade-offs for the CCA-F Exam



Written by Claude Code
Instructed by V.Korostyshevskiy

vladimir.org

CLAUDE CERTIFIED ARCHITECT **FOUNDATIONS**

Patterns, Pitfalls, and Production Trade-offs for the CCAR-F Exam

Written by Claude Code

Instructed by V.Korostyshevskiy

TABLE OF CONTENTS

Disclaimer

This book is an independent, unofficial study guide. It is not affiliated with, endorsed by, or sponsored by Anthropic.

Exam structure, scoring, administration details, and product and model specifications described in this book are current as of May 15, 2026 and are subject to change.

Readers should verify all exam-administration and product specifics against official Anthropic documentation before relying on them.

PREFACE

This book exists because **the CCAR-F exam is not a documentation quiz.**

If Anthropic wanted to test whether you had read the Claude docs, they would give you a search box and a time limit. They did not do that. The exam tests whether you can reason about architecture: which pattern fits which constraint, why one configuration choice forecloses another, where an agentic loop will break under load, and what you do when it does.

That is a different problem. And it requires a different kind of preparation.

What This Book Is

A practitioner reference for the Claude Certified Architect Foundations exam. Twelve chapters, each one mapped to one or more of the five exam domains. The structure follows the domain weighting, not alphabetical order, not a tour of the API surface. Every chapter addresses a class of architectural decisions that appear in the exam scenarios. Every chapter has an executive summary at the top and key takeaways at the bottom, because the way you read a book the first time is not the way you use it the week before the exam.

The material was synthesized from the official Claude documentation, the published CCAR-F exam guide, and the kinds of reasoning gaps that show up repeatedly when engineers first try to build production agentic systems. No invented statistics. No fictional company case studies. No hedging about things that are not yet public.

What This Book Is Not

Not a brain dump. Not a transcript of the docs reformatted into chapters. Not a collection of practice questions with answer keys that let you pass without understanding. If you want to memorize your way through a certification, this is the wrong book. The CCAR-F is specifically designed to defeat that strategy.

It is also not a gentle introduction to Claude. It assumes you have used the API, read at least some of the documentation, and have opinions about when an agent should escalate versus retry. If you are starting from zero, spend a few weeks building something first. Come back when the architecture questions feel real.

Who It Is For

Engineers preparing for the CCAR-F. Solution architects who work with Claude in production or who advise organizations on how to deploy it. Tech leads responsible for systems that use agentic patterns. Anyone who has looked at the exam guide and realized that domain weighting like “27% Agentic Architecture” requires more than skimming a few pages.

On the Voice

Sixty thousand words of passive-voice technical prose is a punishment. The Rands writing style, for readers unfamiliar, is the voice of an engineering leader who has been in enough rooms to have opinions and is tired of hedging them. Conversational but precise. Occasionally dry. It keeps the pages moving. The reference sections, the glossary, the coverage map, those are more straightforward, because clarity matters more than voice when you are looking something up at 11 PM the night before the exam.

There is no “good luck on your journey” at the end of this preface. You do not need luck. You need a clear model of how Claude’s architecture fits together and enough practice reasoning about trade-offs that the exam scenarios feel familiar. That is what this book is for.

HOW TO READ THIS BOOK

Structure Follows Domain Weighting

The CCAR-F exam has five domains. They are not equally weighted. Agentic Architecture is 27% of the exam. That means three chapters cover it. Context Management and Reliability is 15%. That means two chapters cover it. The chapter order reflects those priorities, not a logical tour of the API.

The domain-to-chapter mapping:

- Domain 1, Agentic Architecture (27%): Chapters 1, 2, 3
- Domain 2, Tool Design (18%): Chapters 4, 5
- Domain 3, Configuration and Customization (20%): Chapters 6, 7, 8
- Domain 4, Prompting and Output (20%): Chapters 9, 10
- Domain 5, Context Management and Reliability (15%): Chapters 11, 12

Each Chapter Is Self-Contained

Each chapter opens with an executive summary. Read the summary, decide if you need the detail, proceed accordingly. The summaries are not teasers. They are compressed statements of the chapter's architectural claims. A reader who only reads the summaries will have a weaker mental model but will not have been misled.

Chapters are designed to stand alone as reference material after a first linear read. Cross-references appear where a concept depends on something introduced elsewhere, but the chapter does not require you to have the other one open.

Navigation Aids

Every chapter has Key Takeaways at the bottom. These are the points most likely to surface in exam questions, stated plainly. They are not summaries of what was just covered. They are the architectural facts you need to carry forward.

Most chapters include sample questions. These are not full exam simulations. They are structured to produce the kind of reasoning the exam rewards: scenario plus constraint plus trade-off, not "*which API parameter does X.*"

Endnotes are collected in Appendix G, not inline. The text flags them by number. If you are reading for exam prep, you can ignore the endnotes entirely on first pass. If you want the source reference for a specific claim, Appendix G has it.

Code Examples

Code examples appear in Python and TypeScript. Both languages are represented across the chapters, roughly alternating. The examples are architectural illustrations, not production-ready snippets. They prioritize clarity about the pattern over completeness of error handling.

Suggested Reading Order

Read it linearly the first time. The concepts compound. A coordinator pattern in Chapter 2 makes more sense after the agentic loop in Chapter 1. The CLAUDE.md hierarchy in Chapter 6 is easier to reason about after you have seen hooks in Chapter 3.

After the first pass, use it as a reference. The executive summaries and Key Takeaways support that mode. So does the coverage map in the back matter, which maps exam task statements to specific chapters.

The glossary is at the back. If a term appears in the text and you are not sure of its precise meaning, that is where to look first.

ABOUT THE CERTIFICATION

What the CCAR-F Is

The Claude Certified Architect Foundations certification is Anthropic's foundational credential for engineers and architects working with Claude at a systems level. It validates the ability to design, configure, and reason about Claude-based systems across the full range of architectural concerns: agentic loops, tool ecosystems, configuration hierarchies, prompting strategies, and context management.

The CCAR-F is not a general AI literacy credential. It is specifically about Claude's architecture and the trade-offs that appear when you build production systems with it.

The Five Domains

The exam covers five weighted domains:

- **Domain 1, Agentic Architecture**: 27% of the exam. Covers agentic loop termination, multi-agent coordination patterns, hooks and lifecycle management.
- **Domain 2, Tool Design**: 18% of the exam. Covers tool description quality, tool selection mechanics, MCP integration, and the built-in toolkit.
- **Domain 3, Configuration and Customization**: 20% of the exam. Covers the CLAUDE.md hierarchy, slash commands, skills, plan mode, and CI/CD session isolation.
- **Domain 4, Prompting and Output**: 20% of the exam. Covers explicit prompting criteria, structured output contracts, and the validation loop.
- **Domain 5, Context Management and Reliability**: 15% of the exam. Covers context window discipline, the lost-in-the-middle effect, escalation triggers, provenance, and error propagation.

What the Exam Tests

The exam tests architectural reasoning, not recall. Questions are scenario-based. A scenario describes a system with specific constraints, then asks you to identify the correct architectural pattern, diagnose a failure mode, or choose between two plausible configurations and explain which trade-off is acceptable. Memorizing parameter names is not sufficient. Understanding why those parameters exist and what happens when you choose wrong is what the exam is after.

Administration

The CCAR-F is proctored through ProctorFree and administered through Skilljar. Candidates schedule and sit the exam through the Skilljar platform. ProctorFree handles the remote proctoring session.

The Six Recurring Scenarios

The exam draws four of these six scenarios at random for any given sitting. Each scenario recurs across the certification because they represent the canonical architectural problems that Claude-based systems encounter:

1. **Customer Support Resolution Agent, a multi-tool system with escalation logic**
2. **Code Generation with Claude Code, covering CI/CD integration and session isolation**
3. **Multi-Agent Research System**, using hub-and-spoke coordination with provenance requirements
4. **Structured Data Extraction, focused on structured output and validation**
5. **Developer Productivity with Claude, covering CLAUDE.md hierarchy and permission configuration**

6. **Claude Code for Continuous Integration, testing context window management and tool selection**

Familiarity with all six scenarios is practical preparation even though only four appear on any given exam. They recur because they represent the canonical architectural problems that Claude-based systems solve at scale.

Coverage in This Book

This book's chapter structure maps directly to domain weighting. Three chapters cover Domain 1, two cover Domain 2, three cover Domain 3, two cover Domain 4, and two cover Domain 5. The back matter includes a full coverage map that cross-references exam task statements to specific chapters.

CHAPTER 1: THE AGENTIC LOOP

Summary: *Every agentic system built on Claude reduces to the same primitive: a loop that runs until **stop_reason** signals termination. The model does not decide to stop. The **stop_reason** field in the API response drives termination logic in the orchestrator. This chapter establishes that mental model, then builds the complete operational picture: how turns accumulate into sessions, what each stop_reason value means and demands from your code, how **ResultMessage** subtypes convey termination state in the SDK, and which knobs (**max_turns, max_budget_usd, effort, permissionMode**) give the orchestrator governance over a running loop. The chapter closes by naming the two **anti-patterns** the exam tests repeatedly: parsing natural language output to detect task completion, and substituting iteration caps for proper stop_reason-driven termination logic. Both failures stem from the same misunderstanding. The model does not signal completion in prose. It signals it in a field.*

The Scene That Explains Everything

Two functions. One field. The contrast between them is the whole chapter.

```
# ANTI-PATTERN: parsing natural language
while True:
    response = get_response()
    text = response.content[0].text
    if "task complete" in text.lower():
        break
    if "I'm done" in text.lower():
        break

# CORRECT: check stop_reason field
while True:
    response = get_response()
```

```
if response.stop_reason == "end_turn":
    break  # Claude decided it's done
if response.stop_reason == "tool_use":
    execute_and_continue()
```

The anti-pattern is checking assistant text content to determine loop termination.[1] It feels reasonable until it isn't: the model rephrases its completion signal, your substring match misses it, and the loop spins indefinitely. Or the model produces "I've completed the task as requested" in a tool-call response that needs further processing, and your matcher fires early.

The correct pattern reads a typed field. **stop_reason** is part of every successful Messages API response.[2] It does not appear in the prose. It is structural. The model does not embed completion status in language. The API emits it as a discrete value. This distinction is the foundation of every agentic architecture decision discussed in this book, and it is the first thing the exam tests.[1]

Name this: **stop_reason-driven termination**. The orchestrator loop continues when stop_reason is "**tool_use**" and exits when **stop_reason** is anything else. The exit branch may require different handling depending on which value arrived. But the fundamental control structure is keyed on that field, not on parsed text.

The Loop at a Glance

Before the values, the shape. Every agent session follows the same cycle.[3]

1. **Receive prompt.** Claude receives the prompt, system prompt, tool definitions, and conversation history.
2. **Evaluate and respond.** Claude evaluates the current state and determines how to proceed. It may respond with text, request one or more tool calls, or both.

3. **Execute tools.** The SDK or your code runs each requested tool and collects results. Results feed back to Claude for the next decision.

4. **Repeat.** Steps 2 and 3 cycle. Each full cycle is one turn. Claude continues until it produces a response with no tool calls.

5. **Return result.** The loop ends. The SDK or your code delivers the final output.

A turn is one complete round trip: Claude produces output that includes tool calls, those tools execute, results return to Claude. Turns continue until Claude produces output with no tool calls, at which point the loop ends.[3]

The SDK handles this cycle automatically. A complex task ("refactor the authentication module and update the tests") can chain dozens of tool calls across many turns, with the model adjusting its approach based on each result, without your orchestration code doing anything between turns.[3] This automation is the value. The loop manages itself. What you configure is its boundaries and permissions.

The stop_reason Values

Seven values. Each demands a different response from your code.[2]

"end_turn"

The most common value. Claude finished its response naturally. The conversation can continue with a new user message, or you can take the final output and move on. This is the happy path.[2]

One edge case: sometimes the model returns an empty response with stop_reason: "end_turn" and no content. This typically occurs after tool results, when Claude interprets the assistant turn as already complete. The documented cause is mixing

text content into the same user message as a tool_result block.[2] The fix is addressed in the next section on tool_result placement.

"tool_use"

Claude is requesting one or more tool calls and expects your code to execute them. The response contains tool_use blocks with the tool name and a JSON object of arguments. Your application extracts those arguments, runs the operation, and sends the output back in a tool_result block on the next request.[4] The loop continues.

This value drives the core loop continuation logic. While stop_reason == "tool_use", execute tools and continue. On any other value, exit (or handle the specific condition).[2]

"max_tokens"

Claude reached the max_tokens limit specified in the request. The response is truncated. If the truncated response contains an incomplete tool_use block, a retry with a higher max_tokens value is required to get the full tool call.[2] Your code must handle this as a boundary condition, not as task completion.

"stop_sequence"

Claude encountered one of your custom stop sequences. The response is complete up to that point. Useful for protocol-based conversations where delimiters signal state transitions.[2]

"pause_turn"

Returned when the server-side sampling loop reaches its iteration limit while executing server tools like web search or web fetch. The default limit is 10 iterations per request.[2] The work is not finished. Re-send the conversation, including the paused response, to let Claude continue where it left off. Any agent loop using server tools should handle this value.[4]

"refusal"

Claude declined to generate a response due to safety considerations. Your application should handle this case explicitly rather than treating it as a generic error.[2]

"model_context_window_exceeded"

Claude stopped because the model's context window limit was reached. This value is available by default in Sonnet 4.5 and newer models; earlier models require a beta header to enable it.[2] The response is valid but limited by the context window, not by max_tokens. Knowing which ceiling was hit matters for the recovery strategy.

A complete dispatcher looks like this:

```
def handle_response(response):
    if response.stop_reason == "tool_use":
        return handle_tool_use(response)
    elif response.stop_reason == "max_tokens":
        return handle_truncation(response)
    elif response.stop_reason == "model_context_window_exceeded":
        return handle_context_limit(response)
    elif response.stop_reason == "pause_turn":
        return handle_pause(response)
    elif response.stop_reason == "refusal":
        return handle_refusal(response)
    else:
        # end_turn and stop_sequence land here
        return response.content[0].text
```

The same logic in TypeScript, for the SDK layer:

```
async function runLoop(prompt: string): Promise<string> {
  for await (const message of query({ prompt })) {
    if (message.type === "result") {
```

```
    if (message.subtype === "success") {
      return message.result;
    }
    throw new Error(`Loop ended: ${message.subtype}`);
  }
 }
 throw new Error("Stream ended without result");
}
```

The TypeScript SDK exposes termination state through ResultMessage.subtype, which provides a different vocabulary from the Messages API's stop_reason. The two layers are related but distinct. The next section addresses the SDK layer specifically.

ResultMessage and SDK Termination State

When using the Agent SDK, the loop's termination state arrives in a **ResultMessage**, the final message emitted when the loop ends. The **subtype** field is the primary way to check termination state.[3]

Five subtypes:

Subtype	Meaning	result field available?
"success"	Claude finished the task normally	Yes
"error_max_turns"	Hit the maxTurns limit before finishing	No
"error_max_budget_usd"	Hit the maxBudgetUsd limit before finishing	No
"error_during_execution"	An error interrupted the loop (API failure, cancelled request)	No
"error_max_structured_output_retries"	Structured output validation failed after configured retry limit	No

The result field (final text output) is only present on the "success" subtype. Always check the subtype before reading it.[3] All result subtypes carry **total_cost_usd, usage, num_turns, and session_id** so cost tracking and session resumption remain available even after an error.

A small note the docs flag explicitly: trailing system events such as **prompt_suggestion** can arrive after the ResultMessage. Iterate the stream to completion rather than breaking on the result.[3]

The ResultMessage also includes a **stop_reason** field (*str | None in Python, string | null in TypeScript*) indicating why the model stopped generating on its final turn. On error subtypes, stop_reason carries the value from the last assistant response before the loop ended. The two fields serve complementary purposes: subtype tells you whether the SDK loop succeeded or failed; stop_reason tells you what the model was doing when it ended.

The *tool_result* Block Placement Rule

This is a short rule with a sharp consequence. After each tool call, the result must be appended to conversation history as a tool_result block in a new user-role message. Do not mix text content into the same user message as a tool_result block.[2]

The anti-pattern:

```
# INCORRECT: Adding text immediately after tool_result
{
  "role": "user",
  "content": [
    {"type": "tool_result", "tool_use_id": "toolu_123", "content": "6912"},
    {"type": "text", "text": "Here's the result"},  # Don't do this
  ],
}
```

The correct form:

```
# CORRECT: Send tool results directly without additional text
{
  "role": "user",
  "content": [
    {"type": "tool_result", "tool_use_id": "toolu_123", "content": "6912"}
  ],
} # Just the tool_result, no additional text
```

Mixing text into the same block causes Claude to interpret the assistant turn as already complete, producing the empty end_turn response described earlier.[2] The SDK handles this correctly when you use its built-in tool execution. If you are driving the Messages API loop manually, this rule is a necessary discipline.

Loop Control: Turns, Budget, and Effort

Three options govern how long a loop runs and how deeply it reasons. All three are fields on the options object passed to the SDK *query()* function.[3]

MAX_TURNS AND MAX_BUDGET_USD

max_turns (Python: max_turns; TypeScript: maxTurns) caps the number of tool-use round trips. It counts tool-use turns only. Without a limit, the loop runs until Claude finishes on its own, which is fine for well-scoped tasks but can run long on open-ended prompts.[3] When the limit is reached, the SDK emits a ResultMessage with subtype *"error_max_turns".*

max_budget_usd (Python: max_budget_usd; TypeScript: maxBudgetUsd) caps accumulated cost. When the spend threshold is reached, the loop stops and emits *"error_max_budget_usd".* Setting a budget is a practical default for production agents.[3]

Here is the anti-pattern this pair corrects: using an iteration cap as the primary stopping mechanism. The reasoning is superficially similar to *max_turns,* but the intent is different. An iteration cap substitutes for proper stop_reason-driven control. The agent loop should exit because the model signals completion via *stop_reason,* not because your orchestrator counted to ten. *max_turns* is a safety net, not the primary exit condition.[1]

The exam tests this distinction.

THE EFFORT OPTION

Effort controls how much reasoning Claude applies per turn. Lower effort levels use fewer tokens per turn and reduce cost.[3]

Level	Behavior	Good for
"low"	Minimal reasoning, fast responses	File lookups, listing directories
"medium"	Balanced reasoning	Routine edits, standard tasks
"high"	Thorough analysis	Refactors, debugging
"xhigh"	Extended reasoning depth	Coding and agentic tasks; recommended on Opus 4.7
"max"	Maximum reasoning depth	Multi-step problems requiring deep analysis

The Python SDK leaves effort unset when not specified, deferring to the model's default behavior. The TypeScript SDK defaults to "high".[3]

Extended thinking is a separate feature from effort. They are independent: effort: "low" with extended thinking enabled is a valid configuration, as is effort: "max" without it.[3]

Effort can be set at the session level in query() options, or per subagent via the effort field on AgentDefinition to override the session level. The use of subagents as a composition pattern is covered in Chapter 2 (Coordinator plus subagents).

Permission Mode

The permissionMode option (Python: permission_mode; TypeScript: permissionMode) controls whether the agent asks for approval before using tools that are not covered by explicit allow or deny rules.[3]

Four modes:

"default"

Tools not covered by allow rules trigger the approval callback. No callback means deny. This is the **interactive mode: a human-in-the-loop configuration where the agent surfaces ambiguous actions for review.**[3]

"acceptEdits"

Auto-approves file edits and common filesystem commands (*mkdir, touch, mv, cp,* and similar). Other Bash commands follow default rules. The practical choice for autonomous agents on a development machine where file modification is expected but arbitrary shell execution is not.[3]

"plan"

Read-only tools run. Claude explores and produces a plan without editing source files. The agent can read, search, and analyze, but write operations are blocked. Use this when you want Claude's analysis and recommendations before committing to changes. This value is applied in plan mode workflows covered in Chapter 7 (Explicit-intent execution modes).

"bypassPermissions"

Never prompts. **All tool calls execute without approval regardless of other settings.** Reserve this for CI environments, containers, or other isolated environments where no human is available for approval.[3] This value is the **recommended configuration for CI/CD integration**, covered in Chapter 8 (Review-session isolation).

Note that *permissionMode*: "acceptEdits" does not auto-approve MCP tools; permissionMode: "bypassPermissions" does, because it disables all safety prompts.[3] Chapter 5 covers the specific implication for MCP tool permissions.

What the Loop Looks Like End-to-End

Here is the complete Python loop with all the concepts from this chapter integrated. This is the pattern the exam expects you to recognize and evaluate.

```
import anthropic
from claude_agent_sdk import query, ResultMessage

client = anthropic.Anthropic()
tools = [{"name": "lookup_customer", "description": "...", "input_schema": {}}]
messages = [{"role": "user", "content": "Find customer John Smith"}]

# The Agentic Loop
while True:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=4096,
        tools=tools,
        messages=messages
    )

    # KEY: Check stop_reason to control the loop
    if response.stop_reason == "end_turn":
        break  # Claude is done

    if response.stop_reason == "tool_use":
        tool_block = next(
            b for b in response.content if b.type == "tool_use"
        )
        result = execute_tool(tool_block.name, tool_block.input)
```

```
        messages.append({"role": "assistant", "content": response.content})
        messages.append({
            "role": "user",
            "content": [{"type": "tool_result",
                         "tool_use_id": tool_block.id,
                         "content": result}]
        })
```

The equivalent TypeScript loop using the Agent SDK:

```
import { query, ResultMessage } from "@anthropic-ai/claude-agent-sdk";

async function runAgent(prompt: string): Promise<void> {
  for await (const message of query({
    prompt,
    options: {
      maxTurns: 20,
      maxBudgetUsd: 5.0,
      permissionMode: "acceptEdits",
      effort: "high",
    },
  })) {
    if (message.type === "assistant") {
      // Handle progress: see which tools Claude called this turn
    } else if (message.type === "result") {
      const result = message as ResultMessage;
      if (result.subtype === "success") {
        console.log(result.result);
      } else {
        console.error(`Loop ended: ${result.subtype}`);
      }
```

```
    break;
   }
  }
}
```

Three things to observe in both versions. First: loop continuation is keyed on stop_reason (Messages API) or subtype (SDK). Second: tool results are returned as isolated tool_result blocks in new user-role messages. Third: the loop does not inspect the text of Claude's responses to decide what to do next. The control flow is structural, not semantic.

The Two Anti-Patterns, Named

The exam names these explicitly and tests them repeatedly. They have distinct mechanics, but both originate from the same misunderstanding: the belief that the model communicates completion intent through the content of its text output.

ANTI-PATTERN 1: PARSING NATURAL LANGUAGE FOR COMPLETION SIGNALS

```
# This will eventually fail
if "task complete" in response.content[0].text.lower():
    break
if "I'm done" in text.lower():
    break
```

The model does not guarantee consistent phrasing. Any phrase you match, Claude will eventually rephrase. Worse: Claude might produce completion-sounding language mid-task, in a response that also contains tool calls. The structural stop_reason field is the reliable signal. It is a typed enum value, not a substring in prose.[1,2]

ANTI-PATTERN 2: ARBITRARY ITERATION CAPS AS PRIMARY CONTROL

```
max_iterations = 10
for i in range(max_iterations):
    response = get_response()
    # process...
```

Iteration caps are reasonable safety rails. They are not loop termination logic. A cap that fires before the model signals completion interrupts a working session. A cap set high enough to never interfere provides no safety value. The correct pattern uses stop_reason for primary control and max_turns (or max_budget_usd) as boundary conditions: the loop exits normally on "end_turn" and receives a ResultMessage with "error_max_turns" only if the task ran unexpectedly long.[1,3]

Both anti-patterns appear as exam distractors. They look reasonable to candidates with incomplete knowledge of the API. Knowing why they fail is the knowledge the exam is measuring.

The Architecture Implication

The loop is not Claude's loop. The model generates responses; the orchestrator runs the loop.

This is a meaningful distinction for system design. Claude does not execute tools. It emits a structured request describing what tool to call with what arguments. Your application (or the SDK on your behalf) executes the tool and returns the result. Claude never sees your implementation; it only sees the schema you provided and the result you returned.[4]

That execution model means the **orchestrator owns the termination logic.** Claude cannot break out of your loop. It cannot decide the session is over in any way that

bypasses your code. The stop_reason field is Claude's communication channel for expressing completion intent. Your code reads it and acts.

This separation is why the two anti-patterns are architectural failures, not just coding mistakes. Parsing text for completion signals outsources loop control logic to the model's prose style. That is the wrong layer. The stop_reason field exists precisely to give the orchestrator a reliable, structured, parseable signal.

Everything else in this book builds on that separation: hooks that intercept tool calls in the orchestrator layer (Chapter 3), multi-agent systems where one orchestrator drives subagent loops (Chapter 2), context management that shapes what Claude sees on each turn (Chapter 11). The primitives are all correct loop mechanics at the foundation.

Key Takeaways

- **stop_reason-driven termination** is the fundamental control pattern: continue on *"tool_use", exit on "end_turn", handle "max_tokens", "pause_turn", "refusal", "stop_sequence", and "model_context_window_exceeded"* as distinct conditions.
- The **ResultMessage.subtype** field in the Agent SDK provides termination state at the session level: "success" carries the final text output; the four error subtypes do not include a result field.
- **tool_result** blocks belong in isolated user-role messages; mixing text content into the same block causes empty end_turn responses.
- **max_turns** and **max_budget_usd** are safety bounds, not primary loop control; using an iteration cap as the main exit mechanism is a named anti-pattern.

- The **effort** option controls reasoning depth per turn across five levels; the **TypeScript SDK defaults to "*high*", the Python SDK leaves it *unset*.**
- The four permissionMode values ***("default", "acceptEdits", "plan", "bypassPermissions"***) map to four distinct trust contexts: interactive human approval, autonomous dev machine operation, read-only planning, and CI/container execution.
- **The model does not execute tools and cannot break out of the orchestrator's loop; stop_reason is the only structural channel for signaling completion intent.**

CHAPTER 2: HUB-AND-SPOKE

Summary: *A single agentic loop handles a surprising range of problems. Then it hits a wall. The task decomposes. Different subtasks need different tools, different context windows, or different permission boundaries.* ***The coordinator-plus-subagents pattern*** *is the documented answer to that wall.*

This chapter establishes the hub-and-spoke topology: a coordinator agent that owns decomposition, delegation, and result aggregation, surrounded by specialized subagents that each execute one focused job in an isolated context. It covers the ***Agent tool*** *(previously Task) that makes subagent invocation work, the* ***AgentDefinition*** *fields that configure each subagent, and the precise mechanics of what context a subagent inherits and does not inherit from its parent. It also names and dissects the two failure modes the exam reliably tests: overly narrow decomposition creating gaps between agents, and the coordinator that routes every query through every subagent instead of selecting dynamically.*

The multi-agent research system from Scenario 3 of the exam guide serves as the recurring lens throughout.

The Moment Decomposition Happens

The scenario is specific.[1] A coordinator agent receives a research topic. It needs to search the web, analyze source documents, synthesize findings, and generate a formatted report. Four distinct jobs. Different tool sets for each. Document analysis benefits from deep focus on a narrow set of sources without the noise of raw web results. Synthesis needs the outputs of both prior stages but must not re-run the searches itself. Report generation needs only the synthesis, clean.

One agent doing all four jobs in a single loop accumulates context from every step. By the time synthesis runs, the context window holds raw search results, document

excerpts, intermediate analysis notes, tool call logs, and the original query. Most of that is noise for the synthesis task. The model attends to all of it anyway.

You cannot unsee context once it is there. The only way to keep synthesis clean is to ensure it starts with a clean context. The only way to do that is a subagent.

This is the moment decomposition happens. Not when the task is complex. When different stages of the same task would contaminate each other if they shared a context.

The Hub-and-Spoke Topology

The pattern has a name: **Coordinator plus subagents**.[1,2] The coordinator is the hub. Subagents are the spokes. All communication between subagents routes through the coordinator. Subagents do not talk to each other directly. They do not share state. They do not see each other's outputs unless the coordinator explicitly passes those outputs in a subsequent delegation prompt.

This topology gives the coordinator complete observability. Every result, every failure, every partial finding passes through one place. The coordinator can evaluate synthesis quality and re-delegate to search or analysis subagents with targeted follow-up queries before invoking synthesis again. It can handle a subagent failure without the other agents knowing anything went wrong.[2]

The flat alternative (agents sharing a global state or full conversation history) removes that control surface. When one agent's noisy output pollutes another agent's context, the errors are hard to isolate and harder to reproduce.

The coordinator carries four responsibilities that belong to it alone and to nothing else in the system.[2]

Decomposition. The coordinator reads the incoming query and decides how to split the work. It identifies which subtasks are independent (can run in parallel), which are sequential (stage B needs stage A's output), and whether any subtasks are

unnecessary for this particular query. A query about a well-documented public API might not need document analysis at all. The coordinator figures that out before delegating anything.

Delegation. The coordinator dispatches subtasks to the appropriate subagents. It writes the prompt string for each invocation, deciding exactly what context and instructions to include. This is the most consequential decision the coordinator makes: too little context and the subagent works blind; too much context and you are back to the noise problem the subagent was supposed to solve.

Result aggregation. The coordinator receives all subagent outputs and assembles them into a coherent result. In the research system, this means taking structured findings from multiple analysis passes and passing them to the synthesis subagent in a form that synthesis can use. The coordinator does not just concatenate outputs. It curates them.

Dynamic subagent selection. The coordinator decides which subagents to invoke based on the query's actual requirements, not on a fixed pipeline sequence. This is not optional behavior. It is the mechanism by which the hub-and-spoke topology delivers any advantage over a fixed sequential chain.

The coordinator's decomposition itself follows one of two patterns, and the exam tests the selection rule between them.

Fixed sequential pipeline (prompt chaining). The stages are known before execution starts. A code-review pipeline that analyzes each file individually, then runs a cross-file integration pass for dependency and data-flow issues, is a fixed pipeline: the number of stages, their order, and their inputs are determined by the PR's file list, not by intermediate findings. Use this when the work is predictable and multi-aspect (multiple files, multiple review dimensions) but the decomposition itself does not depend on what any single stage discovers.

Dynamic adaptive decomposition. The subtasks emerge from findings. A legacy-codebase investigation that starts by mapping the directory structure, identifies high-impact areas from that map, creates a prioritized analysis plan, then adapts as

dependency relationships surface is a dynamic decomposition: the second stage cannot be specified until the first stage reports, because what to investigate depends on what exists. Use this when the work is open-ended and the structure of the problem is itself unknown at the start.

The selection rule: if you can enumerate the stages before the first tool call fires, use a fixed pipeline. If the stages depend on intermediate results, use dynamic decomposition. The coordinator's dynamic subagent selection (choosing which agents to invoke per query) is the runtime expression of the second pattern. The "map structure first, then prioritize, then adapt" sequence that appears in the exam's open-ended investigation scenarios is the canonical worked example of the adaptive arm.

(Chapter 9's intervention classifier uses the structural-versus-recognition distinction to separate decomposition fixes from few-shot fixes when the same domain surfaces in an exam stem.)

The Agent Tool

Subagents are invoked via the Agent tool.[3] For a coordinator to spawn subagents, "Agent" must appear in its allowedTools. Without it, the coordinator has no mechanism to delegate.

One version note matters for production code and for the exam. The tool was renamed from "Task" to "Agent" in Claude Code v2.1.63. Current SDK releases emit "Agent" in tool_use blocks, but still use "Task" in the system:init tools list and in result.permission_denials[].tool_name. Checking both values in block.name ensures compatibility across SDK versions.[3]

When you see an exam question about why a coordinator is not spawning subagents, the first thing to verify is whether "Agent" (or "Task", depending on SDK version) appears in allowedTools. It is the most common single-field oversight.

AgentDefinition: What You Configure

Each subagent is described by an **AgentDefinition object** passed in the agents parameter of your query() call. Two fields are required.[3]

description is a natural language statement of when to use this agent. Claude reads it and decides whether to invoke the agent. Write it the way you would write a tool description: specific, bounded, clear about what this agent handles versus what it does not. The coordinator selects subagents based on descriptions. A vague description produces unpredictable selection.

prompt is the subagent's system prompt. This is where the agent's specialized instructions live: its role, its constraints, its output format requirements, its tool-use preferences. The more focused this prompt is on the subagent's specific job, the better.

The optional fields give you per-agent configuration:[3]

- **tools**: array of allowed tool names. If omitted, the subagent inherits all tools available to the coordinator. Specify this to restrict the subagent to only the tools it needs.
- **disallowedTools**: tools to remove from the agent's set, useful when inheriting by default but wanting to block specific capabilities.
- **model**: override the model for this agent. Accepts aliases (*'sonnet', 'opus', 'haiku', 'inherit'*) or a full model ID. Use a lighter model for a fast web-search pass; use a heavier model for the synthesis step that requires careful reasoning.
- **skills**: skill names to preload into the agent's context at startup.
- **memory**: memory source for this agent ('user', 'project', or 'local').
- **mcpServers**: MCP servers available to this agent, by name or inline configuration. gd

- **maxTurns**: maximum agentic turns before the agent stops.
- **background**: run this agent as a non-blocking background task.
- **effort**: reasoning effort level ('low', 'medium', 'high', 'xhigh', 'max', or a number).
- **permissionMode**: permission mode for tool execution within this agent. (The values are defined in Chapter 1.)

The research system example makes the model field concrete. The web-search subagent runs fast and wide; you might use a lighter, faster model and accept some reduction in analysis depth. The synthesis subagent integrates findings from multiple sources and needs to reason carefully; that is a case for a more capable model. The coordinator pays the cost of an extra API call either way. You get to tune that tradeoff per agent.

What a Subagent Inherits (and What It Does Not)

This is the most exam-tested mechanical detail in the chapter. Memorize both sides of the table.[3]

A subagent receives:

- Its own system prompt (AgentDefinition.prompt)
- The Agent tool's prompt string (the task description passed at invocation time)
- The project *CLAUDE.md* (loaded via settingSources)
- Its defined tool set (inherited from parent, or the subset specified in tools)

A subagent does not receive:

- The parent's conversation history
- The parent's tool results
- The parent's system prompt
- Any skill content not listed in AgentDefinition.skills

The only channel from coordinator to subagent is the Agent tool's prompt string.[3] If the synthesis subagent needs the web search results, the coordinator must pass those results explicitly in the prompt it sends to the synthesis agent. There is no automatic inheritance. There is no shared memory. There is no side channel.

This constraint is a feature, not a limitation. It is what makes context isolation work. The synthesis subagent starts with exactly the context the coordinator decided it should have: the distilled outputs of the search and analysis stages, formatted and targeted for synthesis. No raw tool call logs. No intermediate reasoning artifacts. No irrelevant prior turns.

The research system walkthrough from the exam scenarios puts it directly: pass only the context relevant to each subagent's specific task. Sharing the full coordinator conversation history wastes tokens and introduces irrelevant information into the subagent's reasoning.[1]

Context Passing: The Rule and Why It Exists

The rule is simple to state. **Pass only context specific to each subagent's task.**[1,2] The rationale is more interesting.

A coordinator running a multi-topic research job may have, by the time it invokes synthesis, a context that includes: the original query, a planning pass, web search results for three distinct subtopics, document excerpts from seven sources,

intermediate analysis notes for each subtopic, and several rounds of quality checks. That is a realistic coordinator context for a serious research workflow.

The synthesis subagent does not need the planning pass. It does not need the raw search queries. It does not need the quality check history. It needs the analysis outputs: structured findings with source attribution, organized for synthesis.

Passing the full coordinator history to the synthesis agent would mean the synthesis model attends to the planning pass and the raw search queries alongside the analysis outputs. Those earlier elements do not help synthesis. They add noise. They consume tokens that could go to the actual findings. And they create confusing signal: the synthesis agent might repeat reasoning the coordinator already completed, or hedge against questions that were already resolved upstream.

The synthesis agent does not need to know how the search was done. It needs to know what it found.

The structured data approach makes context passing precise. When passing outputs from one subagent to the next, separate content from metadata. Findings go in one field. Source URLs, document names, and page numbers go in another. This preserves attribution through synthesis without embedding source metadata inline in the findings text, where it disrupts reading and confuses the model's summarization logic.[2]

Parallel Execution

Multiple subagents can run concurrently.[3] The mechanism is straightforward: emit multiple Agent tool calls in a single coordinator response. The SDK runs them in parallel. The coordinator receives all results before continuing.

The research system has a natural parallel phase. Web search and document analysis can run simultaneously. Neither depends on the other's output. Synthesis

depends on both. The coordinator emits Agent tool calls for the search subagent and the analysis subagent in the same response turn; both run in parallel; the coordinator receives both results; then it invokes synthesis with both outputs as input.

The contrast is sequential invocation, where the coordinator emits one Agent call, waits for the result, then emits the next. Sequential invocation is correct when stage B genuinely depends on stage A's output. For stages that are independent, parallel invocation reduces end-to-end latency proportionally to the number of agents running in parallel.

The exam question on this topic tests whether you know the mechanism. Parallel subagents require multiple Agent tool calls in a single coordinator response, not multiple calls across separate turns.[2]

The background field in AgentDefinition extends this further. Setting background: true runs a subagent as a non-blocking background task. The coordinator does not wait for the background subagent's result before continuing its own execution. This is useful when a long-running analysis can proceed while the coordinator handles other coordination work, as long as the coordinator does not need that subagent's output for the next immediate step. The coordinator can retrieve the result later when it is ready to proceed to synthesis.[3]

The detection side of parallel execution matters for orchestration code that needs to track which subagents have been invoked. Subagents are invoked via the Agent tool, so detecting subagent invocation means checking for tool_use blocks where name is "Agent". Messages originating from within a subagent's context include a parent_tool_use_id field that links back to the parent's invocation. In Python, content blocks are accessed directly via message.content. In TypeScript, SDKAssistantMessage wraps the Claude API message, so content is accessed via message.message.content.[3]

The No-Nesting Rule

Subagents cannot spawn their own subagents. The Agent tool must not be in a subagent's tools array.[3]

The architecture is flat at depth one. The coordinator is the only agent that delegates. Subagents execute and return results. If you need a subagent to do something that itself requires a subagent, the correct design is to have the coordinator decompose that work into two separate subagent invocations and manage the dependency explicitly.

This constraint matters for exam questions that present a scenario where a deep-research subagent is given the Agent tool so it can spawn its own search agents. The answer is: do not do this. The coordinator owns delegation. Giving delegation capability to a subagent removes the observability that makes hub-and-spoke valuable in the first place.

Coordinator Prompt Design

The exam guide's task statements are precise about what a coordinator prompt should contain.[2] It specifies **research goals and quality criteria**. It does not enumerate step-by-step procedural instructions.

The distinction matters operationally. A coordinator prompt that says "*first call the web-search subagent, then call the document-analysis subagent, then call synthesis*" is a static pipeline. It runs the same sequence regardless of the query. A simple factual question gets the full pipeline including a document analysis pass that contributes nothing. A query where the web search returns no useful results still routes to synthesis with empty inputs.

A coordinator prompt that specifies goals and quality criteria lets the coordinator decide which subagents to invoke based on what the query actually requires. For a simple factual question, the coordinator might invoke only the web-search

subagent and skip document analysis entirely. For a query where preliminary synthesis reveals gaps, the coordinator re-delegates to search and analysis with targeted follow-up queries before re-invoking synthesis.[2]

Dynamic selection is the coordinator's core capability. The coordinator does not route all queries through all subagents. It reads the query, reads the description fields of available subagents, and selects the subagents the query requires.

Two Anti-Patterns the Exam Tests

Both anti-patterns have names in the registry, and both come from the exam guide's task statements.[2]

Overly narrow task decomposition creating coverage gaps. The research system has four subagents. If the coordinator assigns each agent a scope so narrow that some aspects of a broad research topic fall between agents, findings have holes. The web-search agent covers recent publications. The document-analysis agent covers the provided source list. If the query asks about a topic where relevant findings exist in both sources that cross-reference each other, and neither agent is configured to notice the cross-reference pattern, the synthesis agent never sees the connection.

Narrow decomposition is sometimes presented as a virtue (focus, specialization). It becomes an anti-pattern when the task boundaries do not align with the natural structure of the work. The fix is to design agent scopes around the shape of the information, not around a desire for symmetry in the agent count.

Routing all queries through all subagents. The opposite failure. The coordinator always runs the full pipeline regardless of what the query requires. A question that only needs web search runs document analysis anyway. A question that is already well-covered by preliminary synthesis triggers another full search-and-analysis

pass. This wastes tokens, increases latency, and can introduce noise when subagents return results for tasks that are not relevant to the query.

The coordinator's selection logic is not overhead. It is the mechanism by which the hub-and-spoke topology delivers its efficiency.

Both anti-patterns share a root cause: the coordinator is not reasoning about the query. It is executing a fixed procedure. The whole point of putting an LLM in the coordinator role is that it can read the query, assess what the query needs, and choose accordingly.

Decomposing a Multi-Concern Request

Decomposition is not only a multi-agent move. The same principle applies inside a single agent when one user message carries several independent concerns. "I need a refund for order #1234 and I want to update the shipping address on order #5678" is two tasks wearing one sentence. An agent that treats it as a single undifferentiated request tends to address one concern and drop the other, or to cross-wire the parameters between them.

The pattern is: split the message into its distinct concerns, handle each one as a separate unit of work, and synthesize the results into one reply. Critically, the concerns share context rather than being processed in isolation. The customer's verified identity, established once, applies to both the refund and the address change; the agent does not re-verify per concern, and it does not re-fetch the customer record for each task. Shared context established once, applied across every decomposed concern, then a single unified resolution.

This is distinct from the recognition-gap version of the same scenario. If an agent already handles single-concern requests well but its accuracy collapses specifically on multi-concern messages because it fails to notice the second concern, that is a pattern the model can be taught with examples, and few-shot is the cheaper fix. The decomposition pattern here is the structural answer: it applies when the workflow itself is wasteful or error-prone (re-fetching, re-verifying, sequential handling of

independent work), not when the model simply needs to recognize a pattern it otherwise executes correctly. Chapter 9's intervention classifier draws this line explicitly; the discriminator is whether the stem quotes a recognition failure or a structural inefficiency.

Three Ways to Create Subagents

The SDK supports three creation paths.[3]

Programmatic definition (the recommended path): pass AgentDefinition objects in the agents parameter of query(). This is the approach this chapter has been describing. Definitions live in code, are version-controlled, and can be constructed dynamically at runtime based on query characteristics or runtime configuration.

Filesystem-based definition: define subagents as markdown files in .claude/agents/ directories. The file name becomes the agent name. These are loaded at startup. If you create a new agent file while the session is running, restart the session to load it. Programmatically defined agents take precedence over filesystem-based agents with the same name.

Built-in general-purpose subagent: Claude can invoke the built-in general-purpose subagent at any time via the Agent tool without any custom definitions, as long as "Agent" is in allowedTools. This is useful for delegating exploratory research or quick analysis tasks without defining specialized agents upfront.

One practical note on agent creation that appears in the troubleshooting documentation: if Claude completes tasks directly instead of delegating to your defined subagent, verify that the Agent tool is in allowedTools, and verify that the subagent's description field clearly explains when to use it. The description is how Claude matches tasks to the right subagent. A vague description produces missed delegations.[3]

Context Isolation as a Context Window Management Tool

There is a secondary benefit to subagent context isolation that Chapter 11 covers at length. The short version belongs here because it connects directly to why the research system design makes sense.

Intermediate tool calls stay inside the subagent. The parent receives only the subagent's final message.[3] A document-analysis subagent might read and process fifty source files, generating extensive intermediate reasoning across many turns. None of that intermediate content appears in the coordinator's context. The coordinator receives the analysis summary: structured findings, source attribution, confidence levels. That is a few kilobytes instead of the megabytes of raw file content and processing steps.

This is not just an efficiency gain. It is what makes iterative refinement feasible. The coordinator can invoke synthesis, evaluate the output, identify a gap, re-delegate to document analysis with a targeted query, receive the additional findings, and re-invoke synthesis with the enriched inputs. Each pass keeps the coordinator's context manageable because the subagents absorb the exploration cost internally.

The research system's "re-delegate until coverage is sufficient" loop only works because the coordinator is not paying the context cost of each subagent's internal work.[2]

Session State: Resumption and Forking

A session is a conversation transcript. It persists the exchange of messages and tool calls, not the filesystem. When Claude Code closes, the session stays on disk. What you do with that stored transcript on the next invocation is the subject of Task Statement 1.7, and the exam tests three operations: resume, fork, and start fresh.

Resumption. The --resume flag (short form -r) continues a named session: claude --resume "auth-refactor" picks up exactly where that conversation left off, with the full prior transcript restored. The --continue flag (short form -c) resumes the most recent session in the current directory without requiring a name. Named resumption is the safer default in any directory that has hosted multiple unrelated tasks, because --continue silently picks up whichever session ran last, which may not be the one you intend.

Forking. The --fork-session flag, combined with --resume or --continue, branches the session: claude --resume "auth-refactor" --fork-session creates a new session whose transcript starts as a copy of the original, but diverges from that point forward. The original remains untouched. In the Agent SDK, the same operation is the fork_session boolean field on ClaudeAgentOptions, set alongside a resume target. Forking beats resuming the original twice when two approaches need to diverge from a shared analysis baseline (comparing two refactoring strategies, for example, or two test approaches that share the same initial codebase analysis). Forking beats copying context into a fresh session because the fork preserves the full tool-call history; a pasted summary loses it.

The resume-versus-start-fresh decision is the load-bearing exam distinction. Resume when prior context is mostly valid: the codebase has not changed substantially, and the session's cached tool results still reflect reality. When resuming after changes, explicitly inform the agent which files or functions changed so re-analysis is targeted rather than blind. Start fresh with an injected structured summary when prior tool results are stale or large-scale changes have occurred. Stale tool results in a resumed transcript mislead the agent because it treats cached file contents as current. The structured summary (the case-facts and structured-state machinery covered in Chapter 11) gives the new session the conclusions without the stale evidence.

One disambiguation matters because the book uses the word "fork" in two unrelated senses. The context: fork field in a skill's SKILL.md frontmatter (Chapter 7) isolates that skill's execution in a sub-agent context so intermediate exploration does not pollute the parent conversation. The --fork-session flag branches a whole session transcript so two approaches can diverge from the same prior state. They

share a word but not a mechanism: context: fork is about sub-agent isolation within a single invocation; session forking is about branching the trajectory of an entire conversation across invocations.

A fork does not fix stale context. If the session you fork from contains outdated tool results, every branch inherits that staleness. Forking is a divergence tool, not a freshness tool. When the problem is stale cached results rather than wanting to explore two paths, start fresh.

The Research System as a Design Walkthrough

Scenario 3 is worth walking through as a complete design exercise, because it is the scenario where all the concepts in this chapter intersect.[1,2]

The system has four subagents: a web-search agent, a document-analysis agent, a synthesis agent, and a report-generation agent. The coordinator receives a research topic and produces a comprehensive cited report.

The web-search and document-analysis agents are independent. They can run in parallel. The coordinator emits both Agent tool calls in the same response turn. Each agent runs in its own fresh context, with only the tools it needs (the web-search agent has web access; the document-analysis agent has file read access and no web tools) and a prompt that includes only the research topic and any source constraints specific to that agent's job. Neither agent sees the other's work while it runs.

The synthesis agent depends on both prior agents. It cannot run until the coordinator has both sets of results. The coordinator collects the web-search findings and the document-analysis findings, formats them as structured inputs (findings with source attribution separated from raw excerpts), and passes that prepared context in the prompt to the synthesis agent. The synthesis agent receives exactly the distilled outputs it needs. It does not receive the raw search

query, the intermediate tool call logs from the document analysis, or any other coordinator history.

The report-generation agent depends on synthesis output only. It receives the synthesis result and is responsible for formatting: citation formatting, section organization, executive summary generation. Its prompt specifies output format requirements in detail so the coordinator does not need to post-process the report.

Now consider what the coordinator does when the synthesis output is incomplete. The coordinator evaluates the synthesis result against the research topic. If it identifies a gap (a subtopic not covered, a claim not substantiated), it re-delegates to the web-search or document-analysis agent with a targeted follow-up query, waits for the result, and re-invokes synthesis with the original findings plus the supplementary findings. This iterative refinement loop is feasible because the coordinator's context window stays manageable: each subagent invocation absorbs its own tool call costs internally, returning only a concise structured result to the coordinator.[2]

The design decisions that make this work:

- Parallel execution for independent stages reduces latency without coordination overhead.
- Targeted context passing for each subagent prevents noise and keeps each agent's context window focused on the work it is doing.
- Iterative refinement rather than a single-pass pipeline improves result quality without requiring the coordinator to manage the full search-and-analysis context across multiple passes.
- Dynamic subagent selection lets the coordinator skip stages that are not needed for a specific query, rather than running the full pipeline every time.

This is hub-and-spoke applied correctly. The coordinator thinks. The subagents execute. The results flow back through the hub.

What the Exam Actually Tests

Scenario 3 is the hardest scenario in the exam, according to the walkthrough documentation.[1] The traps are documented explicitly.

Sharing full coordinator context with subagents is always wrong. The correct approach is to pass only context relevant to each subagent's task.[1]

Silently dropping subagent failures is always wrong. Structured error propagation requires reporting what was attempted, what failed, and whether the failure was an access error or a valid empty result. Chapter 12 covers the access-failure-versus-empty-result distinction in detail.

Ignoring provenance when resolving conflicting findings from different subagents is always wrong. When two subagents return conflicting statistics, the coordinator annotates the conflict with source attribution and publication dates rather than arbitrarily selecting one value. Chapter 12 covers this as the DataWithProvenance pattern.

The hub-and-spoke architecture questions on the exam follow a pattern. The correct answers involve a coordinator that selects subagents dynamically, passes targeted context, routes all inter-agent communication through itself, and handles failures with structured error reporting. The wrong answers involve flat architectures, full context sharing, static pipelines, and silent error swallowing.

The topology is simple. The discipline required to implement it correctly is not.

Key Takeaways

- The coordinator-plus-subagents pattern emerges when a task decomposes into subtasks that would contaminate each other if they shared a context; context isolation is the primary reason to reach for this topology, not task complexity alone.
- The Agent tool (formerly Task before Claude Code v2.1.63) is the invocation mechanism; it must appear in the coordinator's allowedTools, and checking both "Agent" and "Task" in block.name ensures cross-version compatibility.
- A subagent receives only its AgentDefinition.prompt, the Agent tool's prompt string, the project CLAUDE.md, and its defined tool set; it receives no parent conversation history, no parent system prompt, and no parent tool results.
- Pass only the context each subagent needs for its specific task: sharing the full coordinator history wastes tokens, introduces noise, and is the most common wrong answer on Scenario 3 questions.
- Emit multiple Agent tool calls in a single coordinator response to run subagents in parallel; sequential invocation across separate turns is the wrong mechanism for independent stages.
- Subagents cannot spawn subagents; the Agent tool must not appear in a subagent's tools array.
- The two named anti-patterns are overly narrow decomposition (creating coverage gaps between agents) and routing all queries through all subagents (static pipeline instead of dynamic selection).

CHAPTER 3: HOOKS AND THE LOOP LIFECYCLE

Summary: *The agentic loop has two distinct behavioral layers: what happens inside the model call (governed by system prompts and context), and what happens around it (governed by hooks). System prompts are probabilistic. Hooks execute as code. The exam tests whether you can correctly identify which layer a business requirement belongs to, and the answer determines whether your agent's compliance is a suggestion or a guarantee. This chapter defines the complete hook event taxonomy for both Python and TypeScript SDKs, explains the semantics of PreToolUse and PostToolUse in depth, covers priority rules when multiple hooks apply, and establishes the "Prompt or Hook?" decomposition framework that underlies every enforcement question in the exam.*

The Scene That Starts Every Discussion

A customer support agent processes refunds. The product requirements document says the agent must never approve a refund above five hundred dollars. So someone writes this into the system prompt:

IMPORTANT: Never process refunds above $500. If a refund is above $500, escalate to a human.

Clean. Clear. Documented. And wrong.

Not wrong as in incorrect policy. Wrong as the chosen enforcement mechanism. The model reads that instruction the same way it reads everything else: as context that influences its probabilistic response. On most requests, it complies. Occasionally, under the right combination of conversation history, phrasing, and token sampling, it does not. The failure mode is not dramatic. It does not announce

itself. A $700 refund processes, the customer receives it, and no one discovers the violation until the audit report arrives three weeks later.[1]

Now consider the hook version. A ***PostToolUse*** callback fires every time process_refund completes. The callback reads *tool_input["amount"].* If the amount exceeds 500, the hook returns a block and routes to escalation. No exception path exists. No conversation framing can talk the hook out of it. The rule runs as code.[2]

The distinction between these two implementations is the central argument of this chapter. Call it the **Prompt or Hook boundary**: the architectural decision point where you determine whether a business requirement belongs inside the model call (as context) or outside it (as code).

How Hooks Fit the Loop

The agent loop, as established in Chapter 1, is a cycle: Claude evaluates context, produces tool calls, the SDK executes those tools, results feed back, repeat. The loop terminates on *stop_reason: "end_turn".*

Hooks occupy the seams of this cycle. They are callback functions registered in the hooks field of your agent options, keyed to named events.[3] When an event fires, the SDK collects every hook registered for that event type, runs them, and applies their outputs before proceeding. The model never knows a hook ran. The model sees only the result of what the hook decided to allow, block, or transform.

This is the key architectural fact: **hooks execute in the orchestrator layer, not inside the model call**. A system prompt instruction lives in the context window. A hook lives in your application code. They have completely different execution guarantees.

```
options.hooks = {
  PreToolUse: [{ matcher: "process_refund", hooks: [refundLimitHook] }],
```

PostToolUse: [{ matcher: "get_customer", hooks: [timestampNormHook] }]
}

The matcher field is a regex pattern matched against the tool name. "process_refund" matches exactly. "Write|Edit" matches either file modification tool. "mcp__" (with the double underscore prefix) matches any MCP tool call. A hook without a matcher fires for every event of that type.[4]

The Complete Taxonomy

Both the Python and TypeScript SDKs share a common set of hook events. The TypeScript SDK adds several more for session and workspace lifecycle management.[5]

SHARED EVENTS (BOTH SDKS)

PreToolUse Fires before a tool executes. This is where you make permission decisions. The hook receives *tool_name, tool_input, session_id, cwd,* and *hook_event_name.* When the hook fires inside a subagent, *agent_id* and *agent_type* are also populated. Return a *permissionDecision* to control what happens next.

PostToolUse Fires after a tool returns, before the result reaches the model. This is where you normalize, augment, or replace tool outputs. The hook receives the full tool result, and you can return *additionalContext* (appends information to the result) or *updatedToolOutput* (replaces the result entirely).[6]

PostToolUseFailure Fires when a tool execution fails. Use this to log errors, trigger alerts, or implement fallback logic at the orchestrator level.

UserPromptSubmit Fires when a prompt is submitted to the agent. Use this to inject additional context into every user prompt, enforce access control at the prompt level, or validate prompt content before it reaches the model.

Stop Fires when agent execution finishes. Use this to save session state, emit completion metrics, or trigger downstream workflows.

SubagentStart and **SubagentStop** Fire when a subagent initializes and completes. The coordinator-plus-subagents pattern from Chapter 2 runs through the Agent tool; these hooks let you track, log, and aggregate parallel task activity from outside the subagent's context.[7]

PreCompact Fires before automatic context compaction. Receives a trigger field with value "manual" or "auto". Use this to archive the full conversation transcript before the summarization pass discards it. The full compaction strategy belongs to Chapter 11; the hook's callback interface is defined here.[8]

PermissionRequest Fires when a permission dialog would normally be displayed. Enables custom permission handling without surface-level UI.

Notification Fires for agent status messages: permission_prompt (Claude needs approval), idle_prompt (Claude is waiting for input), auth_success, and elicitation_dialog. Each notification includes a message field and optionally a title. Forward these to Slack, PagerDuty, or any webhook endpoint without disrupting the loop.

TYPESCRIPT-ONLY ADDITIONS

The TypeScript SDK adds hooks for session and workspace lifecycle events not yet available in Python:[5]

- **SessionStart** and **SessionEnd**: session initialization and termination
- **PostToolBatch**: fires once per batch of tool calls before the next model call; use this to inject conventions for the entire batch rather than once per tool
- **TeammateIdle**: a teammate in a multi-agent workspace becomes idle
- **TaskCompleted**: a background task completes
- **ConfigChange**: a configuration file changes; reload settings dynamically

- **WorktreeCreate** and WorktreeRemove: git worktree lifecycle management
- **Setup**: session setup and maintenance tasks

Note: SessionStart and SessionEnd can be registered as SDK callback hooks in TypeScript. In Python, they are only available as shell command hooks defined in settings files like .claude/settings.json.[9]

PreToolUse in Depth: Permission Decisions

PreToolUse is the enforcement hook. It intercepts the tool call before any side effects occur. The hookSpecificOutput you return contains a permissionDecision field with four possible values:[10]

"allow" The operation proceeds. The model receives the tool result as normal.

"deny" The operation is blocked. The model receives a rejection message as the tool result. Claude typically attempts a different approach or reports it cannot proceed. Pair deny with systemMessage to explain the reason and prevent retry loops (more on this shortly).

"ask" The SDK surfaces an approval prompt to the human operator. Used when an action requires human judgment before proceeding.

"defer" (TypeScript only) Ends the current query and defers it for later resumption. Not available in the Python SDK.

MODIFYING INPUT: **UPDATEDINPUT**

PreToolUse is not just a gate. It can transform what actually reaches the tool. Return updatedInput alongside permissionDecision: "allow" to substitute different arguments before execution. A practical example: all Write tool calls have their file_path argument rewritten to prepend a /sandbox prefix, redirecting every write to a controlled directory. The model issues the same call; the hook transparently redirects it.[11]

There is one constraint: you must always return a new object for **updatedInput**. Never mutate the original **tool_input** object. And when using updatedInput, the permissionDecision: "*allow*" must also be present, or the modification does not take effect.

THE **SYSTEMMESSAGE** FIELD

Every hook output supports a top-level systemMessage field. This field injects a message into the conversation that the model can see, independent of any tool result. When you block an operation with deny, the model knows the call was rejected but may not know why. If it does not know why, it will try again with a different phrasing. systemMessage prevents this by injecting a clear explanation:[12]

```
return {
  "hookSpecificOutput": {
    "hookEventName": "PreToolUse",
    "permissionDecision": "deny",
    "permissionDecisionReason": "Refund exceeds agent limit"
  },
  "systemMessage": "Refunds above $500 require human approval. "
                   "Use escalate_to_human with the refund details."
}
```

The model reads this, understands the constraint, and calls escalate_to_human instead of retrying process_refund. One hook, three outcomes: block the bad action, explain the constraint, redirect to the correct action.

THE **CONTINUE_** FIELD

The continue_ field (spelled **continue** in TypeScript, **continue_** in Python to avoid the reserved keyword) determines whether the agent keeps running after the hook completes. When False, the agent stops after the hook fires, regardless of other pending tool calls or turns. Use this to halt execution entirely on critical violations rather than just blocking the immediate tool call.[13]

PostToolUse in Depth: Output Transformation

PostToolUse *fires* ***after*** *the tool returns but* ***before*** *the model sees the result.* Two output fields control what the model receives:[6]

additionalContext Appends information to the tool result. The original result is preserved; additional content is concatenated. Use this to inject metadata, warnings, or supplementary data alongside what the tool returned.

updatedToolOutput Replaces the tool's output entirely before it reaches Claude. The model never sees what the tool actually returned. Use this for wholesale transformation.

DATA NORMALIZATION: THE CANONICAL USE CASE

Consider a customer support agent that calls three different MCP tools: one returns timestamps as Unix epoch integers, one returns them as ISO 8601 strings, one returns HTTP status codes as numbers. The model has to reason across all three tool results. If the formats differ, the model must internally reconcile the representations before doing anything useful with them.[14]

A PostToolUse hook with updatedToolOutput solves this cleanly. The hook intercepts every tool result, detects the format, normalizes it to a consistent schema, and sends that normalized result to the model. Claude sees uniform output regardless of which tool produced it. The tools do not need to be modified. The model does not need prompting to handle inconsistency.

```
def normalize_timestamps(tool_name, tool_input, tool_output):
    if isinstance(tool_output.get("timestamp"), int):
        # Unix epoch to ISO 8601
        from datetime import datetime, timezone
        ts = datetime.fromtimestamp(tool_output["timestamp"], tz=timezone.utc)
        tool_output["timestamp"] = ts.isoformat()
```

```
    return {
        "hookSpecificOutput": {
            "hookEventName": "PostToolUse",
            "updatedToolOutput": tool_output
        }
    }
```

This is Task Statement 1.5 from the exam guide in concrete form: PostToolUse for normalization is not an optimization. It is an architectural choice that prevents the model from needing to compensate for infrastructure inconsistency.[2]

Priority Rules: When Multiple Hooks Apply

When multiple hooks register for the same event and the same call triggers more than one of them, the **SDK runs all matching hooks in parallel**. Completion order is non-deterministic. Write each hook to act independently. Do not design hooks that depend on another hook having already run.[15]

For permission decisions, the SDK applies a strict priority rule:

Deny beats defer, which beats ask, which beats allow.

A single deny from any hook blocks the operation, regardless of what every other hook returns. The SDK does not average or vote on permission decisions. One deny is final.[16]

This rule has an important design implication. If you register a “log everything” hook alongside a “block dangerous operations” hook for the same event, the logger can return {} (empty object, meaning no decision) without interfering with the security hook. The deny wins. The log still runs. The hooks compose cleanly because the most restrictive result always wins.

Asynchronous Hooks: Side Effects Without Blocking

By default, the agent waits for each hook to return before proceeding. This is the correct behavior when the hook's output influences the operation: you need the permission decision or transformed output before the loop continues.

For hooks that only need to perform side effects, this wait is unnecessary latency. Logging, metrics, webhooks, Slack notifications: none of these need to block the agent. Return async: true (or async_: True in Python) to tell the agent to continue immediately without waiting:[17]

```
return {
  async: true,
  asyncTimeout: 5000  // optional millisecond timeout for background work
}
```

The constraint is absolute: **async hooks cannot block, modify, or inject context**. The agent has already moved on. Async mode is for fire-and-forget side effects only. If your hook needs to influence what happens next, it must be synchronous.

MCP Tool Names in Hook Matchers

MCP tools follow the naming pattern mcp__<server>__<action>. A tool called get_customer on an MCP server named support_db is addressed as mcp__support_db__get_customer throughout the hook system.[18]

This pattern matters for matcher design. Consider a hook that should audit every MCP tool call. The matcher "mcp__" (with the trailing double underscore treated as a prefix pattern in the regex context) covers all MCP tools regardless of server or action name. The matcher "mcp__support_db__" narrows to one server. The matcher "mcp__support_db__process_refund" targets a single tool.

The full treatment of MCP server configuration, including how servers are defined and how tools are discovered, is in Chapter 5. The hook matcher syntax is established here.

The Prompt or Hook Boundary

The exam will present you with a business requirement and ask you to choose the right enforcement mechanism. The decision comes down to one question: does this requirement need a guarantee?

Prompt instructions are probabilistic. They influence the model's behavior by adding context to the input the model reasons over. Under typical conditions, a clear system prompt instruction will be followed reliably. But "reliably" is not "always." The model can be argued out of an instruction through conversation framing. It can misweigh the instruction when the context window is long. It can fail to apply it in edge cases the author did not anticipate.[1]

Hooks execute as code. They do not reason. They do not weigh competing considerations. They are functions: given input, return output. If the input matches the condition, the hook applies. Every time. Regardless of what the model was thinking, what the user said, or how the conversation got there.

Here is the decomposition framework. Apply it to every enforcement question:

Use a prompt when: - The requirement is a soft preference: style, tone, formatting conventions - Partial compliance is acceptable: "prefer shorter responses" does not need a hard stop on long responses - The requirement depends on context that only the model can evaluate: "be empathetic when the customer is upset" requires model judgment - A violation is recoverable without business consequence

Use a hook when: - The requirement is a business rule with a defined threshold: refund limits, transaction amounts, record counts - Compliance must be deterministic: identity verification before financial operations, access control gates

- The failure mode is a compliance violation or safety issue, not just a suboptimal response - The requirement applies regardless of conversation framing or model confidence

The customer support scenario from the beginning of this chapter lands squarely in the hook column. “Never process refunds above $500” is a business rule with a defined threshold. Its failure mode is a compliance violation. Probabilistic compliance is not acceptable. A PostToolUse hook on process_refund is the correct mechanism, full stop.[2]

This distinction appears explicitly in Task Statement 1.4 of the exam guide: “when deterministic compliance is required, prompt instructions alone have a non-zero failure rate.”[1] The exam is not asking whether prompts usually work. It is asking whether you understand that “usually” is architecturally insufficient for certain requirements.

Decomposing Requirements: Worked Examples

The Prompt or Hook boundary is the named concept. Decomposition is the skill. Here are three requirement patterns and how they map:

PATTERN 1: WORKFLOW ORDERING

Requirement: The agent must call get_customer and receive a verified customer ID before calling process_refund.

Wrong approach: Add to system prompt: “Always call get_customer first and wait for a verified customer ID before processing any refund.”

Right approach: PreToolUse hook on process_refund that checks whether a verified customer ID is available in session state. If not, return deny with a systemMessage instructing the model to retrieve the customer first.

Why: This is a prerequisite gate. The failure mode if skipped is processing a refund against an unverified customer, which is a business logic violation. Code enforces it; a prompt suggests it.[1]

PATTERN 2: OUTPUT FORMAT NORMALIZATION

Requirement: Multiple MCP tools return timestamps in different formats. The model should receive consistent ISO 8601 timestamps.

Wrong approach: Add to system prompt: "If a tool returns a Unix timestamp, convert it to ISO 8601 before reasoning about it."

Right approach: PostToolUse hook with updatedToolOutput that normalizes all timestamp fields before Claude sees them.

Why: Asking the model to perform format conversion is adding cognitive overhead to a task that code handles trivially. The hook runs in the orchestrator; the model processes clean data.[14]

PATTERN 3: TONE AND STYLE PREFERENCES

Requirement: Responses to customers should be warm and avoid technical jargon.

Right approach: System prompt. This is a style preference that requires model judgment to apply appropriately. No threshold, no determinism requirement, no compliance risk. A hook enforcing "warmth" would be both impossible and absurd.

Wrong approach: Writing a hook that pattern-matches output for technical terms and blocks responses containing them. This produces a brittle, false-positive-heavy system that cannot handle legitimate uses of technical language in technical contexts.

The hook is not smarter than the model on questions of warmth. Stop before you write one that tries to be.

SubagentStart and SubagentStop

The SubagentStart and SubagentStop events connect the hook system to the multi-agent patterns from Chapter 2. When the coordinator spawns a subagent using the Agent tool, SubagentStart fires. When the subagent completes, SubagentStop fires.

These hooks give the parent orchestrator visibility into parallel subagent activity without having access to the subagent's internal context. A SubagentStop hook can aggregate results, detect failures, or trigger coordinator logic when all parallel subagents have completed.[7]

One practical implication: subagents do not automatically inherit parent agent permissions. If the parent agent has a PreToolUse hook auto-approving certain tools, those hooks apply to the parent's tool calls. The subagent runs in its own execution context with its own permission checks. When multiple subagents are spawned in parallel, each one may independently request permissions. Handle this by configuring permission rules that apply at the subagent session level, or use PreToolUse hooks scoped to run inside subagent sessions using the agent_type field on the hook input.[19]

The PreCompact Hook

Context compaction is covered in depth in Chapter 11. The PreCompact hook's place in this chapter is its event mechanics.

The hook fires before compaction occurs, whether triggered automatically (when the context window approaches its limit) or manually (when /compact is sent as a prompt). The trigger field on the hook input tells you which case you are in: "auto" or "manual".[8]

The canonical use case is archiving the full conversation transcript before the summarization pass discards it. The agent loop will continue with a summarized history. The PreCompact hook gives you a window to capture the full record before

that happens: write it to a file, push it to an audit log, send it to an analytics endpoint.

An async hook works correctly here. Archiving is a side effect. The compaction does not need to wait for the archive to complete.

```
def archive_transcript(hook_input):
    transcript = hook_input.get("messages", [])
    trigger = hook_input.get("trigger")
    write_to_audit_log(transcript, trigger=trigger)
    return {"async_": True}
```

Configuring Hooks: The Registration Pattern

Hooks are passed in the hooks field of your agent options. The structure is consistent across both SDKs: keys are event names, values are arrays of matcher configurations, each containing an optional pattern and the callback functions to run when it matches.[3]

```
from claude_agent_sdk import ClaudeAgentOptions
options = ClaudeAgentOptions(
    tools=[lookup_customer, check_order, process_refund, escalate_to_human],
    hooks={
        "PreToolUse": [
            {
                "matcher": "process_refund",
                "hooks": [refund_limit_hook],
                "timeout": 30
            }
        ],
        "PostToolUse": [
            {
```

```
            "matcher": "mcp__support_db__.*",
            "hooks": [normalize_timestamps_hook]
        }
    ],
    "Stop": [
        {
            "hooks": [save_session_state_hook]
        }
    ]
  }
)
```

The timeout field on a matcher defaults to 60 seconds. Increase it for hooks that perform external API calls or database writes. Use AbortSignal in TypeScript to handle cancellation gracefully when a hook times out.

Hook event names are case-sensitive. PreToolUse works. preToolUse does not register correctly.[20]

What the Exam Specifically Tests

The exam draws from Task Statements 1.4 and 1.5. Both task statements are built around the same conceptual axis: knowing when to put business logic in a prompt versus a hook.

Here is the question pattern you will see most often:

A customer support agent has a policy rule about [threshold or prerequisite]. The current implementation uses [prompt instruction or hook]. The agent [sometimes violates / always enforces] the policy. What is the correct implementation?

The answer key every time: hooks for thresholds, prerequisites, and compliance requirements; prompts for preferences, style, and requirements that require model judgment to apply contextually.

Secondary patterns:

- **PostToolUse for data normalization**: when multiple tools return different formats, the hook normalizes before the model processes. The prompt-based alternative (asking the model to convert formats) adds unnecessary reasoning load.
- **systemMessage to prevent retry loops**: pair with deny to explain why an operation was blocked. Without the explanation, the model retries. With it, the model redirects.
- **Priority rules for multiple hooks**: one deny overrides any number of allow returns from other hooks. Write hooks to act independently; do not rely on hook execution order.
- **Async hooks for logging**: side effects that do not need to influence agent behavior should return async: true to avoid adding latency to every tool call.
- **MCP tool name pattern in matchers**: mcp__<server>__<action> is the format; use it in regex matchers to target specific servers or specific tools.

One original sample question to test the concept:

Q: A compliance team requires that all database write operations be preceded by a successful call to verify_access. An architect adds "always call verify_access before any write operation" to the system prompt. During testing, the agent occasionally performs writes without calling verify_access when the user provides authorization context in the conversation. What should the architect do?

A. Add more detail to the system prompt instruction about when verify_access is required.
B. Implement a PreToolUse hook on write operations that checks for a verified access token and denies the call if one is not present.

C. Implement a PostToolUse hook on verify_access that blocks subsequent write calls until it fires.

D. Use tool_choice: "any" to force the model to call a tool on every turn.

The correct answer is B. *The requirement is a deterministic prerequisite gate. Hook-based enforcement is the correct mechanism. Option A fails because the problem is not vague instructions: it is probabilistic enforcement. Option C misapplies PostToolUse (it fires after verify_access, not before write operations). Option D forces tool use but does not address the ordering requirement.*

When hooks are absent, tool descriptions become the only routing mechanism. Chapter 4 is about getting that mechanism right.

Key Takeaways

- Hooks execute in the orchestrator layer, outside the model call. Prompts execute inside the model call, as context. These are different execution environments with different guarantees.
- PreToolUse handles permission decisions (allow, deny, ask; defer in TypeScript only) and input modification via updatedInput. The systemMessage field injects explanations that prevent the model from retrying blocked operations.
- PostToolUse handles output transformation via additionalContext (append) and updatedToolOutput (replace). The canonical use case is normalizing heterogeneous data formats from multiple MCP tools before the model processes them.
- When multiple hooks apply to the same event, they run in parallel. Deny wins over defer over ask over allow. A single deny blocks regardless of other hooks.

- Async hooks return async: true for fire-and-forget side effects. They cannot block, modify, or inject context; the agent has already proceeded by the time they complete.
- The PreCompact hook fires before context compaction, receiving a trigger field of "manual" or "auto". Use it to archive transcripts before summarization. The Prompt or Hook decision: use hooks when deterministic compliance is required; use prompts when model judgment is required to apply the requirement contextually.
- MCP tools are addressed in hook matchers using the pattern mcp__<server>__<action>. Regex patterns like mcp__support_db__.* target all tools on a specific server.

The model is not your enforcement layer. Your code is.

CHAPTER 4: DESIGNING TOOLS THAT GET SELECTED

Summary: *A tool's description is not documentation for humans. It is a routing key for the model. When you write "Retrieves customer information" as your description, you are not explaining the tool to a developer who might read the code. You are giving the model the only signal it has to decide whether this is the right tool for the current request. If that signal is vague, the model guesses. When the model guesses wrong at scale, you have a reliability problem dressed up as an architecture problem.*

This chapter introduces the concept of ***tool descriptions as routing keys*** *and explains why getting tool descriptions right is the highest-leverage act in tool design. It covers the full structure of a tool definition, how the API assembles that structure into a system prompt the model reads, and how tool_choice lets you control selection when the model's autonomy is not what the situation requires. It also explains what happens when agents have too many tools, and why the solution to that problem is not better descriptions: it is better distribution.*

The Bug That Isn't a Bug

Production logs show that the agent frequently calls the wrong tool. A user types "check my order #12345" and the agent calls get_customer instead of lookup_order. Both tools accept similar identifier formats. Both have minimal descriptions: "Retrieves customer information" and "Retrieves order details". The agent is not broken. It is doing exactly what you told it to do, which is to choose between two tools with descriptions so similar that the choice is essentially random noise.[1]

This is Sample Question 2 from the exam guide, and it is worth dwelling on before any technical discussion. The question asks for the most effective first step to

improve selection reliability. Option A proposes adding few-shot examples to the system prompt, showing order-related queries routing to lookup_order. Option C proposes a routing layer that parses user input before each turn and pre-selects a tool based on detected keywords. Option D proposes consolidating both tools into a single lookup_entity tool that handles both cases internally.

The correct answer is B: expand each tool's description to include input formats, example queries, edge cases, and boundaries explaining when to use it versus similar tools.[1]

Option A adds token overhead without fixing the root cause. Option C is over-engineered and bypasses the model's natural language understanding with a brittle keyword-matching layer that needs to be maintained in parallel with the tools themselves. Option D has merit in some situations, but consolidating tools is a structural decision with tradeoffs, not a first step when the actual problem is a description that is three words long. Option B directly addresses why the routing failed. The descriptions gave the model nothing to work with.

That is the thesis of this chapter. And it is also the named concept: **tool descriptions as routing keys**. The model selects tools based on description semantics, not on the tool's actual implementation. A tool that does something sophisticated but has a vague description will be misrouted. A tool that does something simple but has a precise, boundary-rich description will be selected correctly almost every time. Tool design is prompt engineering applied to the tool layer, and the exam tests whether you understand that.

What a Tool Definition Actually Is

When you pass a tools array to the Claude API, the API does not hand those definitions to the model as raw JSON for it to interpret. It constructs a special system prompt that includes the tool definitions, formatting instructions, and any user-specified system prompt.[2]

The constructed prompt starts with something like this:

In this environment you have access to a set of tools you can use to answer the use r's question.
{{ FORMATTING INSTRUCTIONS }}
Here are the functions available in JSONSchema format:
{{ TOOL DEFINITIONS IN JSON SCHEMA }}
{{ USER SYSTEM PROMPT }}
{{ TOOL CONFIGURATION }}

The model reads this before it reads anything you typed into the user message. Every tool definition you wrote is sitting in that constructed prompt in JSON Schema format, and the model's entire decision about which tool to call flows from reading that. Your description field is the prose inside that JSON block. It is the only natural language the model has to reason about why this tool exists and when it should be called.

Each tool definition includes four top-level parameters.[2]

name must match the regex ^[a-zA-Z0-9_-]{1,64}$. No spaces, no dots, no unicode. Maximum 64 characters. The name is also a signal. lookup_order and get_customer are already doing more routing work than search and fetch would do. Names that encode intent reduce the load the description has to carry.

description is plaintext explaining what the tool does, when it should be used, and how it behaves. This is the routing key. Everything else about the definition exists to constrain or clarify the mechanics; the description is the semantic trigger that determines whether the model reaches for this tool or a different one.

input_schema is a JSON Schema object defining the tool's parameters. The schema tells the model what to send when it calls the tool. Well-written property descriptions inside the schema are a secondary routing signal. If a parameter description says "Customer email address (must contain @)" or "Phone in E.164 format, e.g., +15551234567," the model gets format constraints at the point of use.

input_examples is an optional field that accepts an array of example input objects. Each example must be valid according to the tool's input_schema: invalid examples return a 400 error. Examples are included in the prompt alongside your schema, showing the model concrete patterns for well-formed tool calls. This is especially useful for tools with nested objects, optional parameters, or format-sensitive inputs where the schema alone does not make the expected structure obvious.[2]

Here is what the contrast looks like in practice. A weak description:

```
{
  "name": "lookup_order",
  "description": "Retrieves order details",
  "input_schema": {
    "type": "object",
    "properties": {
      "id": { "type": "string" }
    }
  }
}
```

A routing key:

```
{
  "name": "lookup_order",
  "description": "Look up a specific order by order ID, order number, or tracking num
ber. Use this tool when the user references an order, shipment, or package — not w
hen they are asking about their account, profile, or billing. Returns order status, line
 items, shipping address, and estimated delivery date. Input: exactly one of order_id
 (format ORD-XXXXXXXX), order_number (any numeric string), or tracking_number.
Returns an order object or empty array if not found. Empty result means the order
was not found, not a system error. Do NOT use this tool to look up customer accoun
t information — use get_customer for that.",
```

```
  "input_schema": {
    "type": "object",
    "properties": {
      "order_id": {
        "type": "string",
        "description": "Order ID in format ORD-XXXXXXXX"
      },
      "order_number": {
        "type": "string",
        "description": "Numeric order number as a string"
      },
      "tracking_number": {
        "type": "string",
        "description": "Shipping carrier tracking number"
      }
    }
  }
}
```

The second version specifies what the tool returns, what input formats it accepts, when to use it, and explicitly when not to use it. That last part matters. The phrase “Do NOT use this tool to look up customer account information” is not redundant with having a separate get_customer tool. The model needs to see the boundary stated from both sides: what lookup_order does, and what it explicitly does not cover. If only get_customer says “use this for customer information,” the model may still be uncertain whether an order lookup is also a kind of customer information retrieval. State the boundary twice, once in each tool.[1,3]

The Four Modes of Tool Selection

Once you have tools defined, you control how the model uses them with the tool_choice parameter. There are four values.[2]

"auto" tells the model it may call any provided tool or may respond in natural language. This is the default when tools are provided. Use it when the user's request might not require a tool call, and a conversational response is acceptable.

"any" tells the model it must use one of the provided tools but does not force a particular one. The model cannot return conversational text. The API prefills the assistant message to force a tool call, which means the model will not emit natural language before the tool_use block, even if explicitly asked to do so.[2] Use "any" when the task is inherently a tool-calling task and a text response would be a failure mode. Structured extraction workflows are the canonical use case: the system needs JSON, not a paragraph.

{"type": "tool", "name": "..."} (forced tool) tells the model it must call one specific named tool. Like "any", this prefills the assistant message, so natural language preamble is suppressed. Use forced tool selection when you need a specific operation to happen first. The exam guide gives the pattern of forcing extract_metadata before enrichment tools run: the metadata extraction step is a prerequisite, so you force it with tool_choice in the first turn, then process enrichment in follow-up turns with the metadata result in hand.[1]

"none" prevents tool use entirely. This is the default when no tools are provided. You can explicitly set it when you want the model to reason and respond without any tool calls, even though tools are available in the definition list. Useful in multi-turn flows where you want a reasoning step before a tool call step.

There is one important constraint to know for the exam. When using extended thinking, "any" and forced tool selection are not supported. Extended thinking is compatible only with "auto" and "none".[2] This is a gotcha that trips candidates

who design thinking-enabled agents and try to guarantee tool calls at the same time.

When Descriptions Are Not Enough: The Overlap Problem

Good descriptions address the scenario where each tool has a distinct purpose but the description fails to communicate it. A different failure mode arises when two tools have descriptions that are genuinely close because the tools themselves do similar things. analyze_content and analyze_document with near-identical descriptions are the exam guide's example.[1]

In this situation, improving descriptions helps but cannot fully solve the problem. If both tools legitimately operate on overlapping input types and produce overlapping outputs, the model is going to have low confidence in the distinction no matter how carefully you word each description. The tools themselves are the problem.

There are two fixes.

Rename and eliminate functional overlap. If analyze_content and analyze_document differ only in what they are called but do the same thing, merge them into one well-named tool. The exam guide frames this as renaming tools and updating descriptions to eliminate functional overlap.[1] Fewer tools with clearer functions is almost always better than more tools with carefully managed overlap.

Split a generic tool into purpose-specific tools with defined contracts. If a single analyze_document tool is doing three different things (extracting data points, summarizing content, and verifying claims), split it into extract_data_points, summarize_content, and verify_claim_against_source. Each now has a description that is unambiguous because it describes only one thing. The exam guide gives this pattern explicitly as a skill.[1]

The split approach is the more general solution for overloaded tools. A tool that does several things has a description that has to describe several things, which produces a description that is long and hedged and still not precise. A tool that does one thing can have a description that is short, specific, and unambiguous.

System Prompt Keyword Sensitivity

This is the failure mode that surprises architects who have done everything else right. You write careful, boundary-rich descriptions for every tool. You test routing on a set of representative inputs. Routing looks clean. Then a category of request starts misrouting and you cannot figure out why, because the descriptions are correct.

The problem is in the system prompt, not the tool descriptions. The API constructs the system prompt from tool definitions plus any user-specified system prompt. If your system prompt contains a phrase like “always prioritize account verification” or “when in doubt, check the customer record first,” those instructions create keyword associations that can override tool description logic. The word “customer” in a customer-related query matches the keyword association in the system prompt before the model even reads the tool description carefully.[1]

This is system prompt keyword sensitivity. The fix is to review the full constructed prompt, not just the tool definitions, looking for instructions that might create unintended tool associations. Phrases that read as reasonable business logic to a human (“verify the customer before proceeding”) can act as hidden routing overrides to the model. Rephrase them in terms of workflow sequencing rather than entity-type associations: “call get_customer before calling process_refund” is much safer than “always check customer information first” if you have three different tools that touch customer data.

There is a secondary version of this problem: a system prompt instruction intended to shape tone or behavior can accidentally pattern-match to a tool use trigger. An

instruction like “when users ask about account status, be comprehensive” might associate “account status” with get_customer in a way that pulls order-related queries toward the customer tool. Auditing system prompts for accidental keyword-to-tool associations is not optional in production systems; it is part of the tool design process.[1]

The 18-Tool Problem

The exam includes a scenario where a developer productivity agent has 18 tools and is selecting the wrong ones. The instinct is to write better descriptions. The correct solution is different.

The documented optimal number of tools per agent is 4-5. At 18 tools, the decision complexity in the constructed system prompt is high enough that selection reliability degrades.[3] This is not a framing problem or a description quality problem. It is a structural problem. The model is being asked to pick from 18 options for every request, and the cognitive load of navigating 18 tool descriptions is simply too high to maintain reliable selection across diverse input types.

The solution: distribute tools across specialized subagents.[1,3]

Instead of one agent with 18 tools, a coordinator agent with 3-4 delegation tools spawns subagents with 4-5 focused tools each. The coordinator’s job is to route to the right subagent. Each subagent’s job is to pick from a small, coherent set of tools. The coordinator does not need to know which individual tool to call; it needs to know which subagent covers this class of request. The subagent does not need to worry about the full tool surface; it sees only the tools relevant to its domain.

This is the coordinator-plus-subagents pattern covered in Chapter 2. The relevant point here is that 4-5 tools per agent is a tool design constraint, not just an architecture preference. It determines whether tool selection is reliable. If you are

debugging routing errors and the agent has more than 5 tools, tool count is your primary suspect, and description quality is secondary.

```
// Overloaded: one agent, 18 tools, selection degrades
{
  "tools": [
    "lookup_customer", "update_account", "verify_identity",
    "find_order", "process_refund", "update_shipping",
    "track_package", "send_email", "send_sms",
    "create_ticket", "escalate", "search_kb",
    "check_inventory", "apply_coupon", "schedule_callback",
    "log_interaction", "generate_report", "update_preferences"
  ]
}

// Correct: coordinator routes to specialized subagents
// coordinator: ["Task", "summarize_results", "format_response"]
// customer_agent: ["lookup_customer", "update_account", "verify_identity", "check_status"]
// order_agent: ["find_order", "process_refund", "update_shipping", "track_package"]
// comms_agent: ["send_email", "send_sms", "create_ticket", "escalate_human"]
```

Note that the coordinator's tool list is small and semantic: spawn a subagent, summarize results, format a response. The coordinator never has to reason about whether to call track_package or update_shipping. That distinction lives in the order_agent, which has four tools and can reliably distinguish between them.[3]

Writing a Routing Key: The Mechanics

Given everything above, here is what a complete routing-key description includes.

What the tool does in one sentence. This is the disambiguation anchor. It needs to be different in substance, not just word choice, from every other tool in the agent's tool set. If two tools have descriptions whose first sentence could apply to either, the descriptions are not working.

What the tool returns. Output type shapes selection. "Returns an order object with line items, shipping address, and delivery estimate" tells the model something about what using this tool produces. If the next step in the workflow needs an order ID, the tool that returns one gets routed to.

What input formats are expected. Format constraints in the description reduce invalid parameter generation and also help routing when the user's request contains an identifier. "Order ID in format ORD-XXXXXXXX" and "phone in E.164 format" let the model match the user's stated identifier to the tool that handles it.

Explicit when-not-to-use boundaries. State the boundary conditions from this tool's perspective. What does this tool not handle that a similar tool does? If you have lookup_order and get_customer in the same agent, each description should explicitly disclaim the other tool's domain.

What a successful empty result means. This is subtle but consistently tested. A tool that returns an empty array when no matches are found should say so: "Returns empty array if no match found. Empty result means not found, not a system error." Without this, the model may treat a not-found result as ambiguous and retry the wrong tool.[3]

The input_examples field is an additional channel for this information. For tools with complex nested inputs or optional parameters with unclear interplay, examples show the model what a well-formed call looks like. An example with only the required parameters shows that optional ones are genuinely optional. An example with a specific format for a phone number shows the format more concretely than a prose description can.[2]

What the Exam Tests

The exam's Sample Question 2 is the clearest signal about what this domain tests.[1] The correct answer is expanding descriptions, and the explanation is explicit: descriptions are the primary mechanism, and minimal descriptions leave the model without context to differentiate similar tools. The distractors are all recognizable patterns (few-shot prompting, routing layers, tool consolidation) that have legitimate uses elsewhere but are over-engineered or misdirected as first responses to a routing failure caused by bad descriptions.

The exam also tests tool_choice selection reasoning. The pattern to internalize: "auto" when the model needs discretion; "any" when a tool must be called but the specific tool is the model's choice; forced tool when a specific operation must happen in a specific order; "none" when tool calls need to be suppressed even though tools are defined.

The 18-tool scenario from Scenario 4 (Developer Productivity) tests distribution over description. The exam's correct answer is reducing to 4-5 tools per agent and distributing the rest to specialized subagents. Making descriptions longer, fine-tuning the model, or switching to a larger model are the distractors. They are wrong because the problem is structural, not descriptive.[3]

System prompt keyword sensitivity is a less obvious test point but it appears in Task Statement 2.1. Candidates who understand tool design only as "write good descriptions" will miss questions that require reviewing the system prompt for keyword-sensitive instructions that override those descriptions. The full audit scope is the constructed prompt, not just the tool definitions.

Model selection is part of the tool count equation. The Claude Opus model handles complex tool selection better and will seek clarification when a request is ambiguous across multiple tools. Claude Haiku models work for straightforward tool sets but may infer missing parameters rather than asking for them.[2] Know which lever to reach for first.

Key Takeaways

- Every tool definition the model sees is translated into a fragment of the system prompt. The description field is the sentence that tells the model when to reach for this tool. Treat it accordingly.
- A minimal description like "Retrieves order details" is not a valid description. A valid description specifies what the tool returns, what input it expects, what makes this tool distinct from similar tools, and when not to use it.
- tool_choice has four values: "auto" (model decides; default when tools are provided), "any" (model must call a tool but picks which one), {"type": "tool", "name": "..."} (forces a specific tool), and "none" (prevents tool use; default when no tools are provided). Knowing which value to reach for in which situation is a tested exam skill.
- The input_examples field on a tool definition gives the model concrete patterns for valid inputs alongside the schema. Use it for tools with complex, nested, or format-sensitive parameters.
- 4-5 tools per agent is the documented optimal. Giving a single agent 18 tools degrades selection reliability. The solution is not a more elaborate description strategy; it is distributing tools across specialized subagents.
- System prompt keyword sensitivity is a real failure mode. A well-written tool description can still be overridden by a keyword-sensitive instruction elsewhere in the system prompt. Reviewing the full prompt for unintended tool associations is part of the audit.

CHAPTER 5: MCP AND THE BUILT-IN TOOLKIT

Summary: *The Model Context Protocol gives agents a standard interface for discovering and invoking external tools across system boundaries. Claude Code's built-in tools encode an opinionated workflow for exploring unfamiliar codebases:* ***Grep*** *to find entry points,* ***Read*** *to trace flows,* ***Edit*** *to make targeted changes, and* ***Bash*** *only when no built-in alternative exists. The exam tests both the MCP configuration model (in-code versus .mcp.json, transport types, allowedTools, environment variable expansion) and the selection rules for built-in tools. The most common failure pattern is reaching for Bash when a purpose-built tool already exists. MCP servers are configured via two paths: programmatic mcpServers options for single-agent applications, and .mcp.json at the project root for team-shared tooling committed to version control. Transport type follows server location: stdio for local processes, HTTP/SSE for remote servers. Credentials belong in environment variables, never hardcoded. The allowedTools contract governs which MCP tools an agent may invoke, and permissionMode: "acceptEdits" does not auto-approve MCP tools. The Incremental Exploration pattern for codebase navigation applies Grep first to find entry points, Read selectively to trace flows, and stops when enough context exists rather than loading every file upfront.*

The 18-Tool Problem

Scenario 4 from the exam guide opens like this: an agent built for developer productivity has 18 tools and is asked to read a configuration file. The agent calls Bash("cat config.json") instead of Read("config.json"). Two things went wrong simultaneously, and both are testable.[1]

The first problem is the 18-tool count. As Chapter 4 established, selection reliability degrades when an agent carries too many tools. The fix is distribution: reduce to 4-

5 tools per agent and push the rest to specialized subagents. That is a topic Chapter 4 owns.

The second problem is the wrong tool for the job. Read exists precisely to read files. **Bash exists for shell operations where no built-in tool covers the need.** Calling Bash("cat config.json") is not a creative solution. It is a category error. The exam treats it as a hard anti-pattern, and this chapter is where that principle lives.[1]

The Standard: Model Context Protocol

MCP is an open standard for connecting AI agents to external tools and data sources.[2] Before MCP, every integration was bespoke: custom tool implementations, custom authentication flows, custom naming schemes, custom discovery mechanisms. MCP standardizes all of it.

The practical benefit is a growing ecosystem of community servers: GitHub, Jira, Postgres, Slack, and others. When a standard integration exists, you use the community server. Custom MCP servers are for team-specific workflows that the community has not covered.[3]

For the exam, MCP matters across three areas: how you configure servers, how tools get named, and how permissions work.

Configuring MCP Servers

Two configuration paths exist: in-code via the mcpServers option on query(), and declaratively via a .mcp.json file at your project root.[2]

In-code configuration is appropriate when you are building a single agent application and want full programmatic control. You pass the server definition directly in the options object. It does not require any file on disk.

.mcp.json configuration is appropriate for team tooling. Place the file at your project root and version-control it. When query() runs with the default settingSources (which includes "project"), the file loads automatically. Any teammate who clones the repo gets the same MCP servers.[3,4]

The scope distinction the exam tests: .mcp.json at the project root is shared with the team. User-level MCP configuration lives in ~/.claude.json and is personal only. Use user-level config for experimental servers you are evaluating or for personal tools that do not belong in the team's shared setup.[3,4]

TRANSPORT TYPES

Three transport mechanisms are available, and selection depends on where the server runs.[2]

stdio is for local processes. If the server documentation gives you a command to run (something like npx @modelcontextprotocol/server-github), use stdio. The agent and the server communicate via stdin/stdout on the same machine.[2]

HTTP/SSE is for cloud-hosted servers and remote APIs. If the documentation gives you a URL, use HTTP. In programmatic code, the transport type is "http". In .mcp.json and other JSON config files, "streamable-http" is accepted as an alias.[2]

SDK MCP servers let you define custom tools directly inside your application code, without running a separate server process at all.[2]

The alias matters for the exam: "streamable-http" works in .mcp.json; "http" is what the programmatic mcpServers option accepts. They are not interchangeable across contexts.

AUTHENTICATION WITHOUT COMMITTING SECRETS

MCP servers that connect to external services need credentials. **The only acceptable pattern is environment variable expansion.**[2,3]

In .mcp.json, use ${ENV_VAR} syntax in the env field:

```
{
  "mcpServers": {
    "jira": {
      "command": "npx",
      "args": ["@company/jira-mcp-server"],
      "env": {
        "JIRA_URL": "${JIRA_URL}",
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

The ${JIRA_TOKEN} syntax expands from the runtime environment. The secret stays out of the file. The file stays safe to commit.[2]

Hardcoding a literal API key in .mcp.json is the anti-pattern the exam tests. That file will be committed to version control. The key will be exposed. Do not do it.[3]

Tool Naming and the allowedTools Contract

MCP tools follow a deterministic naming pattern: mcp__<server-name>__<tool-name>.[2]

A GitHub server named "github" with a list_issues tool becomes mcp__github__list_issues. The pattern is predictable. You can enumerate exact names or use wildcards.

By default, Claude can see that MCP tools are available but cannot call them without explicit permission. Granting access requires the tool name to appear in allowedTools:[2]

allowedTools: ["mcp__github__list_issues", "mcp__github__create_pr"]

Wildcards work for granting access to an entire server:

allowedTools: ["mcp__github__"]*

This is the preferred approach when you intend the agent to use all tools from a server. The wildcard grants exactly what you specified and nothing more.[2]

THE PERMISSION MODE TRAP

This distinction is on the exam. permissionMode: "acceptEdits" does **not** auto-approve MCP tools. MCP tools remain ungated unless you include them explicitly in allowedTools.[2]

permissionMode: "bypassPermissions" does auto-approve MCP tools because it disables all permission prompts system-wide. That is broader than necessary for most use cases. A wildcard in allowedTools gives you MCP access with surgical precision. The permission mode values themselves are defined in Chapter 1; the exam application here is understanding which mode handles MCP and which does not.[2]

MCP Tool Search

When you configure many MCP servers, their tool definitions accumulate in every request's context. A few servers with many tools can consume significant context before the agent does any actual work.[5]

MCP tool search addresses this. When enabled, tool definitions are withheld from context. The agent loads only the definitions it needs for each turn. Tool search is enabled by default.[2]

The full treatment of context cost and ToolSearch as an on-demand loading mechanism belongs to Chapter 11. For this chapter: tool search exists, it is on by default, and it exists specifically because MCP server schemas are expensive to carry at all times.

MCP Resources

Beyond tools, MCP servers can expose resources: content catalogs that give the agent visibility into available data without requiring exploratory tool calls.[4]

Consider an agent that needs to understand which Jira issues exist before deciding which to fetch. Without resources, the agent calls a search tool speculatively, reads results, calls again with refined parameters. With an MCP resource exposing an issue summary catalog, the agent can inspect the available content and make an informed tool call on the first attempt.[4]

Resources reduce the exploratory overhead. For the exam, know that they exist and that their purpose is reducing unnecessary tool calls by giving the agent catalog-level visibility.

Built-In Tools: The Opinionated Toolkit

Claude Code ships with six built-in tools. Knowing which to reach for, and when, is directly tested in Scenario 4 and across multiple Domain 2 questions.[1,3]

The mental model: built-in tools are purpose-built. Each one does exactly one thing well. Bash is the general-purpose fallback for when no purpose-built tool covers the need. Reaching for Bash when a purpose-built tool exists is always the wrong answer on the exam.

GREP: CONTENT SEARCH

Grep searches file contents for patterns. Use it when you need to find a function name across a codebase, locate error messages, discover all callers of an API, or find import statements.[3,4]

Task: "Find all usages of getUserById"
Correct: Grep("getUserById", "src/")
Wrong: Bash("grep -r 'getUserById' src/")

The wrong answer is not a failed attempt. It is a category violation. The built-in Grep tool exists precisely for this. Invoking Bash to run the shell grep command when Grep is available is the anti-pattern the exam tests repeatedly.[3]

GLOB: PATH PATTERN MATCHING

Glob finds files by name or extension patterns. Use it when you need to enumerate all TypeScript test files, find all configuration files in a subtree, or identify files matching a naming convention.[3,4]

Glob operates on file paths. Grep operates on file contents. That is the complete decision rule: reach for Glob when the question is which files exist, reach for Grep when the question is what those files contain. Bash("find . -name '*.test.ts'") is the shell equivalent of Glob, and it is the wrong tool to reach for when a purpose-built alternative exists.

READ: FULL FILE CONTENTS

Read loads the complete contents of a file. Use it when you need to understand an implementation, review a configuration, trace an import, or view any file in its entirety.[3,4]

Task: "Read the configuration file"
Correct: ***Read("config.json")***
Wrong: Bash("cat config.json")

This is the specific failure from Scenario 4. cat is a shell command. Read is a purpose-built file reading tool with proper permission handling, content formatting, and context integration. Using Bash for a job Read was designed to do is the canonical anti-pattern of this chapter.[1]

WRITE: NEW FILES FROM SCRATCH

Write creates new files. It replaces entire file content, which makes it dangerous on existing files. If the file already exists and you use Write, anything you did not include in the new content disappears.[3]

Creating a new test file is the canonical case: Write("tests/new-test.ts", content). That is correct. The equivalent shell redirect via Bash is not. The exam distinction to carry: Write is for creation, Edit is for modification. Using Write on an existing file without including all the original content destroys the content you omitted.

EDIT: TARGETED MODIFICATION

Edit makes targeted changes to existing files using unique text matching as an anchor. You provide the text to find and the replacement text. Edit leaves everything else in the file untouched.[3,4]

Edit and Write address opposite cases. Edit modifies a known span of an existing file; Write replaces the whole thing. Fixing a bug in server.ts calls for Edit("server.ts", old_text, new_text), not Write with the entire file content. Write on an existing file is destructive: anything not included in the new content disappears.

The failure mode for Edit: when the anchor text is not unique in the file, Edit cannot determine which occurrence to replace. The fallback in that case is Read the file into context, then Write the entire corrected content. Read plus Write is the documented recovery path when Edit cannot find unique anchor text.[4]

BASH: SHELL OPERATIONS

Bash runs shell commands, scripts, git operations, test suites, and build processes. Use it when no built-in tool covers the task.[3]

Running tests, executing build scripts, git commits, arbitrary system operations: these belong to Bash because no purpose-built tool handles them. npm test is Bash. git commit is Bash. Neither Read nor Grep nor Edit has anything to say about running a test suite. The Bash selection rule is not “Bash is for things that are hard.” It is “Bash is for things that have no built-in tool.” The moment a built-in tool exists, that tool wins.

Incremental Exploration

This is the named concept for this chapter: **Incremental Exploration**.

A new engineer inherits a large codebase. The naive move is to open every file and read the whole thing until something familiar appears. That is expensive, slow, and usually counterproductive. Experienced engineers do the opposite: *start at the surface, follow the signal, stop when they have enough.*[4]

Incremental Exploration is that discipline applied to agentic codebase navigation. The pattern has three moves:

1. **Grep to find entry points.** Search for the function name, the error message, the config key, the class name. Grep surfaces the files and line numbers where the thing you care about actually lives. You do not need to know the file names in advance.

2. **Read to trace the flow.** Load the specific file Grep identified. Understand the implementation. Follow the import chain to adjacent files. Read only what the evidence points to.

3. **Stop when you have enough.** The goal is not to read the whole codebase. It is to answer the question in front of you. Incremental Exploration is a discipline of restraint as much as it is a discovery method.

The exam operationalizes this pattern directly. Task Statement 2.5 describes the skill as “building codebase understanding incrementally: starting with Grep to find entry points, then using Read to follow imports and trace flows, rather than reading all files upfront.”[4]

Reading all files upfront is not wrong in the way that Bash("cat") is wrong. It is inefficient. It consumes context tokens that do not contribute to the answer. In a large codebase, it fills the context window with noise before the agent has oriented itself.

The anti-pattern has a name: *premature full-read*. You call Read on every file in the directory before you have established which files matter. Incremental Exploration replaces premature full-read with evidence-driven navigation.

THE TRACE PATTERN

One specific application of Incremental Exploration that the exam tests: tracing function usage across wrapper modules.[4]

The scenario: a codebase exports functions through multiple wrapper layers. You need to find every caller of a specific function across the whole repo. The approach: First, identify all the exported names. Use Grep to find the export statement for the function. Read the module that exports it to understand the interface.

Then, use Grep again to search for each exported name across the codebase. Each hit is a caller. Each caller is a file worth reading in full.

This is the Grep-then-Read trace. It scales. You do not need to know the codebase structure in advance. You follow the evidence.

How MCP and Built-In Tools Interact

A real configuration will have both MCP tools and built-in tools available simultaneously. The model makes selection decisions based on tool descriptions. This creates a specific tension the exam tests: an MCP tool that does the same thing as a built-in tool but with better context for the specific use case.

The exam guide notes that enhancing MCP tool descriptions to explain capabilities and outputs in detail prevents the agent from preferring built-in tools over more capable MCP tools.[4] That applies in reverse too: a built-in tool with a clear mandate (Grep for content search) should win over an MCP tool that also happens to support search, if the MCP tool description does not make its advantage explicit.

Tool selection by the model follows description quality. Vague MCP tool descriptions cede ground to well-specified built-in tools. This is the same principle Chapter 4 establishes for tool descriptions as routing keys. The application here is about cross-type competition: built-in versus MCP, decided by whoever wrote the better description.

Parallel Execution: Read-Only Versus Stateful

One operational detail worth knowing for the exam: built-in tools that are read-only (Read, Glob, Grep) can run concurrently when Claude requests multiple tool calls in a single turn. Tools that modify state (Edit, Write, Bash) run sequentially to avoid conflicts.[5]

This matters for Incremental Exploration. When tracing a large codebase, the agent can issue multiple Grep or Read calls in a single turn and receive results for all of them in parallel. Sequential reading is not forced on read-only operations.

Configuration Errors and Recovery

MCP servers can fail to connect for several reasons: the server process is not installed, credentials are missing or invalid, a remote server is unreachable.[2]

The SDK emits a system message with subtype init at the start of each query. That message includes the connection status for each configured server. Check the status field to detect failures before the agent starts working. A server in “failed” status means its tools are unavailable. If the agent then attempts to call one of those tools, it will receive a rejection rather than a result.[2]

Common failure causes and their fixes:[2]

- Missing environment variables: the env field specifies what the server expects; verify that all referenced variables are set in the runtime environment.
- Server not installed: for npx-based stdio servers, verify the package exists and Node.js is in PATH.
- Invalid connection string: for database servers, verify format and accessibility.
- Network issues: for remote HTTP/SSE servers, verify the URL is reachable and firewalls allow the connection.

Tools that are configured but not called are also worth diagnosing. If Claude can see MCP tools but does not use them, the likely cause is missing allowedTools permission. The tool is visible; the call is blocked. Adding the tool name (or a wildcard) to allowedTools resolves it.[2]

Exam Patterns: What Gets Tested

The exam draws heavily on this chapter's material. Here is how the testable concepts map to question patterns, using original constructions:

Question type: Built-in tool selection. An agent receives a task. Which tool should it use? The answer almost always eliminates Bash when a dedicated built-in exists. The decision tree is: does a purpose-built tool cover this? If yes, use it. If no, use Bash.

Question type: Exploration strategy. An agent needs to understand an unfamiliar codebase before modifying it. The correct approach starts with Grep (finding entry points) and uses Read selectively (following imports), not with reading every file upfront.

Question type: MCP configuration. A team needs to share an MCP server configuration. Which file? .mcp.json at the project root, version-controlled. Personal tools go in ~/.claude.json. Secrets use ${ENV_VAR} expansion, never hardcoded values.

Question type: Permission mode. An agent is configured with permissionMode: "acceptEdits". Can it call MCP tools without additional configuration?
No. acceptEdits approves file edits and filesystem Bash commands only. MCP tools require explicit allowedTools entries.

Question type: Write versus Edit. A task modifies an existing file. Which tool? Edit, because it preserves unchanged content. Write is for new files. Using Write on an existing file without including all the original content destroys the content you omitted.

Question type: Edit fallback. Edit cannot find unique anchor text. What is the recovery path? Read the file into context, then Write the complete corrected content.

Key Takeaways

- MCP tools follow the naming pattern mcp__<server-name>__<tool-name>. Include the exact name or a wildcard mcp__<server>__* in allowedTools to grant access. Without this, Claude sees the tool but cannot call it. permissionMode: "acceptEdits" does not auto-approve MCP tools; use allowedTools for precise MCP access control.

- Transport selection depends on where the server runs: stdio for local processes (command in documentation), HTTP/SSE for remote servers (URL in documentation), SDK MCP servers for in-process custom tools. In .mcp.json, "streamable-http" is accepted as an alias for "http".

- Never hardcode credentials in .mcp.json. Use ${ENV_VAR} syntax. The file goes to version control; secrets must not.
- .mcp.json at the project root is team-shared config. ~/.claude.json is personal. MCP tool search withholds tool definitions from context until they are needed and is on by default; the full context-cost treatment belongs to Chapter 11.
- Built-in tool selection rule: use the purpose-built tool; fall back to Bash only when no built-in covers the task. Bash("cat config.json") when Read("config.json") exists is the canonical anti-pattern.
- Grep is for content search (patterns inside files). Glob is for path patterns (finding files by name or extension). Read is for full file contents. Edit is for targeted modification using unique anchor text; when anchor text is not unique, the fallback is Read the file, then Write the corrected complete content. Write is for new files; it replaces entire content. Bash is the fallback when nothing else covers the job.
- Incremental Exploration: Grep to find entry points, Read to trace flows, stop when you have enough. Do not read the full codebase upfront. Follow the evidence.

CHAPTER 6: THE CLAUDE.MD HIERARCHY

Summary: *Claude Code's configuration system is a four-scope hierarchy (managed policy, project, user, local) that resolves through concatenation, not override. Instructions from more specific scopes are read last and carry more immediate weight, but nothing gets erased. The .claude/rules/ directory and @path/to/import syntax provide modular organization within that hierarchy. Path-scoped rules extend the system by triggering on file type rather than directory, giving teams fine-grained control without fragmented configs. The exam's most reliable trap is instructions living in the wrong scope: user-level configuration is personal and invisible to teammates.*

The Diagnostic Question

Picture it: a new engineer joins the team. They fire up Claude Code on their first real task. Claude generates code that ignores the team's established patterns. Wrong naming conventions. Wrong error handling style.

The team lead checks in. Does the project CLAUDE.md exist? Yes. Is it committed? Yes. Did the new engineer clone the repo? Yes.

The file is there. Claude just isn't following it.

This is Scenario 2 from the certified architect exam, and it is one of the more instructive traps the exam sets.[1] Because the answer is not "your CLAUDE.md is broken." The answer is: "those instructions were in ~/.claude/CLAUDE.md, the user-level file on the team lead's machine, and the new engineer's machine has no such file."

That gap, that single config placement decision, is the named concept this chapter introduces: **Three-layer config inheritance**.

The name is slightly imprecise because there are actually four scopes. But the exam tests three of them routinely, and the fourth (managed policy) is mostly an enterprise concern. The point is that these layers do not override each other. They concatenate. Instructions from a broader scope do not disappear when a narrower scope adds instructions. Everything loads. Everything compounds.

Knowing which layer wins, and when, is an exam competency.

The Four Scopes

Claude Code recognizes four locations where CLAUDE.md files can live, each with a different reach.[2]

Managed policy lives at an OS-specific path that your IT or DevOps team controls:

- *macOS: /Library/Application Support/ClaudeCode/CLAUDE.md*
- *Linux and WSL: /etc/claude-code/CLAUDE.md*
- *Windows: C:\Program Files\ClaudeCode\CLAUDE.md*

This file cannot be excluded by individual users or per-project settings. It is the organization's floor. Whatever is in it applies to every session on every machine where it is deployed. It is delivered by configuration management tooling (MDM, Group Policy, Ansible). Individual developers cannot override it, and cannot opt out of it.[2]

Project scope is ./CLAUDE.md or ./.claude/CLAUDE.md at the root of your repository. This is the team config. It travels with the repository through version control. Every developer who clones the repo gets it. Every CI job that checks out the repo has access to it. This is where team coding standards, architectural conventions, build commands, and shared workflows belong.[2]

User scope is ~/.claude/CLAUDE.md. Personal preferences. All projects, all sessions, only your machine. This is the right place for things like editor keybindings, output format preferences, and personal tooling shortcuts. It is the wrong place for anything the team needs to share.[2]

Local scope is ./CLAUDE.local.md at the project root. Gitignored by convention (add it to .gitignore explicitly; running /init and selecting the personal option does this automatically). This is personal and project-specific: sandbox URLs, specific test data you use during development, credentials for a local environment. It does not travel with the repo. It is invisible to teammates and CI.[2]

The table structure is:

Scope	File location	Shared with	In version control
Managed policy	*OS-specific system path*	All users on the machine	No (deployed via MDM/etc.)
Project	*./CLAUDE.md or ./.claude/CLAUDE.md*	Team	Yes
User	*~/.claude/CLAUDE.md*	Nobody	No
Local	*./CLAUDE.local.md*	Nobody	No (gitignored)

The exam tests whether you can correctly assign instruction types to scopes. Personal preferences do not belong in project config. Team standards do not belong in user config. The new engineer scenario is the canonical illustration: team standards in the wrong scope means teammates start fresh, without context.

How the Files Load

Understanding what “concatenation” means in practice requires knowing the load order.[2]

When Claude Code starts, it walks the directory tree from your current working directory upward. At each level, it checks for CLAUDE.md and CLAUDE.local.md. All discovered files are loaded. Content is ordered from the filesystem root down to the working directory: files at the top of the tree appear first in the assembled context; files closer to where you launched Claude appear last.

Consider a session launched from *projects/myapp/src/:*

1. CLAUDE.md at / (if it exists)
2. CLAUDE.md at /projects/
3. CLAUDE.md at /projects/myapp/
4. CLAUDE.md at /projects/myapp/src/
5. CLAUDE.local.md files appended after CLAUDE.md at each respective level

Order matters.

Within any given directory, CLAUDE.local.md is appended after CLAUDE.md. So **personal notes are always the last thing Claude reads at that level.**[2]

One clarification on what "more specific wins" means in a concatenation model: nothing gets erased. Concatenation places more-specific-scope content later in the assembled context. Because Claude reads sequentially, **later instructions carry more immediate weight when prior instructions conflict.** But it is probabilistic, not guaranteed. Vague or contradictory instructions from different layers produce ambiguous behavior. For deterministic enforcement, hooks exist (that's Chapter 3's territory).

Subdirectory files load differently. Claude Code discovers CLAUDE.md and CLAUDE.local.md files in subdirectories under the working directory, but it does not load them at launch. They are included when Claude reads files in those subdirectories during a session.[2] This is relevant behavior to know for the exam: *a*

CLAUDE.md in src/api/ is not in context when the session starts; it enters context when Claude opens a file from that directory.

An important corollary: after /compact, the project-root CLAUDE.md is re-read from disk and re-injected into the session. Nested subdirectory CLAUDE.md files are not re-injected automatically. They reload the next time Claude reads a file in that subdirectory.[2] Instructions you gave only in conversation do not survive compaction. Add them to CLAUDE.md if they need to persist.

The @import Syntax

CLAUDE.md files can pull in other files using @path/to/import syntax anywhere in the file.[2]

Both relative and absolute paths are allowed. Relative paths resolve relative to the file containing the import, not the working directory. Imported files are expanded and loaded at launch alongside the CLAUDE.md that references them. Recursive imports are supported, with a maximum depth of five hops.

The practical use case is modular configuration. A monorepo with packages serving different domains can have a project-level CLAUDE.md that imports only the standards relevant to each package, rather than loading every rule in every context:

 # .claude/CLAUDE.md
Use TypeScript with strict mode enabled.
@./rules/api-design.md
@./rules/testing.md

One thing the exam will test: imports do not reduce context. Splitting a 400-line CLAUDE.md into four 100-line files and importing all four produces the same total context as the monolith did. The organization benefit is real (teams can own individual files; review is easier; diffs are cleaner), but the context budget does not shrink.[2] For reducing context, path-scoped rules are the correct mechanism.

Importing also works for team-shared references. A team can maintain a standards/ directory with agreed-upon conventions and reference those files from CLAUDE.md with @ syntax. Works with AGENTS.md too: if a repository already uses AGENTS.md for another coding assistant, a CLAUDE.md that starts with @AGENTS.md lets both tools read the same instructions without duplication.[2]

The .claude/rules/ Directory

The ***.claude/rules/*** directory provides an *organized alternative* to a monolithic CLAUDE.md.[2,3]

Place markdown files in .claude/rules/. Each file should cover one topic, with a descriptive filename: testing.md, api-design.md, deployment.md. All .md files are discovered recursively, so rules can be organized into subdirectories like frontend/ or backend/. The rules directory supports symlinks, including links to shared rule sets maintained outside the repository.[2]

There are two kinds of rules, distinguished by YAML frontmatter.

Rules without paths frontmatter load at launch, with the same priority as .claude/CLAUDE.md. They are always in context. Use them for conventions that apply to all work in the project.

Rules with paths frontmatter are path-scoped. They load only when Claude reads files matching the specified glob patterns. They do not consume context tokens unless they are relevant to the current task.[2]

```
---
paths:
  - "**/*.test.tsx"
---
# Testing conventions for React components
```

Every test file must have a corresponding factory function for test fixtures.

This rule enters context when Claude opens any *.test.tsx* file, anywhere in the project. It does not enter context when Claude is working on infrastructure configuration files or backend services.

PATH-SCOPED RULES VS. SUBDIRECTORY CLAUDE.MD

The exam tests when to use path-scoped rules versus a subdirectory CLAUDE.md, and the distinction is meaningful.[3,4]

A subdirectory CLAUDE.md at src/api/CLAUDE.md covers everything Claude reads in that directory. A path-scoped rule with **/*.ts covers all TypeScript files regardless of which directory they are in.

If your conventions are directory-bound (this specific API module has unique auth requirements), a subdirectory CLAUDE.md is correct. If your conventions are type-bound (all TypeScript files must follow these patterns, regardless of whether they live in src/, lib/, test/, or scripts/), a path-scoped rule in .claude/rules/ is correct.

The exam scenario that tests this: a team wants to enforce TypeScript conventions on test files that are distributed throughout a monorepo. A subdirectory CLAUDE.md would require one file per directory containing tests. A path-scoped rule with **/*.test.tsx covers the entire codebase with one file.[3]

Size Guidance and What It Tells You

The documented target is **under 200 lines per CLAUDE.md file**.[2]

Longer files consume more context and reduce adherence. The adherence point is not just about tokens: CLAUDE.md content is delivered to Claude as a user message after the system prompt, not as part of the system prompt itself. Claude reads it

and tries to follow it, but there is no guarantee of strict compliance, especially for vague or conflicting instructions.[2]

This is the architectural reality of the system. It is context, not enforced configuration. The more specific and concise the instructions, the more consistently Claude follows them. “Use 2-space indentation” works better than “format code properly.” “Run npm test before committing” works better than “test your changes.” The measurable, verifiable instruction has a better compliance rate than the directive that requires judgment to apply.

Block-level HTML comments in CLAUDE.md files (<!-- maintainer notes -->) are stripped before the content is injected into Claude’s context. Use them to leave notes for human maintainers without spending context tokens on those notes. Comments inside code blocks are preserved.[2]

For instructions where compliance is genuinely required, the right mechanism is hooks, not CLAUDE.md. Hooks execute as shell commands at fixed lifecycle events and apply regardless of what Claude decides to do. Hooks are Chapter 3’s territory, but the handoff point is important: CLAUDE.md is behavioral guidance; hooks are programmatic enforcement.

User-Level Rules

The user scope applies to more than just ~/.claude/CLAUDE.md. A ~/.claude/rules/ directory works the same way as the project-level .claude/rules/ directory, but applies to every project on your machine.[2] Use it for personal preferences that are not project-specific: output verbosity, preferred code review style, debugging approaches.

User-level rules follow the same two-tier behavior: rules without paths frontmatter load universally; rules with paths frontmatter are path-scoped.

The sharing constraint applies here identically to user CLAUDE.md: personal rules in ~/.claude/rules/ are yours. Teammates do not see them.

The */memory* Command

When instructions are not being followed, the first diagnostic step is /memory.[2,4]

The /memory command lists all CLAUDE.md, CLAUDE.local.md, and rules files loaded in the current session. If a file is not in that list, Claude cannot see it. This is the fastest way to confirm that the file Claude should be reading is actually present in context.

The command also provides a link to open the auto memory folder and lets you toggle auto memory on or off. Selecting any file from the list opens it in your editor.

Common diagnostic flow: team member reports that Claude is not following the project style guide. Run /memory. If the project CLAUDE.md is not listed, either the file does not exist at a location that gets loaded for that session, or there is a claudeMdExcludes configuration blocking it. If the file is listed, check whether the instruction is specific enough. “Format code properly” will not produce consistent results; “use 4-space indentation for Python files” will.[2]

The *claudeMdExcludes* setting is relevant in large monorepos: ancestor CLAUDE.md files may contain instructions that are irrelevant to your current work. The *claudeMdExcludes* setting (available at any settings layer*: user, project, local, or managed policy*) lets you skip specific files by path or glob pattern. Managed policy CLAUDE.md files cannot be excluded.[2]

The Key Trap

Return to the opening scene: the new engineer whose Claude session ignores team standards.

The diagnostic is now clear. Run /memory on the engineer's machine. If the project CLAUDE.md does not appear, start there. If it does appear, check whether the instruction that is being ignored lives in a subdirectory CLAUDE.md that has not been triggered yet (the engineer has not opened files from that directory).

But the most common cause, and the one the exam tests directly, is this: the instruction that was supposed to apply to the team was written in ~/.claude/CLAUDE.md on the team lead's machine.[1,4]

The team lead sees correct behavior. Every other team member sees none. The file is not committed. It is not in version control. It never left the team lead's home directory.

This is the canonical user-level-versus-project-level mistake, and it has a shape that is easy to miss because the author of the instruction experiences no symptoms. Their Claude session follows the rule. They test it. It works. They move on.

The fix is mechanical: move the instruction from ~/.claude/CLAUDE.md to .claude/CLAUDE.md in the repository. Commit it. Done.

Everything the team needs to share goes in the project scope. Everything personal stays in user scope or local scope. That is the complete decision rule.

Putting It Together: A Config Anatomy

Here is the structure of a well-organized project configuration:

```
~/.claude/
  CLAUDE.md              # Personal universal prefs: vim keybindings, verbosity
  rules/
    output-format.md     # Personal rule: always summarize findings
```

```
project/
  CLAUDE.md                # Team entry point: brief, references @imports
  CLAUDE.local.md          # Gitignored: sandbox URL, local test creds
  .claude/
    CLAUDE.md              # Team config: build commands, standards, @imports
    rules/
      testing.md           # Topic-specific, no paths frontmatter: loads always
      api-design.md        # Topic-specific, no paths frontmatter: loads always
      react-components.md  # paths: ["**/*.tsx"]: loads only for .tsx files
      terraform.md         # paths: ["terraform/**/*"]: loads only for tf files
    commands/
      review.md            # /review slash command (Ch7 territory)
    skills/
      refactor/
        SKILL.md           # Refactoring skill with context: fork (Ch7 territory)

  src/
    api/
      CLAUDE.md            # Directory-level: auth validation rules for this module
```

The CLAUDE.md at project root (or .claude/CLAUDE.md) is short. It covers broad team conventions and uses @import to pull in topic-specific files. The .claude/rules/ directory carries the detail. Path-scoped rules handle file-type-specific conventions. CLAUDE.local.md handles personal project-specific context. User-level files handle personal universal preferences.

Nobody wrote 800 lines in one file. Nobody put personal preferences in the committed config. Nobody wrote team standards in their home directory.

Key Takeaways

Four scopes, one direction. Managed policy loads unconditionally for all users. Project loads from version control and applies to the team. User is personal and applies to all projects. Local is personal, gitignored, and project-specific. *More-specific scope content is read later in the assembled context.*

Concatenation, not override. All discovered CLAUDE.md files combine into a single assembled context. Nothing is erased. Conflicting instructions from different scopes produce ambiguous behavior; for guaranteed compliance, use hooks.

Load order and @import behavior. Files above the working directory load at launch; subdirectory files load when Claude reads files from those directories. @import adds context without reducing it: splitting a large CLAUDE.md into imported files produces identical context load. For reducing context, use path-scoped rules in .claude/rules/.

Path-scoped rules are about file type, not directory. A rule with paths: ["**/*.test.tsx"] applies to all matching files anywhere in the codebase. A subdirectory CLAUDE.md applies to all files in that directory. Use path-scoped rules when conventions follow file type across directory boundaries.

Size guidance exists for a reason. Target under 200 lines per file. CLAUDE.md is delivered as a user message, not enforced configuration. Longer files with vague instructions produce less reliable compliance than shorter files with specific, verifiable ones.

/memory is the diagnostic command. If an instruction is not being followed, run /memory first. If the file is not listed, Claude cannot see it. If the file is listed, the instruction is too vague or conflicts with another.

The exam's primary trap. Team instructions in user-level config (~/.claude/CLAUDE.md) are invisible to teammates. Project instructions belong in .claude/CLAUDE.md, committed to version control. This is the most commonly tested configuration mistake in Scenario 2.

Sample Questions

Q1. A team lead configured Claude Code to follow company-specific error handling conventions. After onboarding three new engineers, none of them observe Claude following these conventions. The team lead's environment works correctly. Which configuration change resolves this?

A. Move the error handling instructions from ~/.claude/CLAUDE.md to .claude/CLAUDE.md in the repository and commit the file.
B. Ask each engineer to copy the team lead's ~/.claude/CLAUDE.md to their local machine.
C. Add the instructions to CLAUDE.local.md at the project root and commit that file.
D. Configure the managed policy file to load user-level settings from a shared network location.

Correct: A. *User-level config is personal and not shared via version control. Project-level config in .claude/CLAUDE.md is shared through the repository. CLAUDE.local.md is gitignored (C is wrong). Manually syncing user config (B) is not a durable team solution and does not address the root cause.*

Q2. A monorepo contains TypeScript test files distributed across twelve subdirectories. The team wants Claude Code to apply consistent testing conventions whenever it works with any test file, regardless of directory. Which approach is most appropriate?

A. Create a .claude/rules/testing.md file with paths: ["**/*.test.ts"] frontmatter.
B. Create a CLAUDE.md in each of the twelve subdirectories containing test files.
C. Add the testing conventions to the root CLAUDE.md using @import syntax.
D. Create a .claude/rules/testing.md file without any frontmatter.

Correct: A. *Path-scoped rules with glob patterns apply by file type regardless of directory. B requires twelve files and maintenance. C loads the testing conventions for all sessions regardless of whether tests are being worked on (correct for always-applicable rules, but wasteful here). D loads at launch unconditionally, not path-scoped.*

Q3. A developer imports three large reference documents into CLAUDE.md using @ syntax to keep the file itself under 200 lines. What effect does this have on the session context?

A. The imported files are lazy-loaded only when Claude references them during the session.
B. The imported files load at launch alongside CLAUDE.md, increasing total context proportionally.
C. The imported files replace the content of CLAUDE.md in context, keeping total size the same.
D. The imported files are summarized before loading to reduce context impact.

Correct: B. *Imported files are expanded and loaded at launch. The 200-line guidance applies per file, but importing multiple files still loads all of them. @import aids organization, not context reduction.*

Q4. After a */compact* operation, a developer notices that Claude no longer follows conventions that were specified in src/api/CLAUDE.md. The project-root CLAUDE.md instructions are still being followed. What is the most likely explanation?

A. Compact operations delete CLAUDE.md files from subdirectories.
B. The project-root CLAUDE.md survives compaction and is re-injected; nested subdirectory CLAUDE.md files are not re-injected automatically and reload only when Claude next reads files from those subdirectories.
C. The src/api/CLAUDE.md instructions conflict with the project-root CLAUDE.md and were suppressed.
D. Compact operations only preserve the managed policy CLAUDE.md.

Correct: B*. After /compact, the project-root CLAUDE.md is re-read from disk and re-injected. Nested subdirectory files are not re-injected; they reload the next time Claude reads a file from that directory.*

CHAPTER 7: SLASH COMMANDS, SKILLS, AND PLAN MODE

Summary: *Claude Code exposes three distinct mechanisms for expressing intent: **slash commands** (quick, repeated actions run in the main conversation context), **skills** (reusable capability packages that can fork into isolated sub-agent contexts), and **plan mode** (a read-only exploration phase that produces a plan before a single file is touched). Each mechanism maps to a different point on the spectrum from "I know exactly what to do, just do it" to "I have no idea what this will break, let me think first." The exam tests when to reach for each, how the SKILL.md frontmatter fields (context: fork, allowed-tools, argument-hint) shape skill behavior, and how to combine plan mode with direct execution for complex multi-phase tasks. This chapter also covers the three iterative refinement techniques that appear in Scenario 2: TDD iteration, the interview pattern, and concrete input/output examples.*

The Intent Spectrum

Every request you send to Claude Code sits somewhere on a spectrum. At one end: a task so well-understood that the correct implementation is obvious and the scope is a single file. At the other end: a task with architectural implications, multiple valid approaches, and the kind of side effects that are impossible to enumerate in advance. Between those poles: work that is worth doing repeatedly, on different targets, in different projects, by different team members, where the procedure is known but you do not want to retype it every session.

These three regions of the spectrum correspond to three mechanisms.

Commands live at the quick-and-repeatable end. A slash command is a markdown file that becomes a shortcut. Type /review and the content of .claude/commands/review.md is injected into the prompt. The session does not

fork, the context is not isolated, and the tool set is not restricted. **Commands are the right tool when the action is simple, fast, and trusted to run in the main conversation** without producing exploratory noise that you will have to scroll past for the rest of the session.

Skills live in the middle. A skill is also a markdown file, but packaged differently, and the packaging matters. With the right frontmatter, a skill runs in an isolated sub-agent context. The exploration it does, the intermediate reads and greps and wrong turns, stays inside the fork. The main session receives a summary. This is the mechanism for complex, verbose workflows where the procedure is known but the execution is messy.

Plan mode lives at the deliberate end. It is not a file on disk. It is a permissionMode value: "plan". With plan mode active, Claude explores the codebase, reads files, traces call graphs, and produces a written plan, but does not edit a single file. No changes until the plan is approved. The appropriate mechanism for tasks where committing to an approach before understanding the full implications is how you create technical debt or break things at 11 PM.

The named concept for this chapter is **explicit-intent execution modes**. Each mode requires the developer to be explicit about what kind of engagement they want: fast action, reusable workflow, or deliberate planning.

Custom Slash Commands

A custom slash command is a markdown file stored in a specific directory. The filename, minus the .md extension, becomes the command name. Drop review.md into .claude/commands/ and /review becomes available in that project.[1]

There are two scopes.[1]

Project-scoped commands live in .claude/commands/. They are committed to version control and available to everyone on the team. Use this scope for

commands that encode team standards: a code review format, a commit message template, a migration checklist.

User-scoped commands live in ~/.claude/commands/. These are personal, cross-project shortcuts. They do not show up in the repo, so a teammate does not see them unless they set up their own.

The content of the file defines what the command does. Commands support dynamic arguments via placeholders, can execute bash commands and embed the output, and can include file contents using the @ prefix. The SDK dispatches a slash command the same way it sends any prompt string: the command is included in the prompt text, and the system/init message lists the commands available in the current session.[2]

What commands do not do: they do not fork the context. The command executes in the main conversation. If the command triggers exploration that produces several hundred lines of output, that output stays in the main session. For commands handling bounded, trusted tasks, this is fine. For complex exploration, it is a problem that skills exist to solve.

The current recommended format for new work is .claude/skills/<name>/SKILL.md, which supports the same slash-command invocation plus autonomous invocation by the model.[2] The .claude/commands/ directory remains valid and supported; the two formats coexist in the CLI.

Skills: The Reusable Capability Layer

A skill is a directory inside .claude/skills/ (project) or ~/.claude/skills/ (user) that contains a SKILL.md file.[3] The file has YAML frontmatter and markdown content. The frontmatter is where the interesting decisions happen.

The two scopes are not equal when they collide. A project skill in .claude/skills/ and a personal skill in ~/.claude/skills/ that share the same name do not merge

and do not both activate; the project skill takes precedence and the personal one is shadowed. This has a direct operational consequence the exam tests. When a developer wants to customize a team skill for their own workflow, copying it to ~/.claude/skills/ under the same name does not work: the project version still wins, and the personal customization never runs. The fix is to give the personal version a different name, for example a personal /my-commit alongside the team's /commit. Both are then discoverable, and the model selects between them on description. This mirrors the precedence rule for agent definitions covered in Chapter 2, where a programmatically defined agent overrides a filesystem agent of the same name: same-name collisions resolve by scope precedence, not by merge.

DISCOVERY AND INVOCATION

When Claude Code starts, it reads skill metadata from the filesystem. The model sees each skill's description and decides autonomously when to invoke it based on that description. A well-written description is a routing key: specific, keyword-rich, honest about what the skill does and under what conditions. A vague description produces unpredictable invocation.[3]

Skills can also be invoked explicitly by name in a prompt. Both paths work. Autonomous invocation is powerful for skills that should activate naturally during normal work. Explicit invocation is appropriate when you know exactly which capability you need.

In the SDK, the skills option on query() controls which skills are available. Pass "all" to enable every discovered skill, a list of names to enable only those, or [] to disable all.[3]

THE THREE FRONTMATTER FIELDS

Three frontmatter fields determine skill behavior in CLI usage. Understanding all three is exam-critical.

context: fork

This is the most important field. When a skill has context: fork in its frontmatter, the skill runs in an isolated sub-agent context. The main conversation does not see the intermediate steps: the greps, the reads, the exploratory dead ends.[4] The main session receives the summary output from the skill, not the full transcript of everything the skill did to produce it.

The alternative is a skill without context: fork, which runs in the main context just like a command. For a skill that reads one file and produces a compact answer, this is fine. For a skill that explores a large codebase, brainstorms multiple approaches, and discards three of them before settling on one, running in the main context pollutes the session with noise the developer has to mentally filter for the rest of the conversation.

(Do not confuse this with session forking. The context: fork frontmatter field isolates a skill's exploration in a sub-agent context within a single invocation. The --fork-session CLI flag, covered in Chapter 2, branches a whole session transcript so two investigation paths can diverge from a shared prior state. Same word, unrelated mechanisms.)

Consider a refactoring skill that needs to understand the full dependency graph before making any changes. Without context: fork, every grep result, every file read, every intermediate analysis lives in the main context. The developer ends up in a session where finding the actual changes requires scrolling through hundreds of lines of exploration. With context: fork, the skill does all of that inside the fork, and the main session sees only the final refactored output and a brief summary of what changed.[4]

allowed-tools

The allowed-tools frontmatter field restricts which tools the skill can use during execution.[4] A refactoring skill that only needs to read and edit files should list only Read, Edit, and Grep. Leaving tool access unrestricted when running a skill in a forked context means the skill could, in principle, execute arbitrary bash commands

or write to unexpected locations. Restricting the tool set is defense-in-depth: the scope of what the skill can do is made explicit in the file that defines it.

One critical constraint: allowed-tools in SKILL.md frontmatter applies to CLI usage only. It does not apply when skills are used through the SDK.[3] In SDK usage, tool access is controlled through the main allowedTools option in the query configuration. This is an exam-tested distinction.

argument-hint

When a skill requires a parameter to operate correctly, argument-hint tells Claude Code what to prompt for if the skill is invoked without arguments.[4] A refactoring skill needs to know which file or directory to refactor. An argument-hint like "file or directory to refactor" surfaces this requirement at invocation time instead of letting the skill proceed with missing input and produce an error partway through.

A CONCRETE SKILL.MD STRUCTURE

Here is the frontmatter shape for a refactoring skill, drawn directly from the Domain 3 documentation:[4]

```
---
context: fork
allowed-tools:
 - Read
 - Edit
 - Grep
argument-hint: "file or directory to refactor"
---
```

The body of the file follows as markdown: the skill’s instructions, rules, and any patterns the model should follow. This content becomes the system prompt for the forked sub-agent.

Skills vs Commands: The Decision

The distinction the exam tests is sharper than “use skills for complex things.” The decision criterion is whether execution produces output or exploratory context that would degrade the main session.[4]

Use a **command** when: - The action is quick and bounded. - The output is compact and directly useful in the main conversation. - No exploration phase is required. - The task is simple enough that running it in the main session does not add noise.

Use a **skill with context: fork** when: - The task involves exploration before action: reading many files, tracing dependencies, brainstorming and discarding approaches. - The execution produces verbose output that would pollute the main conversation context. - Tool access should be restricted to only what the task requires. - The same capability will be reused across sessions or by multiple team members with different arguments.

The anti-pattern in Scenario 2 is using a command for complex codebase exploration. The command runs in the main context, fills it with intermediate analysis, and leaves the developer debugging their conversation to find the actual output.[5]

Skills vs CLAUDE.md: The Other Decision

Skills and CLAUDE.md both hold instructions. The distinction is load timing.

CLAUDE.md files load every session, regardless of what task is being done. They are the right location for universal standards: coding conventions, testing requirements, commit message formats, things that apply to every Claude Code session in the project.[4]

Skills load on demand, when invoked. They are the right location for task-specific workflows that are needed sometimes but not always: a refactoring procedure, a

migration checklist, a security audit protocol. Putting these in CLAUDE.md means every session loads instructions for tasks that session may never perform. That is wasted context budget and, for large instruction sets, a noise problem.

The practical rule: if an instruction should govern every Claude Code interaction in the project, put it in CLAUDE.md. If it is a procedure for a specific recurring task, make it a skill. The two mechanisms compose: a skill can reference project standards from CLAUDE.md and apply task-specific logic from its own content.

Plan Mode

Plan mode is the deliberate end of the intent spectrum. It corresponds to permissionMode: "plan" in the SDK. (The semantics of all permissionMode values are established in Chapter 1; this chapter addresses when and why to use "plan" specifically.)

In plan mode, Claude explores the codebase using read-only tools. It reads files, searches with Grep, traces imports, builds a mental model of the system. Then it produces a plan describing what changes it would make and why. It does not write a single file. No edits, no deletions, no shell commands that modify state.[6]

The developer reads the plan, asks questions, requests adjustments, and approves the approach before any execution begins. This is the safe exploration and design phase that prevents costly rework on complex changes.

WHEN TO USE PLAN MODE

The exam provides concrete criteria for plan mode selection. Use plan mode for tasks with:[6]

- **Architectural implications**: restructuring a microservice's internal boundaries, redesigning how modules communicate, changing a fundamental data flow.

- **Multi-file modifications**: changes that touch many files across the codebase, where missing a callsite or an interface definition causes runtime failures.
- **Multiple valid approaches with different infrastructure requirements**: choosing between two integration strategies, each requiring different dependencies and deployment configurations.

The rationale is that tasks matching these criteria have too many unknowns to execute directly. Direct execution commits to an approach before the full scope is understood. Plan mode surfaces the unknowns during the read-only phase, when course-correcting is free.

WHEN NOT TO USE PLAN MODE

Direct execution is appropriate for well-understood changes with clear scope.[6] A single-file bug fix with a clear stack trace is the canonical example from the exam guide: the problem is identified, the location is known, the fix is obvious. Using plan mode here is overhead with no benefit. The developer waits for an exploration-and-planning phase on a task that needs no exploration.

The anti-patterns are symmetric. Always using plan mode is wasteful overhead on simple tasks. Never using it is reckless on complex ones.[5]

THE EXPLORE SUBAGENT

For multi-phase tasks that involve a verbose discovery phase followed by implementation, the Explore subagent prevents context window exhaustion.[6] The pattern: delegate the discovery phase to an Explore subagent. The subagent reads files, traces dependencies, and builds a complete picture of the current state. The main agent receives a summary of what was found, not the full exploration transcript. The main agent's context stays lean for the implementation phase.

This is the subagent-for-context-management pattern applied specifically to codebase exploration. The main agent never needs to see every intermediate grep

result; it needs the conclusions. The Explore subagent produces the conclusions; the exploration transcript stays inside the subagent context and is never returned to the parent.[6]

COMBINING PLAN MODE AND DIRECT EXECUTION

The practical pattern for large tasks: use plan mode to investigate, then switch to direct execution to implement the approved plan.[6] Consider a library migration affecting dozens of files. Plan mode surfaces all the callsites, identifies the compatibility shims needed, and outlines the migration sequence. The developer approves the sequence. Direct execution then implements the plan file-by-file with a known target state. The investigation is thorough; the implementation is bounded.

Iterative Refinement Techniques

Three refinement techniques are tested in Scenario 2 and assigned to this chapter. They address the challenge of communicating expected behavior precisely enough that Claude produces consistent, verifiable output across iterations.

THE TDD ITERATION PATTERN

Test-driven development as an iterative refinement strategy means writing the test before the implementation. The test is the specification.[7]

The cycle:

1. Write a test that specifies the desired behavior, including edge cases and error conditions. The test should fail, because the implementation does not exist yet.

2. Run the test. It fails. This confirms the test is actually checking something.

3. Ask Claude to implement the function to pass the tests.

4. Run the tests again. They should pass.

5. Refine the implementation (performance, error handling, additional edge cases) while keeping all tests green.

The power of this pattern is verification. “Implement this feature” produces an implementation; the developer then needs to evaluate whether the implementation is correct. “Make these tests pass” provides a concrete, runnable definition of correct. Each iteration has a binary outcome: passing or failing. There is no ambiguity about whether progress was made.

This dramatically outperforms vague instructions like “make it better” because the goal is measurable at each step.[5]

THE INTERVIEW PATTERN

Before implementing a solution in an unfamiliar domain, have Claude ask questions. Specifically: have Claude surface the design considerations the developer may not have anticipated before committing to any implementation approach.[7]

Cache invalidation strategies. Failure modes under partial network failure. The behavior when a downstream service returns an unexpected status code. Race conditions in concurrent update scenarios. A developer new to a domain does not know which of these questions are load-bearing until they have already built the wrong thing.

The interview pattern makes these questions explicit before any code is written. Claude asks; the developer answers; the implementation that follows is grounded in answered questions rather than assumptions.[7]

This is most valuable in unfamiliar domains. In well-understood domains where the developer has mental models of all the edge cases, the interview phase adds overhead without proportional value. The technique is calibrated to uncertainty.

CONCRETE INPUT/OUTPUT EXAMPLES

When a prose description of expected behavior produces inconsistent results, the most effective technique is to provide concrete examples.[7] Not “transform the input into a normalized format,” but three pairs of input and expected output, covering the typical case, an edge case, and a failure case.

The exam guide specifies two to three examples as the effective range.[7] Too few examples leave ambiguous cases unresolved. Too many examples shift the problem from specification to enumeration.

The technique is most valuable when the transformation has boundary cases that prose descriptions handle poorly. Natural language descriptions of format normalizations, data extraction rules, and classification criteria tend to underspecify edge case behavior. Examples make the edge cases explicit in the most direct way possible: here is the input, here is the expected output.

INTERACTING VS INDEPENDENT ISSUES

One more refinement principle: when multiple issues exist in an implementation, whether to address them all in a single message or sequentially depends on whether the fixes interact.[7]

If fixing issue A changes the code in a way that affects the correct fix for issue B, they are interacting issues. They must be addressed together in a single, detailed message that describes both issues and the interaction. Sequential iteration, where the developer asks Claude to fix issue A and then separately asks it to fix issue B, risks producing a second fix that is correct in isolation but incorrect given the first fix.

If fixing issue A and fixing issue B are completely independent, they can be fixed sequentially. Sequential iteration on independent issues is preferable because each iteration is scoped, verifiable, and easier to review.

The practical question before sending a message about multiple issues: if Claude fixes one of these, does the correct fix for the others change? If yes, combine them. If no, sequence them.

Composing the Mechanisms

Commands, skills, and plan mode compose because they occupy different layers: commands trigger action, skills package reusable capability, and plan mode gates execution behind explicit approval. The exam tests whether you can identify which layer a given requirement belongs to. CI/CD is where these mechanisms face their hardest test: no human available, session isolated, output machine-parsed. That is Chapter 8.

Sample Questions

Q1. A developer defines a codebase analysis procedure in .claude/commands/analyze.md. Running /analyze fills the main session with hundreds of lines of intermediate grep results and file reads. The actual analysis summary is hard to find. What is the most appropriate fix?

A. Move the analysis instructions to CLAUDE.md so they load with lower priority.
B. Migrate the command to a skill in .claude/skills/analyze/SKILL.md and add context: fork to the frontmatter.
C. Add --compact to the command file to limit output length.
D. Break the command into multiple smaller commands that each run separately.

Correct answer: B. The context: fork field runs the skill in an isolated sub-agent context. Intermediate exploration stays in the fork; the main session receives only the summary.[4,5]

Q2. A team needs to migrate a library used in 45+ files across the codebase. Multiple migration strategies are possible, each requiring different dependency changes. What is the appropriate first step?

A. Direct execution, asking Claude to migrate files one at a time.
B. Plan mode (permissionMode: "plan"), allowing Claude to explore all callsites and produce a migration plan before any files are modified.
C. A slash command that iterates through files in alphabetical order.
D. The TDD iteration pattern, writing migration tests first.

Correct answer: B. The task has multi-file scope and multiple valid approaches with different infrastructure implications. These are the exact criteria for plan mode. Direct execution commits to an approach before the full scope is understood.[6]

Q3. A developer invokes a refactoring skill but does not provide the target file. The skill produces an error partway through execution. Which frontmatter field would have prevented this?

A. context: fork
B. allowed-tools
C. argument-hint
D. description

Correct answer: C. argument-hint prompts the developer for required parameters when the skill is invoked without arguments.[4]

Q4. A team's CLAUDE.md contains a 40-step security audit procedure that applies only when auditing authentication modules. The rest of the time, the procedure is irrelevant but adds to the context budget every session. What is the correct fix?

A. Move the procedure to .claude/rules/ with a paths glob pattern matching authentication modules.
B. Create a skill in .claude/skills/security-audit/SKILL.md containing the procedure.
C. Split the CLAUDE.md into two files and import the relevant one manually.
D. Reduce the procedure to a summary to stay under the 200-line CLAUDE.md limit.

Correct answer: B. Skills load on demand. CLAUDE.md loads every session. Task-specific procedures belong in skills, not CLAUDE.md.[4]

Q5. A developer finds that Claude produces inconsistent output for a data normalization function. The natural language description of the normalization rule is technically accurate but produces different interpretations across attempts. Which technique is most effective?

A. The interview pattern: have Claude ask questions about the normalization requirements.
B. Two to three concrete input/output examples demonstrating the expected transformation, including edge cases.
C. Plan mode, to allow Claude to design the normalization approach before implementing.
D. Increasing specificity of the prose description until all edge cases are covered.

Correct answer: B. Concrete input/output examples are the documented most-effective technique when prose descriptions produce inconsistent results. The exam guide specifies two to three examples as the effective range.[7]

Key Takeaways

- Custom slash commands are markdown files in .claude/commands/ (project-scoped) or ~/.claude/commands/ (user-scoped). They run in the

main conversation context and are best for quick, bounded, repeatable actions. Skills are directories containing SKILL.md files in .claude/skills/ (project) or ~/.claude/skills/ (user); the model invokes them autonomously based on description, or they can be invoked explicitly by name.

- The three SKILL.md frontmatter fields: context: fork isolates skill execution in a sub-agent context; allowed-tools restricts tool access during skill execution (CLI only, not SDK); argument-hint prompts for required parameters when the skill is invoked without arguments.

- Use skills with context: fork for complex exploration or analysis that produces verbose output. Use commands for simple, fast repeated actions. Skills load on demand; CLAUDE.md loads every session. Task-specific procedures belong in skills; universal project standards belong in CLAUDE.md.

- Plan mode (permissionMode: "plan") is for tasks with architectural implications, multi-file scope, or multiple valid approaches. Direct execution is for well-understood changes with clear scope. The Explore subagent handles verbose discovery phases so the main agent receives a summary, not the full exploration transcript.

- The TDD iteration pattern: write the test first (defines the goal), run it (confirms it fails), implement to pass, run again, refine while keeping tests green.

- The interview pattern: have Claude ask questions before implementing in unfamiliar domains, to surface design considerations the developer has not anticipated.

- Concrete input/output examples (two to three) are the most effective technique when prose descriptions produce inconsistent results. For

multiple issues: combine interacting fixes in one message; sequence independent fixes.

CHAPTER 8: CLAUDE CODE IN CI/CD

Summary: *Running Claude Code in a CI/CD pipeline requires a different mental model than running it interactively. There is no human available to answer permission prompts, approve tool use, or intervene when something hangs. The configuration choices that are optional in a terminal session become load-bearing in a pipeline. This chapter covers the flags, the session architecture, and the cost controls that make Claude Code a reliable CI participant: the **-p** flag for non-interactive mode, **--output-format json** with **--json-schema** for machine-parseable output, **permissionMode: "bypassPermissions"** for unattended execution, session context isolation as the core architectural principle, and the Message Batches API for latency-tolerant workloads. The central anti-pattern is same-session self-review: a generation session asked to review its own output retains all prior reasoning context and cannot deliver an independent assessment. Review-session isolation, where the reviewing Claude invocation is a completely separate process with no shared session ID, is the architectural fix. The **--bare** flag enforces reproducibility by disabling discovery of local hooks, skills, and MCP servers, so pipeline behavior is consistent across all machines in the fleet.*

The Scenario That Broke a Pipeline

Consider a CI pipeline built to run automated code review on every pull request. The pipeline works like this: first, Claude Code generates implementation code in one step. Then, in the very next step, the pipeline calls Claude Code again in the same session to review what was just written.

The results look fine. Claude approves the code. Issues that a human reviewer would catch slip through. The generator reviewed its own work. It retained all the reasoning context from generation: the intent, the tradeoffs, the justifications it

had already made internally. Asking the generator to review its own output in the same session is not a second opinion. It is a confirmation.[3]

The pipeline called *--resume* with the original session ID, thinking continuity was a feature. *It was the bug. The reviewer "approves" code it just wrote because it has already committed to the reasoning that produced that code.*

The fix is architectural. The fix is what this chapter calls **review-session isolation**.

The Flag That Everything Else Depends On

Start here. Before session isolation, before structured output, before any of the CI-specific configuration matters: Claude Code, without modification, waits for interactive input. It presents prompts, waits for human responses, and blocks the pipeline indefinitely.[1]

The **-p** flag (equivalently **--print**) switches Claude Code into non-interactive mode. Add it to any claude command. The process runs, outputs its result to stdout, and exits. Without it, a CI job will hang until the runner hits its timeout limit and kills the job with a generic error that tells you nothing useful about what actually went wrong.[1]

claude -p "Review this PR diff for missing error handling"

That is the minimum viable CI invocation.
Everything else in this chapter builds on it.

The documentation is explicit: all other CLI options work with -p.[1] The flag does not remove functionality. It removes the assumption of human presence.

BARE MODE FOR REPRODUCIBILITY

A related concern in CI is reproducibility. By default, claude -p loads the same context an interactive session would: hooks, skills, plugins, MCP servers, auto memory, CLAUDE.md files from the working directory and ~/.claude.[1] A hook in a

teammate's personal ~/.claude directory, or an MCP server in the project's .mcp.json, will run if present.

The ***--bare*** flag **disables all of that discovery.** Only flags passed explicitly take effect. The same invocation produces the same result on every machine in the fleet, regardless of what individual developers have configured locally.[1]

claude -p --bare "Summarize the build log" --allowedTools Read

Bare mode is recommended for scripted and SDK calls. The documentation notes it will become the default for -p in a future release.[1]

Structured Output in CI

Natural language output is not a CI artifact. The pipeline needs something it can parse, route, and post as inline PR comments without a fragile regex in the middle.

--output-format json produces structured JSON. The response payload includes the result, the session ID, usage statistics, and a per-model cost breakdown.[1] The cost field means scripted callers can track spend per invocation without consulting a dashboard.

Combined with *--json-schema*, the response enforces a specific output shape. The structured result appears in the structured_output field of the response.[2]

A concrete CI invocation looks like this:

```
claude -p "Review this PR diff for:
  1. Functions exceeding 50 lines
  2. Missing error handling on async operations
  3. Hardcoded credentials or API keys
  4. Missing unit tests for new functions
Provide results as structured JSON." \
  --output-format json \
  --json-schema '{
    "type": "object",
    "properties": {
```

```
      "issues": {
        "type": "array",
        "items": {
          "type": "object",
          "properties": {
            "file": {"type": "string"},
            "severity": {"type": "string"},
            "description": {"type": "string"}
          }
        }
      }
    }
  }'
```

The downstream script receives a JSON object, not text. It can iterate over issues, filter by severity, and post each one as a comment against the relevant file.[2]

Review-Session Isolation

This is the named concept this chapter introduces, and it is worth being precise about what it means and why it matters.

A Claude Code session retains context. The session that generated a module knows why certain decisions were made. It knows what alternatives were considered and rejected. It carries the internal justifications for every line it wrote. When that same session is asked to review the code, it does not start fresh. It starts with all of that prior reasoning in context, which creates confirmation bias: the reviewer is less likely to question decisions it already made.[2,3]

Review-session isolation means: the session that reviews code must be completely separate from the session that generated it. Not --resume. Not --continue. A **new invocation**, with no session ID linking it to the generator.[2]

(This is the review-specific exception to a general capability. For when session resumption and forking are correct, and the decision rules governing resume-versus-fresh, see the session-state section in Chapter 2.)

The correct CI architecture:

```
# Step 1: Generation (Session A)
claude -p "Implement the auth module per the spec" --output-format json

# Step 2: Review (Session B — completely separate invocation, no --resume)
claude -p "Review this diff: $(git diff main)" --output-format json --json-schema '...'
```

The generator and reviewer see the same code. The reviewer has none of the generator's reasoning context. That is the point.[2,3]

The exam scenario presents this as a tricky tradeoff (doesn't continuity help?) but there is no tradeoff here. Same-session self-review is an anti-pattern with no compensating benefit. An independent instance without prior reasoning context is more effective at catching subtle issues than any self-review instruction.[3]

RE-RUNNING REVIEWS AFTER NEW COMMITS

Isolation applies to fresh reviews. When re-running a review after new commits address earlier feedback, the architecture shifts slightly. A completely fresh session with no context about prior findings will flag the same issues again. That produces duplicate comments on already-addressed problems and trains developers to ignore the review system.[3]

The correct approach: **include the prior review findings in context, and instruct Claude to report only new or still-unaddressed issues**.[3]

```
PRIOR_FINDINGS=$(cat .ci/prior-review.json)
claude -p "Prior review findings: $PRIOR_FINDINGS

Review this updated diff and report ONLY issues that are new or were not addressed
 in the prior findings." \
 --output-format json \
 --json-schema '...'
```

The reviewer still runs in an isolated session. But it is given enough context to know what has already been flagged. The isolation principle is preserved; the duplicate-comment problem is not.

Multi-Pass Review

Review-session isolation solves the bias problem: the reviewer does not share the generator's reasoning context. Multi-pass review solves a different problem: attention dilution across files.

The failure signature is distinctive. A single-pass review of a large changeset (ten or more files in one invocation) produces inconsistent depth: obvious issues caught in one file are missed in another, the same anti-pattern is flagged in file three and approved without comment in file seven, and the reviewer's thoroughness visibly decays as context fills. The root cause is that the model's attention is spread across too many files simultaneously, and the later files receive whatever capacity remains after the earlier files consumed their share.

The fix is structural decomposition of the review itself into two layers. Per-file local passes: each file gets its own focused review invocation, examining internal logic, style violations, and local correctness in isolation. Then a separate cross-file integration pass: a final invocation that receives the per-file findings plus the dependency graph and checks for issues that span boundaries (data-flow mismatches, inconsistent error-handling conventions, API contract violations between modules, import cycles introduced by the changeset as a whole). The local passes catch depth issues; the integration pass catches breadth issues. Neither alone covers both.

This is orthogonal to the multi-instance pattern described above. Three review architectures exist for three distinct failure modes, and the exam expects candidates to distinguish them cleanly. Multi-instance review (independent reviewer, no shared context) corrects for generation-context bias: the generator's retained reasoning would corrupt a same-session review. Multi-pass review (per-file local passes plus cross-file integration) corrects for attention dilution across files: a single pass over many files loses depth. Second-pass self-critique, the

evaluator-optimizer pattern in Chapter 12, corrects for per-case quality variance: the same draft checked against a completeness rubric before delivery. Different failures, different architectures. The stem tells you which failure is in play; match the architecture to it.

CLAUDE.md in CI

A CI-invoked session loads CLAUDE.md from the repository root just as an interactive session would (unless --bare is passed).[4] This is the mechanism for providing project context to the reviewer: testing standards, fixture conventions, review criteria, coding patterns the team has agreed on.

Without it, the reviewer operates with only what the prompt contains. That produces generic feedback disconnected from the project's actual standards. With a well-constructed CLAUDE.md, the reviewer knows the project uses pytest fixtures in a specific directory structure, that all async functions require explicit error handling on network calls, and that functions exceeding 50 lines trigger a mandatory review comment.[3]

A CI-specific CLAUDE.md section might look like this:

```
## CI Review Standards
- Flag any function exceeding 50 lines
- Flag missing error handling on all async operations
- Flag hardcoded strings matching API key or password patterns
- Do not suggest tests for scenarios already covered by files in /tests/unit/
- Testing fixtures are in /tests/fixtures/ — reference them in test suggestions
```

The last two points address a common problem in test generation: Claude suggesting tests for scenarios already covered, or generating tests that reinvent fixtures the project already provides.[3] Document what exists. The CI session will use it.

Permission Mode for CI

Interactive Claude Code runs with permission prompts. The process pauses, asks whether to proceed, and waits. In CI, there is no human to answer. The process blocks, the runner times out, and the job fails with no useful information.

permissionMode: "bypassPermissions" is the recommended permission mode for CI and container environments where no human is available for approval.[5] With it, Claude Code proceeds without pausing for permission checks. All tool use runs automatically.

This is a known value defined in Chapter 1. CI is the use case it was designed for. The tradeoff is real: bypassing permissions means Claude can read, write, and execute without intervention. In a CI environment with known inputs (a PR diff, a set of source files) and bounded tool access (defined by --allowedTools), this is acceptable. In an environment with ambiguous scope, it requires more care.

The combination that appears in production CI configurations:

```
- name: Run Claude Code Review
  run: |
    claude -p "$REVIEW_PROMPT" \
      --output-format json \
      --json-schema "$SCHEMA" \
      --allowedTools "Read,Grep" \
      --max-turns 10
  env:
    ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
```

--allowedTools scopes which tools the session can invoke. Combined with bypassPermissions, it creates an unattended session that can only do what you explicitly permit.

GitHub Actions Integration

The Claude Code GitHub Actions integration wraps the CLI in a workflow step. The key parameters:[4]

- **claude_args:** passes CLI arguments to the underlying claude invocation. This is how --max-turns, --model, --allowedTools, --output-format, and --json-schema reach the process.
- **ANTHROPIC_API_KEY**: stored as a repository secret, referenced as *${{ secrets.ANTHROPIC_API_KEY }}*.

```yaml
name: Claude Code Review
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Claude Code Review
        uses: anthropics/claude-code-action@v1
        with:
          prompt: |
            Review this PR for the issues defined in CLAUDE.md.
            Output structured findings only.
          claude_args: --max-turns 10 --output-format json
          anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
```

The action handles the -p flag internally. The prompt parameter is the instruction; claude_args is where turn limits and output format live.[4]

--max-turns matters in CI. An unbounded agentic loop is a runaway job. Set it explicitly to a value appropriate for the complexity of the review task. Ten turns is a reasonable default for a standard PR review; adjust based on observed behavior.[4]

CLAUDE.md in the repository root guides Claude's behavior without requiring it to be embedded in the workflow file. The separation is clean: the workflow file handles infrastructure; CLAUDE.md handles standards.[4]

The Message Batches API

Some CI workloads are not blocking. A nightly technical debt report, a weekly security audit, test coverage analysis run overnight: none of these block a developer who is waiting to merge. They are **latency-tolerant**. They have a deadline measured in **hours**, not seconds.

The Message Batches API processes requests asynchronously with up to a 24-hour window and offers a 50% cost reduction versus synchronous API calls.[3,5] For latency-tolerant workloads, that is a direct operational cost reduction with no functional tradeoff.

Four things to know about the batch API:[5]

Processing window: Results arrive within 24 hours. There is no guaranteed latency SLA within that window. The batch may complete in minutes; it may take close to the full 24 hours. Plan accordingly.

custom_id: Each request in a batch carries a custom_id field. The response for each item carries the same custom_id, which is how the caller correlates responses to their originating requests. Without custom_id, a batch of 500 files becomes 500 unordered responses with no way to map them back.[5]

No multi-turn tool calling: The batch API does not support multi-turn tool calling within a single request. A batch request cannot execute a tool mid-request and return results to continue reasoning. This rules out batch for agentic tasks that

require tool interaction. It is appropriate for single-turn workloads: analyzing a file, generating a summary, producing structured output from provided content.[5]

Failure handling: When individual requests in a batch fail, resubmit only the failed items by custom_id. Do not resubmit the entire batch. Items may fail for different reasons (context length exceeded, transient errors); handle each failure type appropriately before resubmission.[5]

THE DECISION: BATCH VS SYNCHRONOUS

The decision rule is simple: *if a developer or a deploy process is waiting for the result, use **synchronous**.* If the result is consumed *hours later* by a scheduled process, use ***batch***.

Pre-merge checks are synchronous. A developer has opened a PR and is waiting to see if the review passes before merging. Switching that workflow to batch, then polling for completion, does not save money if the batch takes longer than the developer is willing to wait. It just adds latency and complexity for no benefit.[5]

Nightly reports are batch. Nobody is waiting. The report appears in the morning. The 50% cost reduction applies. The 24-hour window is not a constraint because the workload runs overnight anyway.[5]

The exam question in the guide makes this exact distinction: a blocking pre-merge check and an overnight technical debt report. The proposal is to switch both to batch for the cost savings. The correct answer is to switch only the overnight report. The pre-merge check stays synchronous because it is a blocking workflow.[5]

Configuration Checklist for CI

Putting the pieces together, a production CI configuration for Claude Code review should have:

- **-p flag on every invocation.** Without it, the job hangs.

- **--output-format json for machine-parseable results.** Natural language output is not a CI artifact.
- **--json-schema** when the downstream consumer requires a specific structure. Pair it with --output-format json.
- A **separate session** for every review. No --resume, no --continue from the generation session. Review-session isolation is not optional.
- **ANTHROPIC_API_KEY via secrets.** Never hardcoded in workflow files.
- **--max-turns to prevent runaway jobs.** Set it explicitly.
- **CLAUDE.md at the repository root with project context:** testing standards, fixture locations, review criteria, patterns the team uses.
- **permissionMode: "bypassPermissions"** when the session needs unattended tool access, scoped by --allowedTools.
- **Prior review findings in context** when re-running after new commits. Instruct the session to report only new or unaddressed issues.
- **Message Batches API for latency-tolerant workloads.** Synchronous for blocking checks.

What the Exam Tests Here

The exam does not test whether the candidate can write a GitHub Actions workflow file. It tests whether the candidate understands the architectural reasoning behind each configuration choice.

The -p flag question (Question 10 in the guide) is designed to catch candidates who confuse this with environment variables or stdin redirection. The flag is the

documented mechanism. The other options do not exist or do not address the root problem.[5]

The session isolation question is designed to catch candidates who think continuity aids review. It does not. A reviewer that shares reasoning context with the generator is not reviewing; it is confirming.[2,3]

The batch API question is designed to catch candidates who apply cost optimization indiscriminately. The 50% savings are real. They do not apply to blocking workflows where a human is waiting for the result.[5]

Three facts drive most of the CI/CD scenario questions:

1. -p for non-interactive mode.
2. Separate sessions for code review (review-session isolation).
3. Message Batches API for latency-tolerant workloads; synchronous for pre-merge checks.

The configuration choices here assume the prompt is precise enough to make review findings actionable. Chapter 9 is where that assumption gets tested.

Key Takeaways

- The -p (or --print) flag is required for CI; without it, the process hangs waiting for interactive input. permissionMode: "bypassPermissions" is the appropriate mode for CI and container environments; scope it with --allowedTools to limit which tools run unattended.
- --output-format json combined with --json-schema produces machine-parseable structured output that downstream pipeline steps can consume directly, without fragile text parsing.

- **Review-session isolation** is the core architectural principle for CI code review: the session that reviews code must be completely separate from the session that generated it. Same-session self-review retains generator reasoning context and produces confirmation bias.
- When re-running a review after new commits, include prior findings in context and instruct Claude to report only new or unaddressed issues. This prevents duplicate comments without sacrificing session isolation.
- CLAUDE.md at the repository root provides project context (testing standards, fixture locations, review criteria) to every CI-invoked session that does not use --bare.
- GitHub Actions integration passes CLI arguments via claude_args. ANTHROPIC_API_KEY goes in repository secrets, not workflow files.
- The ***Message Batches API* offers 50% cost savings** with a 24-hour processing window and no guaranteed latency SLA. It is appropriate for latency-tolerant workloads (overnight reports, weekly audits) but not for blocking pre-merge checks, and it does not support multi-turn tool calling. The ***custom_id* field correlates batch requests to their responses**; without it, batch responses cannot be mapped back to their source files.

CHAPTER 9: PROMPTING FOR PRECISION

Summary: *The exam tests prompt engineering as specification writing. A precise prompt eliminates ambiguity by stating explicit acceptance criteria, providing format constraints, and decomposing complex requests into enumerated steps. Vague instructions produce vague outputs; explicit categorical criteria produce consistent, auditable outputs. Two techniques anchor this chapter: the shift from vague directives to explicit categorical criteria (the named concept), and few-shot prompting as the tool for communicating reasoning about ambiguous cases. The alert-fatigue pattern shows how noisy, low-precision review tools destroy developer trust across all rule categories, not just the noisy ones. The sequencing insight is that high false-positive categories should be disabled while their prompts are repaired, rather than running them during revision; this protects the credibility of accurate categories while improvements are made. Output format is not stylistic preference when review output feeds downstream automated tooling: it is a system contract, and the format constraint belongs in the prompt with the same precision as the content criteria.*

The Tool That Cried Wolf

The CI pipeline runs code review on every pull request. The prompt says: *"Review this code for quality issues. Be thorough and flag anything suspicious."*

The reports come back. Hundreds of findings. Duplicate variable names in test files, functions that could theoretically be more efficient, missing JSDoc on internal utility functions that the team deliberately chose not to document. Developers scan the first three findings, realize they're noise, and close the tab.

Two weeks later, someone merges a PR with a hardcoded API key. The review flagged it. Nobody read far enough to see it.

This is the scenario that anchors Task Statement 4.1 in the exam guide.[1] And it is not a story about the model performing badly. The model did exactly what the prompt asked. It was thorough. It flagged suspicious things. The problem is that "thorough" and "suspicious" are not specifications. They are intentions. The model fills in the blanks with its own judgment, and its judgment does not match the team's priorities.

The fix is not to ask the model to "be more selective." That is still not a specification.

The fix is to write a specification.

The Noise Problem Has a Name

When a code review tool flags too many non-issues, developers stop reading the reports. Including the real findings. The exam guide calls this alert fatigue, and it is directly tested.[1,2]

Alert fatigue has a compounding property that makes it worse than it first appears. It is not just that developers ignore the noisy category. High false-positive categories undermine developer trust in accurate categories.[1] If the style section of the report is reliable garbage, the instinct is to distrust the security section too. The tool's overall credibility collapses, and it stops functioning as a safety mechanism even in the areas where it actually works.

The natural first response to a noisy tool is to add a confidence qualifier: "only report high-confidence findings," or "be conservative." This feels like a fix. It is not a fix.[1] Confidence qualifiers are still vague. The model has no calibrated ground truth for what "high confidence" means relative to this codebase, this team's conventions, or this specific review context. The instruction adds words without adding precision.

The correct response is **Explicit Criteria Over Vague Instructions**: replace intentional directives with categorical, measurable conditions.

That is the named concept for this chapter. Say it again: explicit criteria over vague instructions. Not “be thorough.” Not “be conservative.” A list of things to flag, with conditions precise enough that a second model (or a human) could verify whether each finding is correctly categorized.

What Explicit Looks Like

The exam guide provides the contrast directly.[1,2] Compare these two prompts for code review:

 # VAGUE
Review this code for quality issues.
Be thorough and flag anything suspicious.

 # EXPLICIT
Review this code and flag ONLY the following:
1. Functions exceeding 50 lines of code
2. Async operations missing try-catch error handling
3. Hardcoded strings matching API key patterns (sk-, pk-, key-)
4. Public functions missing JSDoc documentation
5. SQL queries constructed with string concatenation

For each issue found, provide:
- File path and line number
- Which rule (1-5) was violated
- Severity: critical (3, 5) | warning (1, 2) | info (4)
- One-line fix suggestion

The vague prompt delegates judgment. The explicit prompt provides a contract.

Notice what the explicit version accomplishes that vague instructions cannot:

Rule 1 (“Functions exceeding 50 lines”) is checkable. The model either finds a function longer than 50 lines or it does not. There is no ambiguity about what counts. A human reviewer can verify any flagged finding in seconds.

Rule 3 (“Hardcoded strings matching API key patterns”) specifies the patterns explicitly. The model is not guessing what an API key looks like. It is matching against sk-, pk-, key-. Edge cases that used to require judgment (“is this a test key?”) can be addressed by extending the pattern list, not by hoping the model’s intuition holds.

Severity is categorical. Not “rate severity 1-10” (which the model will interpret differently on different runs), but a fixed mapping: rules 3 and 5 are critical, rules 1 and 2 are warnings, rule 4 is info. The classification is auditable. Downstream tooling can filter on severity reliably.

The output format is also part of the specification. Four required fields per finding, in a consistent structure. This is not stylistic preference. When the review runs in CI and the output feeds a PR comment bot (as in Scenario 5), the format is a contract that downstream code depends on.[3]

The Alert Fatigue Mitigation Pattern

A development team ships a code review tool with six rule categories. Three of them work well: the security rules, the null-check rules, and the async error-handling rules. Three categories are noisy: style conventions that don’t match the team’s actual patterns, documentation checks that flag intentionally undocumented internal utilities, and complexity metrics calibrated for a different codebase.

The noisy categories do not stay in their lane. They erode trust in the good categories, and after a few weeks, developers have stopped reading the reports. The fix seems obvious: **fix the noisy prompts.**

Fixing prompts takes iteration. While the iteration is in progress, the noisy categories are still running and still burning credibility. The exam guide specifies the mitigation: temporarily disable high false-positive categories while improving prompts for those categories, rather than attempting all categories simultaneously.[1]

This is a sequencing insight, not just a prompt engineering insight. Production systems depend on trust. If the noisy categories are running, they are spending trust budget that the good categories need. Disabling them is not an admission of failure; it is protecting the signal while the noise is eliminated.

The pattern: **identify which categories have high false-positive rates, disable those categories, restore developer trust in the remaining output, then bring each disabled category back after the prompt has been revised and tested.**

Do not attempt to fix all categories simultaneously. Simultaneous repair is slower overall because the whole tool's credibility stays low during the repair period. Sequential repair, category by category, preserves credibility for the working categories throughout.

Severity Criteria Are Specifications Too

One of the skills tested in Task Statement 4.1 is defining explicit severity criteria with concrete code examples for each severity level.[1]

Severity labels without definitions are vague instructions. "Critical," "warning," and "info" sound precise, but they are not. What makes a finding critical versus a warning? If the answer is "judgment," then severity will vary across runs and across different versions of the same model. Downstream tooling that auto-closes warnings but pages on-call for criticals will misfeed.

The specification pattern is the same as for rule criteria: replace intentional labels with conditions. Not "critical means serious security issues" but "critical: findings from rules 3 and 5 (hardcoded credentials and SQL injection vectors)." The severity

is determined by which rule fired, not by a separate judgment call. A finding cannot be “warning severity” if it matched rule 3, regardless of how the model might have otherwise assessed its impact.

When severity must involve a contextual judgment, the specification should include examples. Not definitions, examples. “This is a critical finding” paired with a code snippet that exhibits the pattern. “This is a warning” paired with a different snippet. Examples communicate the decision boundary more precisely than prose descriptions because they anchor the label to a concrete case. This is the same mechanism as few-shot prompting, applied to the severity axis of the output.

Few-Shot Prompting: When and How Many

Few-shot examples are the most effective technique for achieving consistently formatted, actionable output when detailed instructions alone produce inconsistent results.[1]

The exam guide is specific about count: 2-4 targeted examples is the documented optimal range for ambiguous scenarios.[1,2] Not 5-8, not 10. Providing ***too many*** examples (***more than 6***) bloats the prompt without adding value.[2] Providing too few for genuinely ambiguous cases leaves the model without enough signal to generalize.

Where few-shot examples earn their cost is in tasks with ambiguous boundaries. When the boundary between “flag this” and “skip this” requires contextual judgment that prose descriptions cannot fully specify, examples show the reasoning. Not just the outcome, the reasoning.[1]

This is a critical distinction. An example that shows input and correct output teaches the model the answer for that specific case. An example that shows input, correct output, and the reasoning for why that output was chosen (“this is a genuine issue, not a local pattern, because the same function exists in 14 other files with the same missing handler”) teaches the model how to approach similar cases it has not seen.

The exam guide surfaces three specific things few-shot examples should demonstrate:[1]

Consistent output format. Every example should use the same output structure. If the desired output is a JSON object with four fields, every example should show a JSON object with those four fields. The model generalizes the structure from the examples; inconsistent examples produce inconsistent output.

Acceptable patterns vs genuine issues. A finding that looks like a problem but is actually a deliberate local convention is one of the hardest judgment calls for a code review tool. An example that shows this case explicitly, with the reasoning that identifies it as acceptable, teaches the model to distinguish the two rather than defaulting to one or the other.

Ambiguous boundary cases. Provide at least one example where the correct action is not obvious from the rule text alone. Show the example, show the correct classification, and show why. The model uses this to calibrate its judgment on novel patterns that fall near the same boundary.

Consider the code review context. The rule is “flag functions exceeding 50 lines.” But a file has a 55-line function that is a configuration constant block, not logic. Should it be flagged? A rule that says “functions exceeding 50 lines” technically says yes. But the intent was to catch complex functions, not initialization blocks. An example that shows this case and explains the decision teaches the model to apply the rule’s intent, not just its literal text.

Example 3 (Edge case: initialization block):
Input:
File: src/config/defaults.js, line 47
Function: DEFAULT_RETRY_CONFIG (58 lines)
Content: Large object literal assigning default values

Output:

```
{"flagged": false,
 "reasoning": "Rule 1 targets complex logic exceeding 50 lines.
 This function is a configuration constant (object literal, no
 branches, no state). Flagging it adds noise without identifying
 actual maintainability risk. Local pattern: configuration
 defaults are intentionally verbose in this codebase."}
```

That example does three things: shows the input, shows the correct output, and explains why this case falls outside the rule's intent even though it technically matches the rule's text. A model that has seen this example can generalize to other initialization blocks, documentation generators, and similar patterns without requiring a new rule for each.

When Few-Shot Is Unnecessary

Few-shot examples are not universally valuable. They are expensive in tokens, and for tasks with unambiguous boundaries, they add overhead without improving output quality.[2]

The test: is the task boundary clear from the rule text alone? If the answer is yes, explicit criteria are sufficient and examples add cost without benefit. "*Flag SQL queries constructed with string concatenation*" is largely unambiguous. String concatenation in a SQL context is a recognizable pattern with few edge cases. A clear rule without examples will produce consistent output.

The test for few-shot: would a careful human reader, given only the rule text, arrive at the same classification for genuinely ambiguous cases? If yes: skip the examples. If no: the examples are doing real work.

Few-shot is valuable in extraction tasks too, particularly when documents have varied structures.[1] Consider a system extracting citations from research documents. Some documents use inline citations (author, year, in parentheses mid-sentence). Others use footnotes. Others use numbered bibliography entries that appear at the end. Instructions alone cannot reliably handle the variety. Examples

showing correct extraction from each structure teach the model the invariant (what to extract) across varying surface presentations.

The principle is the same as the code review case: examples communicate the decision boundary in contexts where prose descriptions generate inconsistent behavior.

Decomposition as Specification

Explicit criteria work at two levels: the criteria themselves, and the structure of the request.

A prompt that says “review this file for security issues, quality problems, and documentation gaps” asks the model to simultaneously manage three different classification tasks, each with its own criteria, while also producing output in a consistent format. This is not impossible, but it creates unnecessary competition between the sub-tasks.

The alternative: decompose the request. Task Statement 4.1’s skill list includes writing specific review criteria that define which issues to report (bugs, security) versus which to skip (minor style, local patterns) rather than relying on confidence-based filtering.[1] That decomposition is already implicit in the numbered rule list format: each rule is a discrete, independently evaluable criterion.

Explicit decomposition also makes prompt iteration easier. When a category is noisy, you can revise rule 4’s criteria without touching rules 1-3. When a category is missing findings it should catch, you can extend the definition of rule 3 without affecting the severity logic. The structure isolates the knobs.

This connects to how the exam frames iterative refinement for interacting versus independent issues: when fixes interact, provide them together; when they are independent, handle them sequentially.[4] The same logic applies to prompt revision. Independent criteria can be tuned independently. Criteria that share ambiguous boundary regions should be revised together.

The Format Constraint Is Not Optional

One pattern in exam questions about prompt precision: separating the criteria from the output format.[1] It is easy to write detailed criteria and then leave the output format vague. The model will produce plausible-looking output. But "plausible-looking" is not "consistently structured," and downstream tooling requires the latter.

The output format should specify: - What fields are present in each finding - What values are acceptable for categorical fields (severity, rule number) - What the finding omits (when "no issues found" is itself a valid output, specify what that looks like)

The code review example above includes all three. The finding has four fields. Severity values are enumerated. By implication, if no issues match rules 1-5, the correct output is an empty findings list, not a paragraph explaining that the code looks generally good.

Structured output via tool_use (covered in Chapter 10) enforces the format constraint at the schema level. For prompts that do not use tool_use, the format constraint belongs explicitly in the prompt, specified with the same precision as the content criteria.

What the Exam Tests

The exam tests prompt engineering as specification writing. The scenarios it uses to frame these questions are overwhelmingly from the code review context, which is the canonical case for false positives, developer trust, and the contrast between vague and explicit.[1,3]

The patterns the exam tests are narrow and specific:

Explicit over vague. "Flag functions exceeding 50 lines" beats "flag long functions." "Flag comments only when claimed behavior contradicts actual code behavior"

beats “check that comments are accurate.”[1] Every exam question that offers a vague vs explicit option: pick explicit.

Confidence qualifiers fail. “Be conservative” and “only report high-confidence findings” do not improve precision compared to specific categorical criteria.[1] Every exam option that proposes confidence-based filtering as the fix for false positives: discard it.

Disable, then fix. When a category has high false-positive rates, disable it temporarily while revising the prompt, rather than trying to reduce false positives while the category keeps running.[1] This is the sequencing principle.

2-4 few-shot examples. The documented optimal range. For ambiguous boundary cases, include the reasoning. For output format, make the examples consistent. Not 5-8, not 1.[1,2]

Examples show reasoning. A few-shot example that shows input and output without reasoning teaches pattern matching. An example that shows input, output, and why teaches generalization.[1] The exam distinguishes these.

Practice Questions

Q1. A CI pipeline runs code review with the prompt “identify any code quality issues and flag them with appropriate severity.” The team reports the tool flags too many minor style issues and developers have stopped reading the reports, causing critical security findings to be missed. Which change most directly addresses the root cause?

A) Add “be conservative and only flag high-confidence issues” to the prompt.

B) Replace the vague instruction with a numbered list of specific rule categories and explicit severity mappings for each.

C) Reduce the number of files sent to the review per run.

D) Add a confidence score field to the output and filter on scores above 0.8.

Correct: B. A and D add confidence-based filtering, which the exam guide explicitly identifies as failing to improve precision. C reduces scope but does not fix the underlying vague instruction. B replaces intentional directives with categorical specifications.

Q2. A code review tool has six rule categories. Three produce accurate, trusted findings. Three generate consistent false positives. The team wants to restore developer trust in the tool overall while improving the noisy categories. What is the correct sequencing?

A) Improve all six categories simultaneously before re-enabling any of them.

B) Disable the three noisy categories, restore trust in the three accurate categories, then revise and re-enable the noisy categories one at a time.

C) Reduce the number of examples in the few-shot set for the noisy categories.

D) Add “skip minor style issues” to the prompt for all six categories.

Correct: B. The exam guide specifies temporarily disabling high false-positive categories while improving prompts for those categories. A delays the trust restoration. C addresses few-shot quantity, not the underlying criteria. D adds a vague modifier that does not produce categorical precision.

Q3. A team is implementing few-shot prompting for a code review tool that must distinguish between “genuine issues” and “local patterns the team has intentionally adopted.” They are considering 4, 6, or 10 examples. The examples for the ambiguous boundary cases should include what, in addition to input and output?

A) A severity score for each example, to calibrate severity classification.

B) Reasoning that explains why the output was chosen over plausible alternatives.

C) Additional examples from adjacent rule categories to improve generalization.

D) The file path and line number of the example to improve format consistency.

Correct: B. The exam guide specifies that ***examples for ambiguous cases should show reasoning for why one action was chosen over plausible alternatives.*** *A adds severity calibration, which is a different concern. C increases the example count toward the 6+ range that bloats prompts. D addresses output format, not the ambiguous-case decision boundary.*

Choosing the Intervention

Every Domain 4 scenario question hands you a failing system and four plausible fixes. The fixes are not interchangeable. Each one repairs a different *class* of failure, and the exam's wrong answers are almost always the right intervention applied to the wrong failure class. The skill being tested is not "*know what few-shot is*." It is *"read the failure signature and pick the matching tool*." The chapters teach these tools in isolation because the domains are weighted separately. The exam does not respect that separation, so this section collapses the boundary.

There are five interventions in play across Domains 1, 2, and 4. They map to five distinct failure signatures.

The Five Interventions

Intervention	Repairs	Failure signature in the stem
Explicit criteria	Undefined standard of what counts as a violation	"check that X is accurate," vague directive, both false positives and false negatives at once
Few-shot examples	A single consistent pattern the model fails to recognize, or a mapping too complex to verbalize in prose	Clean accuracy drop between two task variants the model otherwise handles well; "interprets prose differently each time" on a fixed transformation
Decompose / parallelize / restructure	A workflow that is structurally wasteful	Inflated tool-call counts, redundant data fetching, sequential investigation of independent concerns, resolution-rate collapse with high round-trips
Self-critique / evaluator-optimizer	Output that is correct but whose quality varies unpredictably per case	"gaps vary by case," "inconsistently explains," omissions that differ case to case, no fixed pattern to the defect
Prompt for a latent native capability	The model can already do the right thing but isn't	Sequential tool calls the model could batch; behavior the model supports natively but has not been instructed to use

The validation loop (Chapter 10) is a sixth tool, orthogonal to these: it catches *semantic* errors (valid structure, wrong values) and belongs to application code, not

the prompt. It is never the answer to a prompt-engineering failure, and it is never substituted by any of the five above.

The Classifier

Run the stem through these questions in order. Stop at the first match.

1. Is the model failing because no clear standard of *what counts as a violation* exists? The prompt names a goal ("be accurate," "be thorough") but never defines the boundary. → **Explicit criteria.** Examples cannot substitute for a missing definition; they teach instances of an undefined rule and fail to generalize.

2. Is the workflow *structurally* wasteful? The stem quotes tool-call counts, round-trips, redundant fetching, or sequential handling of independent work. → **Decompose / parallelize / restructure.** This is a quantitative-efficiency failure. No amount of prompt content reshapes the workflow.

3. Does the model already natively support the correct behavior and simply isn't using it? → **Prompt for the capability.** Do not build infrastructure (composite tools, preprocessing model calls, speculative execution) for a behavior one prompt line unlocks.

4. Is the output *correct* but inconsistent in quality, with the defect *varying by case* in no fixed pattern? → **Self-critique / evaluator-optimizer.** A per-output check against a rubric catches whatever gap appears in that specific case; a finite example set cannot enumerate a moving target.

5. Is there a *single, consistent* pattern the model fails to recognize, or a transformation mapping too complex to express in prose, where the model otherwise performs well? → **Few-shot examples.** This is the only cell few-shot owns.

Few-shot is the default wrong answer when the failure is definitional (1), structural (2), capability-latent (3), or variable-quality (4). It is correct only in case 5.

Few-Shot Elimination Triggers

Specific stem language that *rules few-shot out*:

- "the specific context gaps vary by case" / "inconsistently" / "differ from case to case" → variable quality → self-critique, not few-shot.
- "averages N+ tool calls" / "redundant" / "sequentially" / "round-trips" → structural → decompose, not few-shot.
- "check that X is accurate" / "be conservative" / "be thorough" → undefined criterion → explicit criteria, not few-shot.
- "Claude already supports" / "natively" / "in separate sequential turns even when both are needed" → latent capability → prompt for batching, not few-shot or composite tools.

Specific stem language that *rules few-shot in*:

- A clean accuracy split between two variants of the same task ("94% on single-concern, 58% on multi-concern") where the model handles the easy variant well → recognition gap → few-shot.
- "interprets the requirements differently each time" on a *fixed* transformation target whose mapping is hard to state in prose → few-shot beats rewriting the prose more precisely.

Minimal Pairs

The pairs below are near-identical in surface topic and opposite in correct answer. The discriminator is printed in the last column. Memorize the discriminators, not the scenarios.

Pair 1: Multi-concern customer request

	Scenario	Correct	Discriminator
A	Agent handles single-concern at 94%, multi-concern at 58%; mixes up parameters, addresses only one concern	Few-shot	The agent already executes well; it fails to *recognize* the multi-concern pattern. Recognition gap, single consistent pattern.
B	Complex cases average 12+ tool calls, 54% resolution; investigates concerns sequentially, re-fetches customer data per concern	Decompose + parallelize + shared context	Quantitative-efficiency collapse. The workflow *shape* is wasteful. Few-shot cannot restructure it.

Same domain, same "multi-concern" framing. A is a recognition gap (few-shot); B is a structural gap (restructure). The numbers tell you which: B quotes tool-call counts and redundancy, A quotes a clean accuracy split.

Pair 2: Few-shot vs self-critique

	Scenario	Correct	Discriminator
A	Resolutions technically correct but explanations inconsistent; sometimes omits policy, sometimes timeline, sometimes next steps; gaps vary by case	Self-critique (evaluator-optimizer)	Defect is *variable*. No finite example set covers a moving target. A per-output rubric check does.
B	Agent fails a single consistent pattern it doesn't recognize	Few-shot	Defect is a *fixed* pattern. Examples teach it directly.

	Scenario	Correct	Discriminator
	(e.g., multi-concern routing)		

The trigger word is "vary." When the omission varies case to case, few-shot is eliminated and self-critique wins.

Pair 3: Schema vs few-shot

	Scenario	Correct	Discriminator
A	Output structure varies; fields nested differently; model interprets prose transformation differently each iteration	Few-shot	The *mapping* is wrong, not the container. A schema validates form, not transformation logic. Comprehension gap, not verification gap.
B	Downstream parser breaks on malformed output; missing fields, wrong types	Schema (strict: true)	The *form* is wrong. Schema enforces form at the grammar level.

Schema fixes syntax; few-shot fixes the transformation. They are not substitutes. A schema will happily validate a wrongly-transformed-but-type-correct output.

Pair 4: Explicit criteria vs few-shot

	Scenario	Correct	Discriminator
A	Prompt says "check comments are accurate"; flags acceptable patterns, misses genuinely stale	Explicit criteria	The *criterion is undefined.* "Accurate" has no boundary. Examples paper over a missing definition and fail to generalize to novel contradictions.

	Scenario	Correct	Discriminator
	comments		
B	Criterion is clear; the mapping or boundary is hard to verbalize	Few-shot	The definition exists; examples calibrate the fuzzy edge.

When the underlying rule is undefined, define it first. Few-shot built on a vague instruction is building on sand.

The One-Sentence Rule

Few-shot owns exactly one failure class: a consistent, recognizable pattern (or hard-to-verbalize mapping) the model fails to apply despite performing well otherwise. Every other failure class, definitional, structural, capability-latent, or variable-quality, has a different owner, and selecting few-shot for those is the exam's most common engineered wrong answer.

(For the multi-pass review architecture that decomposes a large changeset into per-file passes plus an integration pass, see Chapter 8. Multi-pass review corrects for attention dilution, not for any of the five intervention failures above; it is a decomposition of the review task itself.)

Key Takeaways

- **Explicit criteria over vague instructions** is the named concept: categorical, measurable conditions (“flag functions exceeding 50 lines”) replace intentional directives (“flag long functions” or “be thorough”). The exam tests this contrast in every prompt engineering question.

- **Alert fatigue** is the documented mechanism by which false positives destroy tool credibility. High false-positive categories undermine trust in accurate categories, not just in themselves.

- **Confidence qualifiers** ("be conservative," "only report high-confidence findings") fail to improve precision. They are vague instructions. The exam treats them as wrong answers.

- **Mitigation sequencing**: temporarily disable high false-positive categories while improving their prompts, rather than running noisy categories while attempting repairs. Restore trust in working categories first.

- **Severity criteria** require the same explicit specification as rule criteria. Categorical mappings ("rules 3 and 5 are critical") are auditable and consistent; contextual severity judgments are not.

- **Few-shot examples**: 2-4 targeted examples is the documented optimal range. Most valuable for tasks with ambiguous boundaries. Examples should show reasoning for ambiguous-case choices, not just input and output. Include examples of acceptable patterns vs genuine issues to teach the distinction rather than just the outcome.

- **Format constraints and output format as contract**: format specifications belong in the prompt with the same precision as content criteria. When review output feeds automated tooling (PR comment bots, severity-based routing), format consistency is a system requirement, not a stylistic preference; inconsistent output format is a downstream contract violation in CI contexts.

CHAPTER 10: STRUCTURED OUTPUT AND THE VALIDATION LOOP

Summary: ***tool_use*** *mode is the contract mechanism for machine-parseable output. When a tool definition includes a JSON schema, the model is constrained by compiled grammar to produce output that matches that schema structurally. This is a hard guarantee: no JSON.parse() errors, no missing required fields, no malformed types. But structural compliance is not semantic correctness. A line-item sum that does not match the stated total is structurally valid JSON. A date field containing a plausible-looking but wrong date is structurally valid JSON. The model filled the containers correctly and put the wrong things inside.*

The production pattern for reliable structured extraction is a three-phase loop: generate using a schema-constrained tool call, validate semantics in application code, retry with specific error feedback if validation fails. Retry specificity is not optional. Generic "there were errors, try again" gives the model no signal about which constraint was violated. "Line items sum to 847.50 but stated_total is 900.00, discrepancy of 52.50" gives the model something it can act on.

The chapter also covers the two complementary structured output features (output_config.format and strict: true), schema design patterns that prevent hallucinated values, the documented complexity limits that trigger 400 errors, property ordering behavior, and the streaming incompatibility that catches developers by surprise.

Scene: The Invoice That Balanced Itself

Scenario 6 in the exam is a structured data extraction pipeline. The developer built it correctly, by most measures. The tool schema is well-designed. The call uses tool_choice to force the extraction tool every time. The response comes back as

valid JSON matching the schema exactly. No parse errors. No missing fields. The validator checks the JSON structure and returns clean.

The invoice goes into the database. The line items sum to $847.50. The stated_total field in the document was $900.00. The system stored the discrepancy and nobody caught it because the validator only checked structure.[1]

This is the central fact about tool_use as a structured output mechanism: it is a JSON contract, not a correctness contract. **The schema defines what shape the output must take. It says nothing about whether the values inside that shape are accurate representations of the source document.** Those are two completely different guarantees, and conflating them is the most consistent anti-pattern the exam tests in Domain 4.

The named concept for this chapter: **tool_use as JSON contract**.

Two Features, One System

Structured outputs in the Claude API are built from two complementary features that can be used independently or together.[2]

The first is output_config.format. This constrains Claude's response itself (the final text output of the messages call) to a JSON schema. Set type: "json_schema" and provide a schema definition; the model is constrained via compiled grammar to produce output matching that schema in `response.content[0].text`. This is useful when the extraction result is the full response, not a tool call.

The second is strict: true on tool definitions. When a tool definition includes strict: true, the API guarantees schema validation on the tool's inputs. The model's tool_use block will always contain JSON matching the declared input_schema for that tool. No extra fields, no missing required fields, no type violations.

Both features use constrained decoding: the API compiles the schema into a grammar that limits which tokens the model can produce at each position. The model is not “trying harder” to comply; the token space available to it at each step is mechanically constrained by the grammar.[2]

Used together, they cover different surfaces: output_config.format governs the response-level JSON when the model finishes its tool use loop; strict: true governs each individual tool invocation during that loop. A workflow that needs guaranteed tool inputs AND a guaranteed structured final synthesis can combine them in a single request.

The incompatibility to know: output_config.format and streaming cannot be used together.[2] Streaming delivers tokens as they are generated. Constrained decoding operates on the complete grammar, so the structured output guarantee requires the full response to be assembled before delivery. Choose one or the other.

Forcing the Tool Call

A tool definition with a schema does not automatically produce structured output. The model still chooses whether to call the tool or respond conversationally. Set tool_choice: "any" to guarantee the model calls a tool rather than returning text.[3]

tool_choice: "any" tells the model it must use one of the provided tools, but leaves the choice of which tool to the model. The behavioral mechanics of "any" and forced tool selection (including assistant-turn prefilling) are defined in Chapter 4; the application here is guaranteeing a tool call in extraction pipelines where a conversational response would be a failure mode. The model goes directly to the tool invocation.[3]

For extraction pipelines where the document type is known in advance, force the specific tool:

```
response = client.messages.create(
    model="claude-opus-4-7",
    tools=[extract_invoice_tool],
    tool_choice={"type": "tool", "name": "extract_invoice"},
    messages=[{"role": "user", "content": f"Extract: {invoice_text}"}]
)
```

When the document type is unknown and multiple extraction schemas exist, *tool_choice: "any"* lets the model select the appropriate schema while still guaranteeing a tool call happens.[4]

The tool_choice values themselves *("auto", "any", forced tool, "none")* are defined in Chapter 4. The concern here is narrower: for structured extraction, "*auto*" is the wrong default because it allows the model to decide that a conversational response is more appropriate than a tool call. If the extraction pipeline depends on always getting structured JSON back, "any" or forced tool is not optional.

Schema Design: Preventing the Model From Inventing Answers

The schema is not just a shape declaration. It is an instruction to the model about what it should and should not fill in. Get the design wrong and the model fills required fields with plausible-sounding fabrications rather than admit the source document doesn't contain the information.

Three patterns from the exam guide address this directly.[4]

Make fields optional (nullable) when the source may not contain the information. A field marked required tells the model it must produce a value. If the source document genuinely lacks that value, the model will invent one that fits the type constraint. An invoice without a purchase order number should produce

"po_number": null, not a guessed PO number. Mark the field optional and let null be a valid answer:

```
{
  "type": "object",
  "properties": {
    "vendor_name": {"type": "string"},
    "invoice_number": {"type": "string"},
    "po_number": {"type": ["string", "null"]},
    "total": {"type": "number"}
  },
  "required": ["vendor_name", "invoice_number", "total"],
  "additionalProperties": false
}
```

Use "unclear" as an enum value for genuinely ambiguous fields. When the model cannot determine the correct value from the source document, it needs somewhere to put that uncertainty. Without an explicit ambiguity option, it picks the most plausible enum value anyway. Adding "unclear" as a valid enum value gives the model a truthful option:

```
"payment_terms": {
  "type": "string",
  "enum": ["net_30", "net_60", "immediate", "unclear"]
}
```

Use "other" plus a detail field for extensible categorization. Enums with a fixed set of values will misclassify documents that don’t fit any category. The "other" plus detail pattern handles this:

```
"document_type": {
  "type": "string",
  "enum": ["standard_invoice", "credit_note", "proforma", "other"]
```

```
},
"document_type_detail": {
  "type": "string",
  "description": "Required if document_type is other"
}
```

The document_type_detail field is optional in the schema but operationally required when document_type is "other". The prompt instructions should state this explicitly, because JSON Schema cannot express conditional requirements of this form in a way that constrained decoding enforces.

These three patterns share a common goal: reduce the pressure on the model to produce a confident answer when the source material does not support one. The schema should make it easy to say “I don’t know” or “this doesn’t fit.”

Property Ordering: The Behavior Nobody Reads About Until It Breaks Something

The structured output documentation specifies property ordering behavior that is not intuitive.[2]

Properties appear in the output in schema order, with one modification: required properties appear first (in their schema order), then optional properties (in their schema order), regardless of how the properties are ordered in the schema definition itself.

Consider this schema:

```
{
  "type": "object",
  "properties": {
    "notes": {"type": "string"},
```

```
    "name": {"type": "string"},
    "email": {"type": "string"},
    "age": {"type": "integer"}
  },
  "required": ["name", "email"],
  "additionalProperties": false
}
```

notes is defined first in the properties object, but it is optional. The output will order properties as: *name, email, notes, age.* The required fields appear first even though they are not listed first in the schema.

If property order in the output matters to downstream parsing logic, mark all fields as *required* (or account for this reordering in the parser). If the required/optional split is architecturally meaningful for the schema but the output consumer needs a specific order, the consumer must sort after parsing.

If order matters, mark everything required.

The Validation Loop

The schema guarantee eliminates an entire class of failures: JSON syntax errors, missing required fields, wrong types. What it cannot eliminate is the class of failures that live above the schema level. Values in the wrong fields. Sums that don't add up. Dates formatted correctly but extracted from the wrong part of the document. Vendor names from the footer instead of the header.

These are semantic errors. The JSON is valid. The schema is satisfied. The data is wrong.

Application code is the only place semantic validation can happen. The validation loop is the production pattern for catching and correcting these failures.[4]

The loop structure:

```
def extract_with_validation(document, max_retries=3):
    messages = [{"role": "user", "content": f"Extract: {document}"}]

    for attempt in range(max_retries):
        response = client.messages.create(
            model="claude-opus-4-7",
            tools=[extract_tool],
            tool_choice={"type": "tool", "name": "extract_invoice"},
            messages=messages,
        )

        data = parse_tool_response(response)
        errors = validate_semantics(data)

        if not errors:
            return data

        # Append SPECIFIC errors and retry
        messages.append({"role": "assistant", "content": response.content})
        messages.append({
            "role": "user",
            "content": (
                f"Validation failed. Fix these specific errors and re-extract:\n"
                + "\n".join(f"- {e}" for e in errors)
            )
        })

    raise ExtractionError(f"Extraction failed after {max_retries} attempts")
```

The critical word in that structure is "*specific*." The retry message must tell the model which field failed, what the constraint is, and what the actual vs expected value is. "*There were validation errors, please try again*" gives the model no signal. It will produce the same extraction or a random variation of it.

Compare these two retry messages:

Wrong: *"The extraction had errors. Please try again."*

Right: *"Line items sum to 847.50 but stated_total is 900.00. Either the line items are incomplete or the stated_total was extracted incorrectly. Re-examine the document and correct the extraction."*

The right version tells the model which field failed, what the arithmetic shows, and where to look. The model can act on that. It cannot act on "there were errors."[4]

The semantic validation function is application-specific, but a few patterns appear in extraction pipelines reliably:

```
def validate_semantics(data):
    errors = []

    if data["total"] <= 0:
        errors.append(f"Total must be positive, got {data['total']}")

    if "line_items" in data and data["line_items"]:
        calculated = sum(item["amount"] for item in data["line_items"])
        if abs(calculated - data["subtotal"]) > 0.01:
            errors.append(
                f"Line items sum to {calculated:.2f} but subtotal is "
                f"{data['subtotal']:.2f}"
            )

    if not re.match(r"\d{4}-\d{2}-\d{2}", data.get("date", "")):
```

```
        errors.append(f"Date must be ISO 8601 format, got: {data.get('date')}")
    return errors
```

Numeric consistency checks, date format verification, referential integrity between fields. None of these can be enforced by JSON Schema in a way that constrained decoding will catch at generation time. They are logic constraints, not type constraints.

Self-Correction Fields: Building the Check Into the Schema

The validation loop is reactive: *extract, check externally, retry if wrong*. A complementary approach is to build the self-check into the schema itself, so the extraction produces both the answer and the material for verifying it.[4]

The stated_total vs calculated_total pattern:

```
{
  "type": "object",
  "properties": {
    "stated_total": {
      "type": "number",
      "description": "The total amount as stated in the document"
    },
    "calculated_total": {
      "type": "number",
      "description": "Sum of all line item amounts"
    },
    "totals_match": {
      "type": "boolean",
      "description": "True if stated_total equals calculated_total
within 0.01"
    },
    "conflict_detected": {
      "type": "boolean",
      "description": "True if any extracted values are internally
inconsistent"
    }
```

```
  },
  "required": ["stated_total", "calculated_total", "totals_match",
"conflict_detected"]
}
```

By requiring the model to extract both the stated total and compute the sum of line items, the downstream validator does not need to re-sum the line items. The totals_match flag is set by the model during extraction. When conflict_detected is true, the application routes the document for human review or triggers a targeted retry.

This approach shifts some of the validation work into the extraction pass itself. The model is doing arithmetic and consistency checking as part of filling the schema, rather than requiring a separate validation pass in the application. The tradeoff is schema complexity: more fields, higher chance of hitting complexity limits.

The detected_pattern field serves a different purpose: it captures metadata about the extraction itself rather than the document content.[4]

```
"detected_pattern": {
  "type": "string",
  "enum": ["standard_layout", "multi_page", "table_heavy",
"narrative_heavy", "unclear"],
  "description": "The structural pattern of the source
document"
}
```

When developers review extraction results and dismiss findings as incorrect, the detected_pattern field allows systematic analysis: are dismissals concentrated in documents with a particular structural pattern? If so, few-shot examples covering that pattern (as noted in Chapter 9) or schema adjustments for that pattern are the fix. Without detected_pattern, dismissal analysis requires manual review of source documents.

(For the decision framework that distinguishes schema fixes from few-shot and from other interventions when an extraction pipeline fails, see the intervention classifier in Chapter 9.)

When Retries Are Not the Answer

Retries address format mismatches and structural errors. They do not address missing information.

If the source document does not contain the data required by the schema, retrying will not produce it. The model will either fabricate a plausible value (bad), return null for optional fields (acceptable), or fill the field with content from somewhere else in the document (bad and hard to detect). More retries, more specific error messages: none of it helps when the constraint is the source material itself.[4]

Distinguish these two failure modes:

Retryable: The model extracted the date in MM/DD/YYYY format but the schema requires ISO 8601. The date is in the document. The extraction got the format wrong. A retry with "date must be YYYY-MM-DD format" will fix this.

Not retryable: The schema requires a purchase order number, but this vendor does not use PO numbers and the document has none. The model extracted something from the document that looks like a number. Retrying will produce a different wrong answer.

The diagnostic question: *is the error due to how the model extracted the information, or due to the information not being present?* The answer determines whether to retry or to route the document for human review or accept null.

The exam tests this distinction directly. The correct response to "*required field is absent from source document*" is not more retries with more specific errors. It is recognizing that the information constraint is architectural: the schema requires data the document does not have. Address it at the schema level (make the field optional) or the pipeline level (route document type to a different schema).[4]

Complexity Limits: The 400 Error Nobody Expects

Constrained decoding works by compiling schemas into grammars. Complex schemas produce large grammars, and large grammars take longer to compile. The API enforces explicit limits to protect against excessive compilation times.[2]

These limits apply to all requests with output_config.format or strict: true:

Limit	Value	Description
Strict tools per request	**20**	Maximum tools with strict: true. Non-strict tools don't count.
Optional parameters total	**24**	Total optional parameters across all strict schemas and JSON output schemas. Each parameter not in required counts.
Union type parameters total	**16**	Parameters using anyOf or type arrays (e.g., "type": ["string", "null"]).

The per-request nature of the optional parameter limit is the trap. **Four tools with six optional parameters each adds up to 24: exactly at the limit, with no room.** The limits apply to the combined total across all strict schemas in a single request.[2]

Beyond the explicit limits, internal grammar size limits can trigger a 400 error even when all three explicit limits are satisfied. Schema complexity does not reduce to a single dimension: optional parameters, union types, nested objects, and the number of strict tools interact with each other. The compiled grammar can grow disproportionately large from combinations of these features. The API enforces a 180-second compilation timeout as a final stop, and schemas that pass all checks but produce very large grammars may hit it.

When hitting complexity limits, the documented strategies in order of preference:[2]

1. Mark only critical tools as strict. If the schema violation for a simpler tool is tolerable (the model naturally produces valid inputs anyway), reserve strict: true for the tools where violations cause real problems.

2. Reduce optional parameters. Each optional parameter roughly doubles a portion of the grammar's state space. Making a parameter required and having the model supply a default explicitly is cheaper than leaving it optional.

3. Flatten nested structures. Deeply nested objects with optional fields compound complexity.

4. Split across multiple requests or subagents. If the tool set is too large for a single strict request, distribute the tools across separate calls or subagent invocations.

The 400 error message is "Schema is too complex for compilation." When it appears, the cause is schema complexity, not a malformed request.

Schema Compliance Without Semantic Correctness: The Core Exam Distinction

The exam scenario walkthrough for Scenario 6 states the distinction in unambiguous terms: tool_use guarantees structure, not semantics.[1]

Structure: the JSON is valid, the required fields are present, the types match, additionalProperties is respected.

Semantics: the values inside the structure accurately represent the source document.

The structural guarantee comes from constrained decoding, a mechanical property of how the API generates tokens. The semantic guarantee comes from the

validation loop, a property of the application code that checks the extracted values against business rules, arithmetic constraints, and domain knowledge.

A system that trusts the structural guarantee and skips semantic validation is making an architectural error. The schema says the output is shaped correctly. It says nothing about whether the values are right.

The production architecture combines both layers. Use tool_use with strict: true to eliminate the entire class of JSON syntax failures. Use application validation to catch semantic errors the schema cannot express. Use the retry loop to correct format mismatches and structural errors in the extraction. Know when to route for human review instead of retrying.

This is not a critique of structured output as a mechanism. The structural guarantee is valuable precisely because it eliminates one entire class of failures, which makes the remaining failures (semantic) easier to reason about. The error isn't trusting tool_use. The error is trusting it for more than it guarantees. Every tool schema in the request contributes to context window cost; Chapter 11 covers the architectural implications of that budget.

Sample Questions

Question 1: A structured extraction pipeline uses tool_use with a strict JSON schema. After extracting invoice data, the validator finds that the line-item amounts sum to a different value than the stated_total field. The JSON structure is valid and all required fields are present. What type of error has occurred?

A. A schema compliance failure, which strict: true should have prevented
B. A *semantic* error, which requires application-level validation to detect
C. A token limit error causing truncated output
D. A tool_choice configuration error preventing correct tool selection

Correct answer: B. *The JSON structure is valid (strict: true succeeded). The discrepancy between line-item sums and stated total is a* ***semantic error:*** *the values are internally inconsistent, which is above what schema validation can enforce.*

Question 2: A validation retry loop is appending errors to the prompt and retrying. After three retries, the extraction still fails because a required field (contract_reference) is not present anywhere in the source document. What is the correct next step?

A. Add more specific error feedback about the contract_reference field and retry again
B. Switch to output_config.format instead of tool_use for better extraction
C. Make the contract_reference field optional in the schema and handle null downstream
D. Increase max_tokens to give the model more room to find the field

Correct answer: C. *The required information is absent from the source document.* ***This is not a retryable error****; more retries will produce fabricated values or the same failure. The schema should treat the field as optional (nullable), and downstream logic should handle null gracefully.*

Question 3: A request using strict: true on all 6 tools fails with a 400 error: "Schema is too complex for compilation." Each tool has 4 required parameters and 5 optional parameters. None of the parameters use union types. Which limit has been exceeded?

A. The 20 strict-tools-per-request limit
B. The 24-optional-parameters total limit
C. The 16 union-type parameters limit
D. None of the explicit limits; the grammar size limit was exceeded

***Correct answer: B**. Six tools with 5 optional parameters each equals 30 optional parameters total**, exceeding the documented limit of 24**. The 20-tool limit is not exceeded (only 6 tools). The 16 union-type limit does not apply (no union types). This question tests whether candidates can calculate combined totals across all strict schemas, not just recall individual limit numbers.*[1]

Question 4: An extraction pipeline uses tool_choice: "auto" with a schema-constrained extraction tool. In testing, some documents are processed with tool calls and others receive natural language responses. What configuration change guarantees a tool call for every document?

A. Add strict: true to the tool definition
B. Change tool_choice to "any" or force the specific tool
C. Move the schema to output_config.format
D. Increase the max_tokens parameter to give the model more room

***Correct answer: B.** tool_choice: "auto" allows the model to return conversational text instead of calling a tool. "any" or forced tool selection guarantees a tool call. strict: true controls schema validation, not whether the tool is called.*

Key Takeaways

- tool_use with ***strict:true*** is a structural contract: the output will match the JSON schema, but it is not a semantic contract. The two structured output features are output_config.format (constrains the response-level JSON) and strict: true (constrains tool input JSON); they address different surfaces and can be combined.

- ***tool_choice: "any"*** guarantees a tool call rather than a conversational response. Without it, tool_choice: "auto" allows the model to skip the tool entirely.

- Schema design directly affects extraction quality. Optional (nullable) fields prevent fabrication. "unclear" enums give the model a truthful option. "other" + detail fields handle edge cases without misclassification.
- The validation-retry loop must include specific error feedback: which field, which constraint, what value was found. Generic “try again” gives the model no signal.
- Retries fix format mismatches and structural errors. They do not fix missing information. When the required data is not in the source document, the fix is schema design (make the field optional) or routing (send document for human review), not more retries.
- Self-correction schemas extract both *stated_total* and *calculated_total*, letting the schema itself surface arithmetic conflicts. The *detected_pattern* field enables systematic analysis of extraction failures by document structure; without it, dismissal analysis requires manual document review.
- Complexity limits are per-request and cumulative: **20 strict tools maximum, 24 optional parameters total across all schemas, 16 union-type parameters total**. Required properties appear before optional properties in structured output regardless of schema order. Streaming and output_config.format cannot be used together.

CHAPTER 11: CONTEXT WINDOW DISCIPLINE

Summary: *The context window is finite, and its middle is unreliable. The lost-in-the-middle effect means information placed in the center of long inputs is less likely to influence model output, a structural property of how attention works across long sequences. That constraint has architectural consequences: how to position critical facts, how to manage tool output accumulation, when to compact or delegate, and how to design for sessions that outlive a single context window. This chapter covers all of it. The case facts block is the primary defense against progressive summarization: a structured, immutable reference section placed at the start of context and never compressed through summarization passes. Tool output accumulation is the dominant context cost in long sessions; a single verbose tool result can persist and consume space across every subsequent turn. The ToolSearch mechanism addresses MCP server overhead by loading tool schemas on demand rather than preloading all definitions. Subagent delegation keeps the coordinator's context lean by absorbing exploration transcripts internally, returning only concise findings to the parent.*

The Session That Ate Its Own Memory

The customer record was simple. John Smith. Account ACC-12345. Order #98765. Charged $150.00 instead of the promotional price of $99.99. An overcharge of exactly $50.01. Seven years as a customer. High priority.[1]

By the fifth turn of the conversation, after the first summarization pass, the record looked different. "Customer called about billing issue with promotion." No name. No account number. No order number. No amounts.

By the tenth turn, after the second pass, it was four words: "Customer has a billing issue."

The name, the account, the order, the exact dollar figures, the promotion code, the tenure, the priority rating: all gone. Not corrupted. Not misread. Gone, silently, through the perfectly normal operation of progressive summarization compressing context to make room for new turns.[1]

This is the problem. The context window is not just a size limit. **It is a degradation engine when left unmanaged.** And the model cannot tell you that it lost something, because it no longer knows the something existed.

The Lost-in-the-Middle Effect

Context windows have a size. That size is real and fixed per model. But size is not the only structural property worth understanding.

Research into model behavior on long-context retrieval tasks reveals a consistent pattern: models reliably recall information positioned at the beginning and end of long inputs, and *they are significantly less reliable at recalling information placed in the middle*.[2] The effect is not about compression or summarization. It shows up even when the full context is present. It is a property of how attention mechanisms distribute weight across long sequences. The middle gets less.

This is what the domain calls the *lost-in-the-middle effect*. Name it, because once named it becomes an architectural constraint to design around rather than a vague "context problem" to worry about later.[1]

The practical implication: where you put information in context is not neutral. Placing a customer's account number in turn 3 of a 40-turn conversation and expecting it to remain salient in turn 40 is not a safe assumption. Placing a 3000-token tool output in the middle of a 180K-token context is not the same as placing it at the beginning.

Position-aware ordering is the corrective. Put the most important information at the beginning and end of context. Organize aggregated results with explicit section

headers so key findings surface at the top of each section. When preparing inputs for a synthesis step, put the summary first, then the detail.[2]

The effect is real. Plan for it.

Progressive Summarization Risk

Summarization is useful. It compresses verbose history into manageable context, extends how long a session can run, and reduces the token cost of subsequent turns. The problem is not summarization. The problem is summarizing the wrong things.

Progressive summarization risk is what happens when numerical values, dates, amounts, account identifiers, and customer-stated expectations get flattened into vague prose through successive compression passes.[2] The John Smith example above is the canonical illustration. The summarizer is not broken. It correctly identifies that "*John Smith, ACC-12345, order #98765, charged $150 versus $99.99 promotional price*" is about a billing issue. It preserves the semantic category while discarding the specific facts inside it.

For a conversational system where the outcome is "*resolve this customer's billing issue*," those specific facts are not decoration. They are the task. The exact overcharge amount is what the refund calculation requires. The order number is what the lookup tool needs. The account ID is what the authentication check verifies. Strip those and the agent can no longer do its job, even if it still knows the category of job it is supposed to do.

The progressive nature of the risk makes it worse. A single summarization pass might preserve enough. Two passes almost certainly will not. By the third pass, even the category might collapse into something so vague ("billing matter under review") that the agent cannot act on it at all.

The detection problem is quiet. The model does not announce that it has lost critical facts. It continues to respond confidently, often substituting "typical

patterns" for the specific case details it no longer has. A support agent that has lost the order number will ask for it again, or worse, attempt to operate without it, producing plausible-sounding but factually wrong responses.

The Case Facts Block

The architectural response to progressive summarization risk is not to avoid summarization. It is to protect what must not be summarized.

The case facts block is a structured, immutable reference section placed at the beginning of context, outside the summarized history, that contains all critical transactional facts for the current session.[1,2] It is never summarized. It is never compressed. It is present in every request, at the beginning where recall is highest.

For a customer support scenario, the block looks something like this:

```
## CASE FACTS (Do not summarize — reference directly)

| Field          | Value                          |
|----------------|--------------------------------|
| Customer       | John Smith                     |
| Account ID     | ACC-12345                      |
| Order          | #98765                         |
| Expected Price | $99.99 (promotion SUMMER2026)|
| Charged Price  | $150.00                        |
| Overcharge     | $50.01                         |
| Customer Since | 2019                           |
| Priority       | High                           |

## RULES
- Refund amount ($50.01) is within $500 agent limit
- This case qualifies for immediate resolution [1]
```

The block includes the instruction to not summarize, not as a magic string the compactor recognizes, but because putting the intent in writing provides the compactor with context about how to treat it. The compactor reads your CLAUDE.md and your case facts block alike.[3]

The structure matters. Plain prose facts are easier to summarize into vague categories than a labeled table. A table resists compression because it presents information as data, not as narrative. The compactor has less to compress it into.

For multi-issue sessions, each distinct issue gets its own structured entry in the block, preserving all relevant specifics as a separate context layer from the summarized conversation history.[2]

The case facts block is a position and structure decision, not a prompt instruction. Prompt instructions asking the model to "*remember important details*" will not survive compaction. The block survives compaction because it lives outside the history being compacted, injected afresh into each request from persistent storage.

What Consumes Context in the SDK

Understanding where context goes is prerequisite to managing it. The SDK accumulates context from multiple sources across a session, and not all of them are obvious.[3]

The system prompt loads with every request. Small fixed cost, but always present.

CLAUDE.md files load at session start via settingSources. If your project has a large CLAUDE.md or several nested files, they consume context on every request, every turn, before any conversation begins.[3]

Tool definitions load with every request. This is where architects most often underestimate cost. Every tool schema, field description, and example is part of the context every time Claude runs. For an agent with eight built-in tools and three MCP servers, the tool definition cost alone can be substantial.

MCP servers are the specific pressure point. Each MCP server adds all its tool schemas to every request.[3] A single large MCP server with many tools can consume thousands of tokens before the agent does any work. Two or three such servers, and the tool definition overhead becomes nontrivial. This is the context cost that *ToolSearch* addresses: instead of loading all tool definitions upfront,

ToolSearch lets the agent discover and load tools on demand, turn by turn, reducing the baseline overhead to a manageable minimum.[3,4]

Conversation history accumulates across turns. Tool inputs and tool outputs both go into the history. This is the dominant cost in long-running sessions. A single verbose tool result from an order lookup that returns 40 fields when only 5 are relevant accumulates in the history and stays there, consuming space on every subsequent turn.[2]

The implication: trim tool outputs to relevant fields before they accumulate. A *PostToolUse* hook that strips irrelevant fields from verbose MCP tool responses is doing context hygiene, not just data normalization.[3]

Prompt caching handles the static pieces. Content that stays the same across turns, including the system prompt, tool definitions, and CLAUDE.md, is automatically prompt cached, which reduces cost and latency for repeated prefixes.[3] Caching does not reduce context window usage, but it does reduce the token cost and round-trip time for those fixed elements.

Automatic Compaction and the compact_boundary Event

When the context window approaches its limit, the SDK automatically compacts the conversation. It summarizes older history to free space, keeping the most recent exchanges and key decisions intact. This is compaction as a default behavior, not something the architect has to implement.[3]

The SDK emits a signal when this happens. In Python, it appears as a SystemMessage with subtype: "compact_boundary". In TypeScript, it is a distinct SDKCompactBoundaryMessage type. Listening for this event tells you when compaction occurred, which is useful for logging, audit trails, and state management.[3]

The critical operational implication of automatic compaction: specific instructions from early in the conversation may not survive it. If you put deployment restrictions in the first user message and the context later compacts, those restrictions may disappear from the model's working context. The model will not notice. It will continue running, now missing the constraint.

The fix is to *put persistent rules in CLAUDE.md*, loaded via settingSources, not in the initial prompt. CLAUDE.md content is re-injected on every request because it is loaded from the project configuration on each turn, not stored in conversation history. Instructions in CLAUDE.md survive compaction because they never get compacted.[3]

You can also include a summarization instructions section in CLAUDE.md telling the compactor what to preserve. The section header is not a magic string; the compactor matches on intent. The section can say something like "*Always preserve: customer account numbers, order IDs, specific dollar amounts, escalation decisions.*" The compactor reads this alongside everything else and uses it as guidance for what to keep.[3]

Manual compaction is available via */compact* sent as a prompt string. It is SDK input, not CLI shorthand. Send /compact as a prompt to trigger compaction on demand, before the automatic threshold hits, when you can see that the session is filling with verbose discovery output that is no longer relevant.[3]

The *PreCompact* hook fires before compaction and is where the full transcript can be archived before the SDK summarizes it. Chapter 3 owns the full hook mechanics. The relevant architecture point here is that compaction is a signal, not a failure, and the PreCompact hook is where the system can respond to that signal with preservation logic.[5]

Context Degradation Symptoms

Context degradation is insidious because the model does not report it. It degrades gradually and silently, substituting general knowledge for specific session findings as the specific findings become less accessible.[1,2]

The two canonical symptoms:

Inconsistent answers. The model gives a different answer to the same question asked at turn 40 than it gave at turn 10. Not because anything changed, but because what was salient at turn 10 has drifted toward the middle of a 150K-token context and is receiving less attention.

“Typical patterns” substitution. Instead of referencing a specific finding from earlier in the session, the model references what is generally true about this kind of system. “Typically in this architecture…” instead of “Based on the auth module analysis from earlier…” This is the model doing the best it can with what is accessible, which is increasingly general knowledge rather than specific session work.[2]

Both symptoms are recognizable in practice. Both are architectural failures before they are model failures. The architecture let the context fill without mitigation, and the model’s reliability degraded as a result.

Mitigations

Three structural mitigations, each addressing a different failure mode.

Scratchpad files. A scratchpad is a file the agent writes to during exploration, recording key findings as it discovers them. Before a verbose analysis phase, the agent creates a scratchpad. After each major finding, it updates the scratchpad. If context fills and compaction occurs, the agent re-reads the scratchpad to restore the specific findings that might otherwise have been lost to summarization. The scratchpad is not context. It is external state, persistent across context resets.[1,2]

The scratchpad pattern requires the agent to know it should use it. That instruction belongs in CLAUDE.md or in the system prompt, where it survives compaction.

/compact on demand. When the agent can detect that context is filling with verbose output that served its purpose and is no longer needed, triggering /compact before the automatic threshold clears that material while preserving a summary. Manual compaction gives more control over when compression happens and what the summary covers. Used in combination with summarization instructions in CLAUDE.md, it can produce more targeted compaction than the automatic behavior.[3]

Subagent delegation. This is the structural mitigation for verbose exploration phases. A subagent starts with a clean conversation: no prior message history, no accumulated tool outputs from the parent session. It does its work, and only its final response returns to the parent agent as a tool result.[3] The parent's context grows by that summary, not by the full exploration transcript.

For a codebase analysis that requires reading dozens of files and following import chains across multiple passes, the verbose output of that exploration should stay inside a subagent. The coordinator gets the findings. The coordinator's context stays lean, and the coordinator's recall stays sharp across the full session.[2]

Subagent delegation does not mean the coordinator loses visibility. It means the coordinator's context contains summaries and findings rather than exploration transcripts. That is the right shape.

Crash Recovery Manifests

Long-running agent sessions introduce a failure mode that shorter sessions don't have: the session can crash after significant work has been done, and none of that work is automatically recoverable.

The architectural response is crash recovery through structured state exports. Each agent exports its state to a known location at meaningful checkpoints, including

what it has completed, what findings it has produced, and where it was in the task. The coordinator maintains a manifest: a structured file that describes what has been done and by which agents.[1,2]

On session resume, the coordinator loads the manifest and injects the relevant state summaries into each agent's initial context. The agents do not start from scratch; they start from the checkpoint. The coordinator knows which phases are complete and which need to continue.[2]

The manifest is persistent external state, not conversation history. It survives crashes because it lives on disk. It survives context compaction for the same reason. A session that resumes with a well-designed manifest can continue as if the interruption were minor, because the critical state is not inside the context window at all.

Designing for crash recovery is designing against the assumption that sessions are atomic. For any agent session that runs for more than a few minutes and does meaningful work, that assumption will eventually be wrong. When the crash originates from a subagent failure, Chapter 12's error propagation patterns govern how that failure is communicated before the session terminates.

Context Window Sizes

The size of the ceiling matters when the work is large.

Claude Opus 4.7, Claude Opus 4.6, and Claude Sonnet 4.6 have 1M-token context windows. Other Claude models, including Claude Sonnet 4.5, have 200K-token windows.[4]

A 1M-token window does not eliminate the need for context discipline. The lost-in-the-middle effect operates at 1M tokens. Tool output accumulation happens at 1M tokens. Compaction will still fire if the session runs long enough. But the ceiling being five times higher changes what is feasible in a single session and what requires subagent delegation or cross-session state management.

Claude Sonnet 4.6, Claude Sonnet 4.5, and Claude Haiku 4.5 feature context awareness, which means these models receive information about their remaining token budget after each tool call and can adapt their behavior accordingly. A model with context awareness can make informed decisions about depth of analysis as context fills, rather than operating blind until the compaction threshold hits.[4]

The model selection decision is also a context architecture decision. If a task genuinely requires holding 500K tokens of accumulated research in memory simultaneously, the choice of model is constrained by which models can accommodate that window.

MCP Server Context Cost and ToolSearch

MCP server context cost deserves separate treatment because it catches architects who understand conversation history accumulation but underestimate the static overhead.

Every configured MCP server contributes all its tool schemas to every request. Not just the tools that will be called in this turn. All of them, every turn, before the conversation begins.[3] A server with 12 tools, each with detailed descriptions and parameter schemas, can consume thousands of tokens on every request. Three such servers, and the agent is starting each turn with significant context already spent before the first message.

ToolSearch is the architectural response. Instead of loading all tool definitions from configured MCP servers upfront, ToolSearch allows the agent to discover and load tools on demand. The agent asks ToolSearch what tools are available matching a query, gets back the relevant schema, and then calls the tool. The full schema for the remaining tools never enters the context window on turns where those tools are not needed.[3,4]

The tradeoff is one additional ToolSearch call per tool discovery event. For agents that use a predictable subset of a large MCP server's tools on each turn, the token

savings from not loading unused schemas can significantly outweigh the overhead of the discovery call.

For agents that genuinely use all available tools regularly, preloading may be more efficient. ToolSearch is a tool for the case where the tool set is large but usage is selective.

Exam Sample Questions

These questions are original constructions. They do not reproduce exam content.

Question 1. A multi-turn support agent is handling a billing dispute. By turn 15, the agent asks the customer for their account number, even though the customer provided it in turn 2. Which of the following best explains this behavior?

A. The model exceeded its max_tokens limit and reset the conversation.
B. Progressive summarization compressed the account number into a vague category description, and the case facts block was not used.
C. The PostToolUse hook stripped the account number from the tool result before it was processed.
D. The model's context awareness feature determined the account number was no longer relevant.

Correct answer: B. *Progressive summarization is the mechanism; absence of a case facts block is the architectural gap. A would manifest as an error, not continued operation. C describes a hook behavior, not a summarization effect. D mischaracterizes context awareness, which reports remaining token budget, not relevance.*

Question 2. A codebase exploration agent is running a long multi-file analysis. After 30 turns, the agent begins referencing "typical patterns in this kind of codebase" rather than specific findings from its earlier file reads. Which mitigation most directly addresses this symptom?

A. Increase the model's effort setting to "max" to improve recall.

B. Enable the PreCompact hook to archive transcripts before compaction fires.

C. Have the agent maintain a scratchpad file recording specific findings and re-read it periodically.

D. Reduce the number of tool calls per turn to limit context growth.

Correct answer: C. *Scratchpad files externalize key findings to persistent state outside the context window, making them accessible even after compaction or middle-of-context drift. A affects reasoning depth, not recall. B archives the transcript but does not make findings accessible after compaction. D slows context growth but does not address the recall problem.*

Question 3. A coordinator agent has access to three MCP servers, each with approximately 15 tool schemas. The coordinator uses only 2-3 tools per session. Which configuration reduces per-turn context overhead most effectively?

A. Reduce the number of MCP servers from three to one.

B. Use ToolSearch to load tools on demand rather than preloading all schemas.

C. Set tool_choice: "none" to prevent tool calls from occurring unnecessarily.

D. Increase the context window by switching to a 1M-token model.

Correct answer: B. *ToolSearch loads tool schemas on demand, preventing unused schemas from consuming context on every request. A would require removing servers the agent legitimately needs. C prevents tool use entirely, which contradicts the agent's function. D increases the ceiling but does not reduce the per-request overhead.*

Key Takeaways

- The lost-in-the-middle effect is structural: information at the center of long inputs receives less attention than information at the beginning and end.

Design context layout accordingly with critical facts at the top, key findings summarized at the top of each section, and immutable reference blocks anchored to the beginning.

- Progressive summarization destroys specific facts silently. The case facts block is the architectural counter: an immutable structured block outside summarized history, present in every request at the beginning of context, never compressed through summarization passes.
- Tool output accumulation is the dominant context cost in long sessions. Trim verbose outputs to relevant fields before they accumulate. MCP server schemas are a separate static cost that ToolSearch addresses through on-demand loading rather than preloading all definitions every turn.
- Automatic compaction handles the overflow but strips early instructions. Persistent rules belong in CLAUDE.md, loaded via settingSources, not in initial prompts that will not survive compression.
- **Three structural mitigations for context degradation**: ***scratchpad files*** for persisting findings across context resets, ***/compact on demand*** to clear verbose material when it has served its purpose, and ***subagent delegation*** to keep the coordinator's context lean while verbose exploration happens elsewhere.
- Crash recovery through structured state exports means sessions can survive interruption. The manifest is external state, not conversation history; design for it when sessions are nontrivial.
- Context window size is a model selection variable. 1M-token models (Claude Opus 4.7, Opus 4.6, Sonnet 4.6) change what is feasible in a single session, but context discipline remains necessary regardless of ceiling height.

CHAPTER 12: ESCALATION, PROVENANCE, AND ERROR PROPAGATION

Summary: *A reliable agentic system knows when not to act. Valid escalation triggers are objective and verifiable: an explicit customer request for a human agent, a genuine policy gap, an inability to make progress, a business threshold violation, or an ambiguous match requiring clarification. Negative sentiment and model self-confidence are not valid triggers. Error propagation must be structured and distinguishable: an access failure and a valid empty result look identical to a coordinator that receives only an empty array, yet the downstream consequences differ entirely. Provenance tracking must survive synthesis steps, because a claim without its source citation is not a claim you can act on. The DataWithProvenance pattern preserves claim-source mappings through synthesis by pairing each extracted value with its source, confidence category, and retrieval timestamp. Stratified accuracy metrics reveal per-category failure rates that aggregate numbers mask; a pipeline at 97% aggregate accuracy may be failing 30% of one document type. Structured handoff summaries ensure that human agents receiving escalated tickets have the specific facts they need without re-interviewing the customer.*

The Ticket That Should Have Closed Itself

The human agent opens the ticket. Billing discrepancy. A straightforward overcharge. The support agent had routed it to the human queue with a note that said “negative sentiment detected.”

The overcharge is real. It is also trivially within the agent’s own resolution authority: two tool calls, get_customer then process_refund. The whole interaction would have resolved without escalation. Instead, a human is now involved, the

customer waited in the escalation queue, and the agent's first-contact resolution rate took a hit.

This is the trap the exam calls out directly: sentiment as a proxy for complexity.[1] The customer was frustrated. The task was simple. Those two things are independent, and the system conflated them. This chapter is about the patterns that prevent that conflation: knowing when to escalate, how to communicate failure, and how to preserve the chain of evidence that makes a decision auditable.

Knowing When Not to Act Autonomously

The named concept for this chapter is "Knowing when not to act autonomously." It sounds passive. It is actually one of the harder design problems in agentic systems, because the failure mode is invisible. An agent that escalates too much costs human time and degrades resolution rate metrics. An agent that escalates too little resolves things it should not. Both failures hurt real customers. The exam focuses on the triggers that distinguish them.

VALID ESCALATION TRIGGERS

The exam source material identifies five valid escalation triggers.[2] Each one is objective: it can be evaluated without inspecting the model's internal state or reading emotional tone from message text.

The customer explicitly requests a human agent. This is the most important trigger. Honor it immediately, without first attempting investigation or offering resolution. The code pattern is straightforward: if *context.customer_requested_human is true*, escalate and return the reason. Do not try to resolve the issue first to "save" the escalation. That is a design choice that prioritizes the metric over the customer's stated preference, and the exam treats it as an anti-pattern.[2]

Policy is ambiguous or silent on the specific situation. Consider a customer asking about competitor price matching when the company's policy only addresses price adjustments for the company's own previous sales. The policy document does not cover this case. That silence is a policy gap, and a policy gap is a valid escalation trigger. The agent cannot invent policy. Escalation is the correct behavior.[2]

The agent cannot make meaningful progress. The clearest version of this is exhausting retry attempts. A transient failure that persists beyond the retry limit represents a capability ceiling the agent cannot cross locally. This is distinct from the sentiment-based anti-pattern: the agent is not escalating because it is uncertain, it is escalating because an objective condition (retry count) has been met.[2]

A business threshold is exceeded. The $500 refund limit example from the customer support scenario is the canonical case.[1,3] An agent has a defined financial authority. Requests above that limit are not within the agent's authority to resolve, regardless of the agent's assessment of the request's legitimacy. The threshold is objective. It is either exceeded or it is not.

Multiple customer matches requiring clarification. When a query returns multiple matching records, the agent must ask for additional identifiers. Selecting heuristically from ambiguous results is an anti-pattern. The heuristic might be right most of the time. It will be wrong at the worst possible moments, on accounts where the stakes are highest.[2]

INVALID ESCALATION TRIGGERS

Two specific anti-patterns appear in the exam source material and show up in scenario questions.[1,2,3]

Negative sentiment. An angry customer asking to update their address does not need a human agent. The emotional state of the message is not correlated with the complexity of the task. Routing based on sentiment conflates two unrelated signals.

Self-reported model confidence below a threshold. Model confidence scores are not reliable proxies for actual case complexity. A model might report low confidence on a routine task that it handles well, and high confidence on an edge case where it is about to make an error. Building escalation logic on self-reported confidence means the confidence number is doing work it cannot actually do.

The should_escalate function in the exam source makes this distinction explicit in code: the valid triggers are checked against objective context fields; the anti-patterns are commented out with the labels WRONG!.[3] The exam is essentially testing whether you can read that comment correctly.

THE FRUSTRATION CASE

There is a specific nuance the exam tests separately from the binary "escalate or not" decision. When a customer expresses frustration but the issue is within the agent's capability, the correct behavior is to acknowledge the frustration while offering resolution, and escalate only if the customer reiterates the preference for a human agent.[2]

This is different from ignoring the customer's emotional state. Acknowledgment is appropriate. Automatic escalation is not. The customer's stated preference for a human agent is what triggers immediate escalation, not the emotional register of the message.

Structured Handoff Summaries

When escalation does happen, the agent has an obligation to the human receiving the ticket. That obligation is information.

Human agents do not have access to the conversation transcript.[4] This is the architectural reality the handoff pattern is designed around. If the agent escalates with a generic status like "*customer issue escalated*" and nothing else, the human agent receives a ticket with no context. They have to start the conversation over

from scratch. The customer, who just spent time explaining the situation to the automated agent, now has to explain it again.

The ***structured handoff summary*** solves this. When compiling an escalation, the agent includes: *customer ID, root cause analysis, the amounts involved,* and a *recommended action*.[4] These are not summaries in the progressive-summarization sense, where details get lost in compression. They are structured extractions: specific, identifiable fields that a human can act on without re-interviewing the customer.

The handoff summary is the bridge between the agent's context and the human's context. Build it deliberately.

Self-Critique: The Evaluator-Optimizer Pattern

There is a failure mode that looks like an escalation problem but is not. The agent resolves the case correctly. The customer is not asking for a human. No threshold is exceeded, no policy gap exists, no match is ambiguous. And satisfaction still drops, because the explanation is inconsistent: one resolution omits the relevant policy, the next omits the timeline, a third omits the next steps. The resolution is right; the communication around it is unreliable, and the specific gap is different every time.

None of the escalation triggers fire here, and none of them should. This is not a "when to hand off" problem. It is an output-quality problem, and the tool for it is the evaluator-optimizer pattern.

The pattern is a same-model second pass. After the agent drafts a response, it evaluates that draft against an explicit completeness rubric before presenting it: Does this address the customer's actual concern? Does it include the relevant policy context? Does it state the timeline and the next steps? Does it anticipate the obvious follow-up question? If the draft fails the rubric, the model revises and re-checks. Only the passing version reaches the customer.

Distinguish this carefully from the same-session self-review anti-pattern in Chapter 8. Those are not the same operation, and conflating them is an exam trap. The

Chapter 8 anti-pattern is a generator reviewing its own work for correctness: the session that wrote the code is asked whether the code is right, and it cannot answer honestly because it retains all the reasoning that produced the code and is biased toward confirming its own decisions. The fix there is an independent instance with no shared context. The evaluator-optimizer pattern is a different operation applied to a different target. It checks a draft for completeness against a rubric, not for correctness against the model's own prior reasoning. There is no confirmation-bias problem because the model is not being asked to second-guess a conclusion; it is being asked to check a response against a checklist. The same model can do that reliably in the same session.

This is also why self-critique, not few-shot, is the correct intervention when the defect varies case to case. Few-shot examples teach a fixed pattern. When the omission is a moving target (policy this time, timeline next time, next-steps the time after), a finite example set cannot enumerate the space of possible gaps. A rubric check catches whatever gap appears in this specific output, because the rubric is defined abstractly over the dimensions of completeness rather than over a list of pre-seen cases. The intervention classifier in Chapter 9 turns on exactly this discriminator: the word "vary" in the stem eliminates few-shot and selects self-critique.

Keep this separate from the two review architectures the exam groups under Task 4.6: multi-instance and multi-pass review. Multi-instance review is a separate model instance with no shared reasoning context, the independent-reviewer pattern from Chapter 8; it is correct when the target is correctness and the generator's retained context would bias the review. Multi-pass review, also covered in Chapter 8, splits a large multi-file review into per-file passes plus a cross-file integration pass to counter attention dilution; it is a decomposition move, not a self-critique move. The evaluator-optimizer pattern described here is a third, distinct operation: the same model running a second-pass check of a single draft against a completeness rubric. Three patterns, three failures. Per-case quality that varies unpredictably calls for the second-pass self-critique here; a generator biased toward confirming its own reasoning calls for multi-instance review; attention diluted across many files calls for multi-pass review. Read the stem for the failure it

actually describes; the options will usually offer more than one, and the wrong choice is the right pattern aimed at the wrong failure.

Access Failure vs Valid Empty Result

This is the most-tested distinction in Domain 5, and the failure mode is catastrophically silent.[5]

Consider a subagent that queries a customer database. Two different things can happen that result in an empty response:

1. The database query succeeded and returned no matching records.
2. The database was unreachable due to a timeout, and the query never ran.

In both cases, if the subagent returns {"customers": []}, the coordinator sees an empty array. The coordinator concludes: no customers found. In the first case, that conclusion is correct. In the second case, it is wrong. The coordinator does not know the search failed. It proceeds on the assumption that the search succeeded and found nothing.

The consequences depend on what happens next. In a billing dispute scenario, “no matching customers” might cause the agent to tell the customer it cannot find their account. In a research system, “no results” might cause the synthesis agent to conclude that no relevant sources exist on a topic that is actually well-documented.

The correct pattern separates these two outcomes with distinct response structures.[5]

An access failure is reported as: - *isError*: true - *errorCategory* field indicating the failure type (transient, validation, permission, or business) - *isRetryable boolean* so the coordinator can decide whether to retry

A valid empty result is reported as: - *isError*: false - *customers*: [] (the empty array) - Metadata confirming the query parameters that were searched

These are structurally different responses. The coordinator can distinguish them. It cannot distinguish them if both cases return only [].

The *errorCategory* field matters for recovery decisions. **A transient timeout is retryable**. **A permission error is not.** A validation error means the query itself was malformed and retrying without modification will not help. Generic errors like "*search unavailable*" strip this information out before it reaches the coordinator, making intelligent recovery impossible.[5]

Local Recovery Before Coordinator Escalation

A related principle governs how errors move through the system: subagents implement local recovery for transient failures and propagate only errors they cannot resolve.[5]

This keeps the coordinator's decision surface clean. If every subagent propagates every transient error immediately, the coordinator spends its attention on failures that the subagent could have retried successfully on the second attempt. The coordinator exists to manage complex decisions, not to be a retry queue. For the recovery side of this story, including crash recovery manifests and structured state exports, see Chapter 11.

The subagent's local recovery responsibility has a scope limit: transient failures within the subagent's authority. When a failure is not transient, when it represents a genuine capability gap or a threshold violation, the subagent propagates the error. When it does, it includes what was attempted and any partial results.[5]

Partial results matter. Consider a subagent that retrieved data for most of its requested topics before failing on the remainder. The coordinator needs to know which topics succeeded and which failed, so it can decide whether to retry the failures, fill the gap from another source, or proceed with an annotated gap in

coverage. Propagating only the failure without the partial results discards the successful work.

The taxonomy from Task Statement 2.2 covers four error categories: transient (timeouts, service unavailability), validation (invalid input format), business (policy violations), and permission (access denied).[5] Each category implies a different coordinator response.

Transient: retry with backoff.

Validation: fix the query.

Business: escalate.

Permission: notify administrator.

Structured Error Context

Generic error propagation is an anti-pattern for the same reason generic status codes are anti-patterns: they hide recovery-relevant information behind a label.

"Search unavailable" tells the coordinator that something went wrong. It does not tell the coordinator what was attempted, which query parameters were used, whether any partial results exist, or whether an alternative approach might succeed. The coordinator receiving "search unavailable" can only decide: retry or abort. It cannot make an informed judgment about which other subagent might cover the gap, or what the cost of proceeding without this data point would be.[5]

Structured error context includes:[5]

- Failure type (from the taxonomy above)
- What was attempted (the query, the parameters, the approach)
- Partial results (whatever was retrieved before the failure)
- Alternative approaches (other tools or sources that might cover the gap)

This is more tokens. It is also the information that separates a coordinator that can recover intelligently from one that can only guess. The extra context is the interface contract between the subagent and the coordinator.

Coverage Annotations

Synthesis output has the same information-completeness problem as error propagation, in a different form. When a synthesis agent compiles findings from multiple subagents, some topic areas may be well-covered and others may have gaps because sources were unavailable or subagents failed.[5]

Without annotations, the synthesis output looks uniformly confident. A reader of the report has no way to know which sections are based on multiple corroborating sources and which sections are based on a single low-confidence signal or, worse, on nothing at all.

Coverage annotations are explicit markers in the synthesis output indicating which findings are well-supported and which topic areas have gaps. They preserve the provenance signal through the synthesis step rather than flattening it into a uniform surface.

The exam tests this in the context of the multi-agent research system scenario.[1] A synthesis agent that does not propagate coverage information forces the end reader to trust the output uniformly. That trust is not warranted. Coverage annotations are the honest interface.

Claim-Source Mappings and the *DataWithProvenance* Pattern

Provenance tracks where information came from. The problem it solves is that synthesis destroys provenance by default. When a synthesis agent takes findings

from three subagents and combines them into a summary paragraph, the individual claim-source relationships are lost unless they are explicitly preserved.

The DataWithProvenance pattern is the structural mechanism for explicit preservation.[6] This pattern applies specifically in the coordinator-plus-subagents topology from Chapter 2, where subagent findings route through a coordinator synthesis step that can compress or discard source information. Each claim carries its source alongside it: the source URL or document name, a relevant excerpt, and metadata indicating the confidence level and retrieval time. When a synthesis agent processes these claims, it preserves the source mappings through the combination step rather than discarding them.

The pattern is implemented as a dataclass with fields*: value, source, confidence (one of "verified", "extracted", "inferred", "estimated"), retrieved_at, and agent_id*.[6] The confidence field is not a probability; it is a categorical indicator of how the value was obtained. "*Verified*" means from an authoritative source like a live database. "*Extracted*" means parsed from a structured document. "*Inferred*" means derived from context. "*Estimated*" is a best guess.

Requiring subagents to output structured claim-source mappings (source URLs, document names, relevant excerpts) is the upstream design decision that makes downstream provenance preservation possible.[7] A subagent that returns findings as a prose paragraph has already destroyed the provenance. The coordinator cannot reconstruct it from the summary.

This is why the exam places claim-source mappings at the subagent-to-coordinator interface, not at the final output stage. Provenance must be captured at the point of retrieval and preserved through every transformation. Retrofitting it at synthesis time is not possible.

Conflicting Statistics

Multi-source research systems will produce conflicting data. Two subagents searching different sources will find different numbers for the same metric. The naive resolution strategies are wrong in specific, auditable ways.

Averaging conflicting values without understanding why they conflict treats a measurement disagreement as statistical noise. If one source reports a value from a study conducted in 2022 and another reports a value from a study conducted in 2025, averaging them produces a number that corresponds to neither study and may mislead more than either individual value would.

Arbitrarily selecting one value removes the conflict from the output without resolving it. The coordinator sees a single number and has no way to know it was contested.

The correct approach is explicit annotation: include both values with their source attribution, confidence levels, and publication or collection dates.[7] The synthesis agent does not resolve the conflict; it presents the conflict with full context, allowing downstream humans or system components to make an informed reconciliation decision.

Requiring publication and collection dates in structured outputs is the specific mechanism for distinguishing temporal differences from genuine contradictions.[7] Two studies measuring the same quantity at different points in time may both be correct: the quantity changed between measurements. Without dates, that temporal difference looks like a factual disagreement. With dates, it looks like a historical trend, which is a more accurate representation.

Stratified Accuracy Metrics

Aggregate accuracy metrics are comfortable lies.

Consider a document extraction pipeline that processes invoices, receipts, and contracts. An aggregate accuracy of 97% looks excellent. But if invoices are at 70%, receipts at 99%, and contracts at 100%, the aggregate number is masking a significant failure category.[6] The invoice pipeline is failing nearly a third of the time. The aggregate number would not tell an operator to look there.

Stratified accuracy metrics track performance per document type and per field rather than across the entire corpus.[6] The implementation tracks a dictionary keyed by document type, incrementing correct and total counts separately. Accuracy is computed and reported per category, not just in aggregate.

The exam connects this directly to the decision about when to automate high-confidence extractions. An aggregate 97% accuracy figure is not a reliable basis for automation decisions if the per-category breakdown shows that one category is at 70%. Before reducing human review on any category, validate accuracy by document type and field segment.[8]

Field-level accuracy adds a second dimension: a pipeline might perform well on some fields (product name, invoice date) while failing consistently on others (line-item subtotals, tax calculations). Tracking both dimensions reveals where the failures actually live, which is a prerequisite for fixing them.

Practice Question: Putting It Together

Consider this scenario. A multi-agent research system has a web search subagent, a document analysis subagent, and a synthesis subagent. The web search subagent experiences a database timeout while searching for recent statistics. It returns {"results": [], "status": "ok"} to indicate it found nothing. The synthesis subagent receives this result and compiles a report concluding that no recent data exists for the queried topic. The report does not indicate any gaps in source coverage.

There are three separate failures here. The web search subagent returned a valid-empty-result response for what was actually an access failure, hiding the error from the coordinator. The synthesis subagent had no visibility into the failure because the error reporting was absent. And the synthesis output contained no coverage annotations indicating that a source category was unavailable.

The corrected design addresses each independently. The web search subagent reports *isError: true* with *errorCategory: "transient"* and *isRetryable: true*. The coordinator makes a retry decision rather than forwarding a misleading empty result. The synthesis subagent structures its output with explicit coverage annotations for any topic area where source data was unavailable or questionable. The human receiving the report knows exactly where the gaps are.

This is what “knowing when not to act autonomously” actually means in practice: it is not only about escalation thresholds. It is about every design decision that preserves signal through the pipeline rather than silently dropping it.

What the Exam Tests

The exam tests escalation and error propagation as a system of objective decisions, not judgment calls.

For escalation triggers, the exam distinguishes valid triggers (explicit customer request, policy gap, retry exhaustion, threshold exceeded, ambiguous match) from the two named anti-patterns: negative sentiment and self-reported model confidence. Both anti-patterns appear in scenario answer choices labeled as wrong, not as edge cases.

For access failures versus valid empty results, the exam tests whether candidates recognize these as structurally distinct events. A subagent returning {"results": []} when a timeout occurred is a misclassification that propagates silently. *The correct*

design uses isError: true with errorCategory and isRetryable for failures, and isError: false with metadata for genuine empty results.

For error propagation, the four-category taxonomy (transient, validation, business, permission) maps directly to coordinator recovery decisions. The exam tests whether each category implies the correct next action: retry with backoff, fix the query, escalate, or notify an administrator.

For structured handoff summaries, the exam tests what must be included when escalating to a human agent who lacks transcript access: customer ID, root cause, amounts involved, and recommended action.

For the DataWithProvenance pattern, the exam tests provenance as a design decision made at the subagent output stage, not at synthesis. Provenance cannot be reconstructed after the synthesis step compresses or discards source mappings.

Coverage annotations and conflicting statistics are tested together: the exam asks what the correct synthesis behavior is when sources disagree or when a source category was unavailable. Annotation with source attribution and dates is the answer; averaging or silently discarding gaps is the anti-pattern.

Key Takeaways

- Valid escalation triggers are objective: explicit customer request (honor immediately), policy gap, capability limit or retry exhaustion, business threshold exceeded, multiple ambiguous matches requiring clarification. Invalid triggers are subjective proxies: negative sentiment does not equal task complexity; self-reported model confidence is not a reliable proxy for actual case complexity.[1,2,3]

- When a customer explicitly requests a human agent, honor the request immediately without first attempting resolution. When a customer expresses frustration without explicitly requesting a human, acknowledge

the frustration and offer resolution; escalate only if the customer reiterates the preference.[2]

- Structured handoff summaries (customer ID, root cause, amounts, recommended action) are the interface contract to human agents, who do not have access to the conversation transcript.[4]

- An access failure and a valid empty result are not the same event: an access failure must be reported as isError: true with errorCategory and isRetryable; a valid empty result is isError: false with an empty array and metadata. Subagents implement local recovery for transient failures and propagate only errors they cannot resolve, always including what was attempted and any partial results.[5]

- Structured error context enables coordinator recovery: failure type, attempted query, partial results, and alternative approaches. Coverage annotations in synthesis output indicate which topic areas are well-supported and which have gaps; without annotations, synthesis output appears uniformly confident when it is not.[5]

- The DataWithProvenance pattern preserves claim-source mappings through synthesis steps; provenance must be captured at retrieval time and cannot be reconstructed after synthesis. Conflicting statistics should be annotated with source attribution and collection dates rather than averaged or arbitrarily selected; publication dates distinguish temporal differences from genuine contradictions.[6,7]

- Aggregate accuracy metrics mask per-category failures. Stratified accuracy tracking per document type and per field reveals where failures actually occur, which is a prerequisite for safe automation decisions.[6,8]

The system that knows when not to act, and that reports its failures honestly, is the system that a human can actually trust.

SIX SCENARIO REFERENCE

The six exam scenarios are not isolated vignettes. They are recurring architectural contexts that appear across multiple questions and multiple domains. Each one tests a different cluster of trade-offs. Knowing the scenarios before you sit the exam means you are not spending cognitive load on “what kind of system is this” when the question is actually “which configuration choice is correct here.”

Scenario 1: Customer Support Agent

Primary domains: Domain 1 (Agentic Architecture), Domain 5 (Context Management and Reliability)

What it tests: Multi-tool orchestration, escalation logic, and the decision boundary between autonomous action and human handoff. This scenario focuses on when an agent should stop and route rather than continue acting. Tool selection under ambiguous user intent. Managing session state across a support conversation.

Chapters: 1, 11, 12

Scenario 2: Code Review Pipeline

Primary domains: Domain 3 (Configuration and Customization), Domain 1 (Agentic Architecture)

What it tests: CI/CD integration patterns, session isolation using the –bare flag and -p flag, and the risks of persistent state across review sessions. How CLAUDE.md configuration interacts with automated pipeline runs. Preventing context bleed between independent review jobs.

Chapters: 3, 6, 8

Scenario 3: Multi-Agent Research System

Primary domains: Domain 1 (Agentic Architecture), Domain 5 (Context Management and Reliability)

What it tests: Hub-and-spoke coordination, coordinator-to-subagent delegation, and the provenance problem: knowing which subagent produced which output and whether that output can be trusted. Handling subagent failure without corrupting the coordinator's result. Context partitioning across a multi-agent graph.

Chapters: 2, 11, 12

Scenario 4: Document Processing Pipeline

Primary domains: Domain 4 (Prompting and Output), Domain 2 (Tool Design)

What it tests: Structured output contracts using tool_use as a JSON schema enforcement mechanism. The validation loop pattern: generating output, validating against schema, retrying on failure. Designing tool descriptions that produce consistent extraction results across varied document formats.

Chapters: 4, 9, 10

Scenario 5: Enterprise Deployment

Primary domains: Domain 3 (Configuration and Customization), Domain 2 (Tool Design)

What it tests: The three-layer CLAUDE.md hierarchy: global, project, and folder-level configuration. Permission inheritance and override semantics. How slash commands and skills interact with organizational policy. The permissionMode settings and when bypassPermissions is appropriate versus dangerous. MCP server scoping in an enterprise context.

Chapters: 5, 6, 7

Scenario 6: Monitoring Dashboard Agent

Primary domains: Domain 5 (Context Management and Reliability), Domain 2 (Tool Design)

What it tests: Context window discipline in a long-running agent that continuously ingests telemetry. The lost-in-the-middle effect and its practical consequences for alert detection. Tool selection strategy when many tools are available and the agent must not hallucinate tool calls. Knowing when to summarize, when to truncate, and when to escalate because the context is too degraded to act reliably.

Chapters: 4, 11, 12

COVERAGE MAP

This map shows how the book's chapters align to the five CCAR-F exam domains and their task statements. Use it to identify which chapters to review when drilling a specific domain.

Domain 1: Agentic Architecture (27%)

Chapters: 1, 2, 3

Task statements covered:

- Identify the correct stop_reason and its implications for loop continuation or termination
- Distinguish between end_turn, tool_use, max_tokens, and pause_turn as termination signals
- Design a coordinator-subagent delegation pattern for a given task decomposition requirement
- Select the appropriate hook type (PreToolUse, PostToolUse, PreCompact, Notification) for a given lifecycle intervention
- Diagnose a runaway or prematurely terminating agentic loop given a scenario description
- Explain the difference between a system prompt hook and a UserPromptSubmit hook in terms of execution context

Domain 2: Tool Design (18%)

Chapters: 4, 5

Task statements covered:

- Write or evaluate a tool description for routing precision and ambiguity risk

- Identify why a model selected the wrong tool given a set of tool descriptions and a user prompt
- Distinguish between tool_choice modes: auto, any, and tool
- Select the appropriate built-in Claude Code tool for a given file or shell task
- Evaluate the scope and trust implications of a given MCP server configuration
- Apply incremental exploration principles to MCP tool design

Domain 3: Configuration and Customization (20%)

Chapters: 6, 7, 8

Task statements covered:

- Describe the three-layer CLAUDE.md hierarchy and explain inheritance and override semantics
- Identify which configuration layer should own a given policy or instruction
- Distinguish between a slash command and a skill in terms of invocation and scope
- Explain what plan mode changes about Claude's execution behavior and when to enable it
- Configure a CI/CD pipeline run using the –bare flag and -p flag correctly
- Identify session isolation risks in a multi-job pipeline and select the correct remediation
- Explain the permissionMode options and their appropriate use cases

Domain 4: Prompting and Output (20%)

Chapters: 9, 10

Task statements covered:

- Identify vague prompt instructions and rewrite them using explicit criteria
- Distinguish between prompting strategies that produce reliable structured output versus those that do not
- Describe the tool_use-as-JSON-contract pattern and explain why it enforces schema compliance
- Design a validation loop for structured output including retry logic and failure conditions
- Identify prompt patterns that increase hallucination risk in tool-calling contexts
- Select the appropriate output format for a given downstream consumer

Domain 5: Context Management and Reliability (15%)

Chapters: 11, 12

Task statements covered:

- Explain the lost-in-the-middle effect and its architectural consequences for long-running agents
- Select a context management strategy (summarization, truncation, partitioning) for a given scenario
- Identify the conditions that should trigger escalation rather than autonomous retry

- Describe the DataWithProvenance pattern and explain what it prevents
- Diagnose an error propagation failure in a multi-agent system given a scenario description
- Explain what context window exhaustion looks like from a stop_reason perspective and how to handle it

GLOSSARY

Terms are listed alphabetically. Each definition covers the concept as it applies to Claude architecture and the CCAR-F exam. Where a term has a specific technical meaning that differs from its colloquial use, the technical meaning is given.

–bare flag A Claude Code CLI flag that suppresses interactive UI elements and produces machine-readable output. Used in CI/CD contexts where Claude is invoked as a pipeline step rather than an interactive session.

-p flag The “print” flag in Claude Code CLI invocations. Enables non-interactive, single-turn execution. Combined with –bare for fully automated pipeline runs.

agentic loop The execution cycle in which Claude receives a message, produces a response that may include tool calls, executes tools, receives tool results, and continues until a terminal stop_reason is reached.

coordinator In a hub-and-spoke multi-agent system, the coordinator is the top-level Claude instance responsible for task decomposition, subagent delegation, and result aggregation. It does not perform the leaf-level work itself.

context window The total token budget available in a single Claude API call, encompassing the system prompt, conversation history, tool definitions, tool results, and the current response being generated.

DataWithProvenance An architectural *pattern* in which each piece of data produced by a subagent is wrapped with metadata identifying its source, the subagent that produced it, and the conditions under which it was generated. Prevents silent data corruption in multi-agent pipelines.

end_turn A stop_reason value indicating that Claude has completed its response and does not require further tool execution. The loop terminates normally.

escalation trigger A condition or threshold that causes an agent to stop autonomous action and route the task to a human or a higher-authority system. Defined explicitly in system prompts or hooks.

CLAUDE.md hierarchy The three-layer configuration system in Claude Code: global (user-level, applies to all projects), project (repository root, applies to all sessions in the project), and folder (subdirectory-level, applies when Claude operates within that folder). Lower layers can override or extend upper layers.

hook A mechanism for injecting behavior at specific points in the Claude Code agentic loop lifecycle. Hooks are not model instructions; they are executable scripts that run in response to lifecycle events.

hub-and-spoke A multi-agent architecture pattern in which a central coordinator delegates subtasks to specialized subagents. The coordinator aggregates results. Subagents do not communicate with each other directly.

JSON schema A declarative specification of the structure, types, and constraints of a JSON object. Used in tool definitions to specify input parameters and in structured output patterns to enforce response format.

lost-in-the-middle effect The empirical tendency for language models to underweight information that appears in the middle of a long context window relative to information at the beginning or end. A known reliability risk in long-running agents with large histories.

max_tokens A stop_reason value indicating that Claude's response was cut off because it reached the token limit set by the caller. Requires explicit handling; the response is incomplete.

MCP (Model Context Protocol) An open protocol for connecting Claude to external tool servers. MCP servers expose tools, resources, and prompts over a standardized interface. Claude Code supports local and remote MCP servers.

pause_turn A stop_reason value indicating that the agentic loop has paused and is waiting for external input or a condition to be resolved before continuing. Distinct from end_turn in that the loop is not complete.

permissionMode A Claude Code configuration setting that controls how the agent handles file and shell operations. Values include default (requires confirmation for destructive actions), acceptEdits (auto-approves file edits), and bypassPermissions (skips all permission checks, intended for trusted CI environments).

plan mode A Claude Code execution mode in which Claude generates a plan for a task without executing any actions. The user reviews and approves the plan before execution begins. Reduces risk of irreversible autonomous actions.

PreCompact hook A lifecycle hook that fires before Claude Code compresses the conversation history to free context window space. Used to inject summary instructions or preserve specific content before compaction.

PreToolUse hook A lifecycle hook that fires before Claude executes a tool call. Can inspect, modify, or block the tool call. The primary mechanism for implementing tool-level policy enforcement.

PostToolUse hook A lifecycle hook that fires after a tool call completes and before the result is returned to Claude. Can transform or annotate tool results.

Notification hook A lifecycle hook that fires when Claude Code emits a notification event, such as a long-running task status update. Used for routing alerts to external systems.

session isolation The property of a pipeline or CI/CD configuration in which each Claude invocation runs with a clean context and no shared state from previous runs. Prevents cross-job context bleed.

skill A reusable, named instruction set stored in a CLAUDE.md file or configuration that can be invoked by name. Skills encapsulate multi-step procedures. Distinct from slash commands in scope and invocation mechanism.

slash command A Claude Code invocation pattern using a leading slash, such as /review or /test. Slash commands trigger predefined behaviors or skill sequences. They are explicit intent signals, not natural language prompts.

stop_reason The field in a Claude API response that indicates why the model stopped generating. Values include end_turn, tool_use, max_tokens, and pause_turn. Correct handling of each stop_reason is required for a robust agentic loop.

structured output Output from Claude that conforms to a predefined schema, typically JSON. Achieved most reliably through the tool_use mechanism, which enforces schema compliance at the API level.

subagent In a hub-and-spoke architecture, a Claude instance that receives a delegated subtask from the coordinator, executes it, and returns a result. Subagents typically have narrower context and more specific tool access than the coordinator.

tool_choice An API parameter that controls how Claude selects tools. Values: auto (model decides whether to use a tool), any (model must use one of the available tools), tool (model must use a specific named tool).

tool_use A stop_reason value indicating that Claude has generated a tool call and is waiting for the result before continuing. Also used to refer to the content block type that represents a tool call in the API response.

UserPromptSubmit hook A Claude Code lifecycle hook that fires when the user submits a prompt, before Claude processes it. Used to inject context, timestamps, or policy checks at the start of each turn.

validation loop An architectural pattern in which Claude generates structured output, an external validator checks it against a schema, and the result is fed back to Claude for correction if validation fails. Continues until output passes or a retry limit is reached.

APPENDIX G: ENDNOTES

Superscript numbers in square brackets in the body text refer to the per-chapter endnotes below.

Chapter 1: The Agentic Loop

1. Agentic Architecture & Orchestration, Exam Domain 1, claudecertifications.com/claude-certified-architect/domains/agentic-architecture, accessed Q2 2026

2. Handling stop reasons, platform.claude.com/docs/en/build-with-claude/handling-stop-reasons, accessed Q2 2026

3. How the agent loop works, code.claude.com/docs/en/agent-sdk/agent-loop, accessed Q2 2026

4. How tool use works, platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works, accessed Q2 2026

Chapter 2: Hub-and-Spoke

1. Claude Certified Architect Exam Scenarios: Deep Dive Walkthroughs for All 6 Scenarios. claudecertifications.com/claude-certified-architect/scenarios. Published 2026-03-26. Scenario 3: Multi-Agent Research System.

2. Agentic Architecture & Orchestration: Claude Certified Architect Exam Domain 1. claudecertifications.com/claude-certified-architect/domains/agentic-architecture. Published 2026-03-26. Sections: Multi-Agent Orchestration; Exam Tips for Domain 1. Also: Claude Certified Architect Foundations Certification Exam Guide. Task Statements 1.2 and 1.3.

3. Subagents in the SDK. code.claude.com/docs/en/agent-sdk/subagents. Sections: Overview; Benefits of using subagents; Creating subagents;

AgentDefinition configuration; What subagents inherit; Parallelization; Tool restrictions; Detecting subagent invocation.

Chapter 3: Hooks and the Loop Lifecycle

1. Claude Certified Architect Exam Guide, Task Statement 1.4: “The difference between programmatic enforcement (hooks, prerequisite gates) and prompt-based guidance for workflow ordering. When deterministic compliance is required (e.g., identity verification before financial operations), prompt instructions alone have a non-zero failure rate.” Anthropic, PBC.

2. Agentic Architecture & Orchestration – Claude Certified Architect Exam Domain 1, claudecertifications.com. Section: “Hooks & Programmatic Enforcement.” The $500 refund limit code example and the anti-pattern commentary appear in this section verbatim. Also: Claude Certified Architect Exam Scenarios, Scenario 1 (Customer Support Resolution Agent): “PostToolUse hook that programmatically blocks refund tool calls above $500 and escalates” is the correct answer; “Adding ‘never process refunds above $500’ to the system prompt” is the anti-pattern. claudecertifications.com.

3. Intercept and control agent behavior with hooks – Claude Code Docs, code.claude.com. Section: “Configure hooks.” The hooks field structure, event name keys, matcher arrays, and callback registration pattern are documented here.

4. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Matchers.” The matcher field is described as “a regex string that matches against a different value depending on the hook event type.” MCP tool name pattern mcp__<server>__<action> is noted in the matcher description for tool-based hooks.

5. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Available hooks.” The full table lists Python SDK and TypeScript SDK availability for each event. TypeScript-only events: PostToolBatch, SessionStart, SessionEnd, TeammateIdle, TaskCompleted, ConfigChange, WorktreeCreate, WorktreeRemove, Setup.

6. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Outputs.” The distinction between additionalContext (appends to tool result) and updatedToolOutput (replaces tool output before Claude sees it) is documented under PostToolUse hook-specific outputs. Also: Claude Certified Architect Exam Guide, Task Statement 1.5.

7. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Available hooks.” SubagentStart fires on “Subagent initialization” with use case “Track parallel task spawning.” SubagentStop fires on “Subagent completion” with use case “Aggregate results from parallel tasks.”

8. How the agent loop works – Claude Code Docs, code.claude.com. Section: “Automatic compaction.” The PreCompact hook is described as firing “before compaction occurs” and receiving “a trigger field (manual or auto).” Also: Intercept and control agent behavior with hooks, section “Available hooks.”

9. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Session hooks not available in Python.” SessionStart and SessionEnd are available as SDK callback hooks in TypeScript but are only available as shell command hooks in the Python SDK.

10. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Outputs.” permissionDecision values "allow", "deny", "ask" documented for both SDKs. "defer" noted as TypeScript-only.

11. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Modify tool input” example. The sandbox path rewrite example demonstrates updatedInput with permissionDecision: "allow". The constraint “you must also include permissionDecision: 'allow'” and “always return a new object rather than mutating the original tool_input” are from this section.

12. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Add context and block a tool” example. The systemMessage field is described: “injects a reminder into the conversation so the agent receives context about why the operation was blocked and avoids retrying it.”

13. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Outputs.” The continue (TypeScript) / continue_ (Python) field “determines whether the agent keeps running after this hook.”

14. Claude Certified Architect Exam Guide, Task Statement 1.5: “Implementing PostToolUse hooks to normalize heterogeneous data formats (Unix timestamps, ISO 8601, numeric status codes) from different MCP tools before the agent processes them.”

15. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Register multiple hooks.” “When an event fires, all matching hooks run in parallel… Because completion order is non-deterministic, write each hook to act independently rather than relying on another hook having run first.”

16. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Outputs” (priority rules paragraph). “When multiple hooks or permission rules apply, deny takes priority over defer, which takes priority over ask, which takes priority over allow. If any hook returns deny, the operation is blocked regardless of other hooks.”

17. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Asynchronous output.” The async: true return pattern and the constraint “Async outputs cannot block, modify, or inject context into the operation since the agent has already moved on” are documented here.

18. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Matchers.” MCP tools follow the naming pattern with mcp__ prefix. The regex matcher example "mcp__" for targeting all MCP tools is from the “Filter with regex matchers” example.

19. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Subagent permission prompts multiplying.” “Subagents do not automatically inherit parent agent permissions.”

20. Intercept and control agent behavior with hooks – Claude Code Docs. Section: “Hook not firing.” “Verify the hook event name is correct and case-sensitive (PreToolUse, not preToolUse).”

Chapter 4: Designing Tools That Get Selected

1. Claude Certified Architect Exam Guide (Anthropic, Confidential NTK). Task Statement 2.1: tool descriptions as primary routing mechanism; ambiguous overlapping descriptions causing misrouting; system prompt keyword sensitivity. Sample Questions: Q2 correct answer B with explanation. Scenario 1: get_customer vs lookup_order in Customer Support agent. Task Statement 2.3: tool_choice forced selection pattern.

2. Define tools — Claude API Docs. platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools. Tool definition parameters (name, description, input_schema, input_examples); name regex ^[a-zA-Z0-9_-]{1,64}$; how API constructs system prompt from tool definitions; tool_choice values and behaviors; prefill behavior for "any" and forced tool; input_examples field mechanics; extended thinking constraints on tool_choice.

3. Tool Design & MCP Integration — Claude Certified Architect Exam Domain 2. claudecertifications.com/claude-certified-architect/domains/tool-design-mcp. 4-5 tools per agent as optimal; 18-tool overload pattern and coordinator-plus-subagent solution; tool_choice options; lookup_customer example description (good vs poor contrast).

Chapter 5: MCP and the Built-In Toolkit

1. Claude Certified Architect Exam Scenarios – Deep Dive Walkthroughs for All 6 Scenarios. claudecertifications.com. Published 2026-03-26. “Developer Productivity with Claude” scenario: 18-tool overload, Bash('cat config.json') anti-pattern, correct approach using Read tool, .mcp.json with environment variable expansion, Edit vs Write distinction.

2. Connect to external tools with MCP – Claude Code Docs. code.claude.com/docs/en/agent-sdk/mcp. MCP server configuration (in-code and .mcp.json); transport types (stdio, HTTP/SSE, "streamable-http" alias); tool naming pattern mcp__<server-name>__<tool-name>; allowedTools with wildcards; permissionMode: "acceptEdits" does not auto-approve MCP tools; permissionMode: "bypassPermissions" does but disables all safety prompts; environment variable expansion via ${ENV_VAR}; MCP tool search; error handling and troubleshooting.

3. Tool Design & MCP Integration – Claude Certified Architect Exam Domain 2. claudecertifications.com. Published 2026-03-26. Built-in tool selection rules (Grep, Glob, Read, Write, Edit, Bash with correct vs wrong examples); MCP configuration with .mcp.json vs ~/.claude.json; environment variable expansion for secrets; choosing community MCP servers over custom implementations for standard integrations.

4. Claude Certified Architect – Foundations Certification Exam Guide. Anthropic, PBC. Confidential Need to Know. Task Statement 2.4: MCP server scoping (project-level .mcp.json vs user-level ~/.claude.json);

environment variable expansion; MCP resources as content catalogs. Task Statement 2.5: Grep for content search, Glob for path patterns, Read/Write for full file ops, Edit for targeted modification; Read+Write fallback when Edit anchor text is non-unique; incremental exploration pattern (Grep entry points, Read to trace); tracing function usage across wrapper modules.

5. How the agent loop works – Claude Code Docs. code.claude.com/docs/en/agent-sdk/agent-loop. Built-in tool table; parallel tool execution: read-only tools (Read, Glob, Grep) can run concurrently; stateful tools (Edit, Write, Bash) run sequentially; MCP tool search reduces context cost for large tool sets.

Chapter 6: The CLAUDE.md Hierarchy

1. Claude Certified Architect Exam Scenarios – Deep Dive Walkthroughs for All 6 Scenarios, Scenario 2: Code Generation with Claude Code. claudecertifications.com. Published 2026-03-26. “Where should team coding standards go? .claude/CLAUDE.md (project-level, version-controlled, shared with team). Anti-pattern: ~/.claude/CLAUDE.md (user-level, personal only).”

2. How Claude remembers your project – Claude Code Docs. code.claude.com/docs/en/memory. Source URL: https://code.claude.com/docs/en/memory. Covers: four CLAUDE.md scopes and locations; load order (directory tree walk, root-to-cwd); CLAUDE.local.md ordering; subdirectory loading behavior; @path/to/import syntax (relative and absolute; recursive up to five hops; loads at launch); .claude/rules/ directory; path-specific rules with YAML paths frontmatter; size guidance (target under 200 lines); CLAUDE.md delivered as user message not system prompt; /memory command; claudeMdExcludes setting; compaction behavior for nested files.

3. Claude Code Configuration & Workflows – Claude Certified Architect Exam Domain 3 Claude Certifications. claudecertifications.com. Published 2026-03-26. Source URL: https://claudecertifications.com/claude-certified-architect/domains/claude-code-config. Covers: CLAUDE.md hierarchy (user, project, directory layers); @import syntax; .claude/rules/ directory; path-specific rules with glob patterns; anti-pattern of monolithic CLAUDE.md.

4. Claude Certified Architect – Foundations Certification Exam Guide. Anthropic, PBC. Task Statement 3.1: Configure CLAUDE.md files with appropriate hierarchy, scoping, and modular organization. Covers: configuration hierarchy (user-level, project-level, directory-level); user-level settings not shared with teammates; @import syntax; .claude/rules/ directory; /memory command for verifying loaded files; diagnosing new team member not receiving instructions due to user-level vs project-level placement. Task Statement 3.3: path-specific rules; glob-pattern advantage over subdirectory CLAUDE.md for file-type-spanning conventions.

Chapter 7: Slash Commands, Skills, and Plan Mode

1. Claude Certifications. “Claude Code Configuration & Workflows – Claude Certified Architect Exam Domain 3.” claudecertifications.com, 2026. Section: Custom Commands & Skills. Custom slash commands stored in .claude/commands/ (project-scoped) or ~/.claude/commands/ (user-scoped); filename minus .md extension becomes command name.

2. Claude Code documentation. “Slash Commands in the SDK.” code.claude.com/docs/en/agent-sdk/slash-commands. Commands dispatched as prompt strings; system/init lists available commands in the session. .claude/commands/ is the legacy format; .claude/skills/<name>/SKILL.md is the recommended current format

supporting both slash-command invocation and autonomous model invocation.

3. Claude Code documentation. “Agent Skills in the SDK.” code.claude.com/docs/en/agent-sdk/skills. Skills discovered from .claude/skills/ and ~/.claude/skills/; model autonomously invokes based on description; skills option on query() controls which skills are available; allowed-tools frontmatter applies to CLI only, not SDK.

4. Anthropic. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 3.2. Project-scoped commands in .claude/commands/; skills in .claude/skills/ with SKILL.md frontmatter fields (context: fork, allowed-tools, argument-hint); context: fork prevents skill exploration from polluting main conversation; allowed-tools restricts tool access; choosing skills (on-demand) vs CLAUDE.md (always-loaded).

5. Claude Certifications. “Claude Certified Architect Exam Scenarios – Deep Dive Walkthroughs for All 6 Scenarios.” claudecertifications.com, 2026. Scenario 2: Code Generation with Claude Code. Use skill with context: fork for complex refactoring isolation; TDD iteration as best refinement strategy; plan mode for multi-file architectural changes; direct execution for simple well-defined fixes.

6. Anthropic. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 3.4. Plan mode for tasks with architectural implications (microservice restructuring, library migrations affecting 45+ files, choosing between integration approaches); direct execution for well-understood changes (single-file bug fix, adding a date validation conditional); Explore subagent for verbose discovery phases; combining plan mode for investigation with direct execution for implementation.

7. Anthropic. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 3.5. Concrete input/output examples as most

effective technique for inconsistent transformations; 2-3 examples; TDD iteration cycle; interview pattern for unfamiliar domains to surface design considerations; interacting issues in single message; independent issues in sequential iteration.

Chapter 8: Claude Code in CI/CD

1. “Run Claude Code programmatically.” Claude Code Docs. https://code.claude.com/docs/en/headless. Describes the -p/--print flag, --bare mode, --output-format options including json and stream-json, and the --json-schema combination. Notes that all CLI options work with -p and that --bare skips discovery of hooks, skills, MCP servers, and CLAUDE.md.

2. “Claude Code Configuration and Workflows: Claude Certified Architect Exam Domain 3.” Claude Certifications. https://claudecertifications.com/claude-certified-architect/domains/claude-code-config. Published 2026-03-26. Source for: -p flag, --output-format json, --json-schema, session context isolation for code review, anti-pattern of same-session self-review, Message Batches API 50% savings and 24-hour window, custom_id for batch tracking.

3. “Claude Certified Architect Exam Scenarios: Deep Dive Walkthroughs for All 6 Scenarios.” Claude Certifications. https://claudecertifications.com/claude-certified-architect/scenarios. Published 2026-03-26. Source for: Scenario 5 (Claude Code for CI/CD) walkthrough; three facts to memorize; anti-pattern of same-session self-review; separate session for review; batch API for non-urgent tasks.

4. “Claude Code GitHub Actions.” Claude Code Docs. https://code.claude.com/docs/en/github-actions. Source for: claude_args parameter, --max-turns via claude_args, ANTHROPIC_API_KEY via ${{ secrets.ANTHROPIC_API_KEY }}, CLAUDE.md for behavior configuration, separate session for review.

5. “Claude Certified Architect Guide.” Anthropic, PBC. Confidential Need to Know. Source for: Task Statement 3.6 (-p flag, --output-format json, --json-schema, CLAUDE.md as project context, session context isolation, prior review findings, fixture documentation); Task Statement 4.5 (Message Batches API: 50% cost savings, 24-hour window, no guaranteed latency SLA, no multi-turn tool calling, custom_id correlation, batch vs synchronous decision rule, failure resubmission by custom_id); Task Statement 4.6 (self-review limitations, independent review instances); Question 10 (correct answer: -p flag); Question 11 (correct answer: batch for overnight reports, synchronous for pre-merge checks); permissionMode: "bypassPermissions" as recommended for CI/containers.

Chapter 9: Prompting for Precision

1. Claude Certified Architect Exam Guide (Claude+Certified+Architect+Guide.md): Task Statement 4.1 (explicit criteria over vague instructions; false positive impact on developer trust; severity criteria with code examples; temporarily disabling high-false-positive categories); Task Statement 4.2 (2-4 targeted few-shot examples; reasoning for ambiguous-case choices; output format consistency; acceptable vs genuine-issue examples). Tier 2 source.

2. Prompt Engineering and Structured Output, Claude Certified Architect Exam Domain 4 (claudecertifications.com): explicit criteria vs vague instructions; false positive / alert fatigue mechanism; 2-4 examples optimal range; anti-pattern of >6 examples bloating the prompt without value; few-shot most valuable for ambiguous boundaries; vague prompt vs explicit prompt code comparison. Tier 1 source.

3. Claude Certified Architect Exam Scenarios, Deep Dive Walkthroughs (claudecertifications.com): Scenario 5 (Claude Code for CI/CD): “design

prompts that provide actionable feedback and minimize false positives." Tier 1 source.

4. Claude Certified Architect Exam Guide (Claude+Certified+Architect+Guide.md): Task Statement 3.5, addressing interacting versus independent issues: provide all issues in a single detailed message when fixes interact; use sequential iteration for independent issues. Tier 2 source.

Chapter 10: Structured Output and the Validation Loop

1. Claude Certified Architect Exam Scenarios: Deep Dive Walkthroughs for All 6 Scenarios. Scenario 6: Structured Data Extraction. claudecertifications.com. Published 2026-03-26. Source URL: https://claudecertifications.com/claude-certified-architect/scenarios

2. Structured outputs, Claude API Docs. platform.claude.com. Source URL: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

3. Define tools, Claude API Docs. platform.claude.com. Source URL: https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools

4. Claude Certified Architect Exam Guide, Domain 4: Prompt Engineering & Structured Output, Task Statements 4.3 and 4.4. Anthropic, PBC. Confidential Need to Know.

Chapter 11: Context Window Discipline

1. Context Management & Reliability – Claude Certified Architect Exam Domain 5. claudecertifications.com. Published 2026-03-26. https://claudecertifications.com/claude-certified-

architect/domains/context-management. Progressive summarization example (John Smith, ACC-12345, order #98765); lost-in-the-middle effect definition; case facts block structure and immutability requirement; context degradation symptoms; scratchpad files, /compact, and subagent delegation as mitigations; crash recovery manifests.

2. Claude Certified Architect – Foundations Certification Exam Guide. Anthropic, PBC. Task Statement 5.1: progressive summarization risks; lost-in-the-middle effect (models reliably process beginning and end, may omit middle); tool output accumulation and trimming to relevant fields; case facts block pattern; position-aware ordering; scratchpad files; subagent delegation; crash recovery manifests (Task Statement 5.4). Task Statement 5.4: context degradation symptoms (inconsistent answers, “typical patterns” substitution).

3. How the agent loop works – Claude Code Docs. code.claude.com. What consumes context in the SDK (system prompt, CLAUDE.md via settingSources, tool definitions, conversation history, tool inputs and outputs); prompt caching for static content; automatic compaction; compact_boundary event (SystemMessage in Python, SDKCompactBoundaryMessage in TypeScript); CLAUDE.md content re-injected on every request; summarization instructions in CLAUDE.md; PreCompact hook; /compact as prompt string; subagent context isolation (only final response returns to parent); MCP server context cost (all tool schemas per request); ToolSearch for on-demand tool loading.

4. Context windows – Claude API Docs. platform.claude.com. Context window sizes: Claude Opus 4.7, Claude Opus 4.6, Claude Sonnet 4.6 have 1M-token windows; Claude Sonnet 4.5 and other models have 200K-token windows. Context awareness feature (Claude Sonnet 4.6, Sonnet 4.5, Haiku 4.5): models receive token budget information after each tool call. Context rot

phenomenon: accuracy and recall degrade as token count grows, making curation of context as important as available space.

5. Intercept and control agent behavior with hooks – Claude Code Docs (referenced via agent loop doc). PreCompact hook fires before compaction; receives trigger field (manual or auto). Full hook mechanics assigned to Chapter 3.

Chapter 12: Escalation, Provenance, and Error Propagation

1. Claude Certifications. “Claude Certified Architect Exam Scenarios – Deep Dive Walkthroughs for All 6 Scenarios.” claudecertifications.com. Published 2026-03-26. Customer Support Resolution Agent section: escalation criteria, hook-based enforcement, anti-patterns.

2. Anthropic, PBC. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 5.2: Design effective escalation and ambiguity resolution patterns. Valid escalation triggers, sentiment-based escalation as anti-pattern, multiple customer matches, honoring explicit requests.

3. Claude Certifications. “Context Management & Reliability – Claude Certified Architect Exam Domain 5.” claudecertifications.com. Published 2026-03-26. Escalation triggers section: valid escalation triggers, invalid triggers, should_escalate code example.

4. Anthropic, PBC. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 5.2: structured handoff summaries (customer ID, root cause, refund amount, recommended action) for escalation to human agents lacking transcript access. Structured handoff summaries are part of the escalation and error propagation patterns in Domain 5.

5. Anthropic, PBC. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 5.3: structured error context, access failure vs valid empty result distinction, local recovery before coordinator escalation, coverage annotations. Also Task Statement 2.2: isError flag, errorCategory, isRetryable, distinguishing transient/validation/permission/business errors.

6. Claude Certifications. “Context Management & Reliability – Claude Certified Architect Exam Domain 5.” claudecertifications.com. Published 2026-03-26. DataWithProvenance dataclass definition, resolve_conflict function, stratified accuracy metrics code example, aggregate-vs-per-category accuracy illustration.

7. Anthropic, PBC. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 5.6: claim-source mappings, conflicting statistics handling, publication/collection dates for temporal disambiguation, structured outputs preserving provenance through synthesis.

8. Anthropic, PBC. “Claude Certified Architect – Foundations Certification Exam Guide.” Task Statement 5.5: stratified random sampling, accuracy by document type and field, validating accuracy before automating high-confidence extractions.


CLAUDE
CERTIFIED
ARCHITECT
GUIDE
Written by Claude Code
Instructed by V.Korostyshevskiy
vladimir.org